

ĐỀ CƯƠNG

Lập trình

hướng đối tượng

Trường Đại học Công nghệ thông tin

Có lý thuyết, bài tập và đề thi mẫu.
Bản 2025.

MỤC LỤC

MỤC LỤC	3
A. CÁC KHÁI NIỆM CƠ SỞ	7
I. TỔNG QUAN.	7
1. Các phương pháp lập trình.	7
a. Lập trình không có cấu trúc (Non-structured programming).	7
b. Lập trình hướng cấu trúc (lập trình thủ tục, hướng chức năng, Structure oriented programming, SOP).	7
c. Lập trình hướng đối tượng (Object oriented programming, OOP).	7
II. TỔNG QUAN VỀ CÁC THÀNH PHẦN CƠ BẢN VÀ CÁC TÍNH CHẤT CỦA OOP.	9
1. Tổng quan các thành phần cơ bản của OOP.	9
a. Đối tượng (Object).	9
b. Lớp (Class).	9
c. Thuộc tính (Attribute).	9
d. Hành vi (Operation) và Phương thức (Method).	9
2. Quan hệ giữa các lớp.	9
3. Sơ đồ lớp (Class diagram) và Thiết kế lớp.	10
4. Tổng quan về các tính chất của OOP.	11
a. Tính đóng gói (Encapsulation).	11
b. Tính kế thừa (Inheritance).	11
c. Tính trừu tượng (Abstraction).	11
d. Tính đa hình (Polymorphism).	11
B. CÚ PHÁP CỤ THỂ CỦA OOP C++	12
I. VẬN DỤNG OOP CƠ BẢN.	12
1. Các thành phần chính của OOP.	12
a. Khai báo.	12
b. Truy xuất.	14
c. Gán giá trị cho thuộc tính.	14

d. Xây dựng header.	16
2. Phạm vi truy cập (Access modifier / Access control label).	17
3. Hàm bạn (Friend function) và Lớp bạn (Friend class).	18
a. Hàm bạn.	18
b. Lớp bạn.....	19
4. Phương thức khởi tạo (Constructor) và bài toán gán giá trị cho các thuộc tính.	20
a. Vị trí & ý nghĩa của constructor.	20
b. Tham số của constructor và phân loại constructor.	20
c. Initializer.....	21
5. Phương thức hủy (Destructor) và bài toán tiết kiệm bộ nhớ.	23
6. Thành phần tĩnh (Static member).	25
7. Nạp chồng phương thức (Method overloading).	26
8. Nạp chồng toán tử (Operator overloading) và ép kiểu.	27
a. Bản chất hàm của toán tử.	27
b. Cú pháp nạp chồng toán tử.	27
c. Ép kiểu bằng constructor và Nạp chồng toán tử ép kiểu.	29
d. Nạp chồng toán tử gán (=).	34
f. Nạp chồng toán tử ++ và --.	35
e. Xử lý luồng nhập - xuất, Nạp chồng toán tử >> và <<.	36
II. VẬN DỤNG CÁC TÍNH CHẤT CỦA OPP.	38
1. Tính đóng gói (Encapsulation).	38
a. Tính đóng gói.....	38
b. Setter và getter.....	38
2. Tính kế thừa (Inheritance).	39
a. Quan hệ giữa các lớp đối tượng.....	39
b. Tính kế thừa.	39
c. Lớp con (Subclass) và lớp cha (Parent class).	40
d. Phạm vi kế thừa.	41
e. Kế thừa constructor.	42
f. Kế thừa destructor.	42
g. Con trỏ trong kế thừa, Ép kiểu con về cha.....	42

h. Đa kế thừa (Multiple inheritance), Vấn đề hình thoi (Diamond problem) và Kế thừa ảo (Virtual inheritance).	43
3. Tính đa hình (Polymorphism) và Tính trừu tượng (Abstraction).	45
a. Tính đa hình.	45
b. Phương thức ảo (Virtual method), Liên kết tĩnh (Static binding) và Liên kết động (Dynamic binding).	45
c. Tính trừu tượng, Phương thức thuần ảo (Pure virtual method), Lớp cơ sở trừu tượng (Abstract class).	46
MỘT SỐ THUẬT TOÁN THÊM	48
I. LỚP PHÂN SỐ (cPhanSo).	48
1. Kiểm tra phân số.	48
2. Xuất phân số.	49
3. Rút gọn phân số.	49
4. Quy đồng phân số.	49
II. LỚP ĐIỂM VÀ ĐA GIÁC (cDiem, cDaGiac).	50
1. Các phép toán với điểm.	50
2. Các phép toán với đa giác có n đỉnh.	50
IV. LỚP GIỜ-PHÚT-GIÂY (cTime).	51
V. LỚP NGÀY-THÁNG-NĂM (cDate).	52
1. Kiểm tra năm nhuận.	52
2. Kiểm tra ngày-tháng-năm hợp lệ.	52
3. Kiểm tra thứ trong tuần.	53
4. Thêm ngày-tháng-năm theo số ngày.	54
4. Bớt ngày-tháng-năm theo số ngày.	55
6. Tìm khoảng cách giữa 2 ngày-tháng-năm.....	56
VI. LỚP ĐƠN THỨC VÀ ĐA THỨC (cDonThuc, cDaThuc).	57
1. Kiểm tra đa thức.	57
2. Xuất đa thức dạng chuẩn.....	58
3. Chuẩn hóa đa thức để tính toán.	60
4. Cộng, trừ hai đa thức.....	61
LUYỆN TẬP KẾT THÚC MÔN	63

I. Câu hỏi lý thuyết.....	64
1. Giới thiệu OOP.....	64
2. Phương thức khởi tạo và phương thức hủy.....	66
3. Các tính chất của OOP.....	67
II. Hoàn thiện cài đặt lớp đơn giản.....	71
Câu 2 [HK1 22-23].....	71
Câu 2 [HK2 23-24].....	73
Câu 2 [HK2 22-23].....	74
Câu 2 [HK2 21-22].....	75
III. Thiết kế và cài đặt lớp phức tạp qua đa hình và kế thừa.....	76
Câu 3 [HK1 24-25].....	77
Câu 3 [HK2 23-24].....	80
Câu 3 [HK2 22-23].....	90
Câu 3 [HK2 18-19].....	97
Câu 3 [HK2 16-17].....	106
Câu 3 [HK2 15-16].....	113
Câu 3 [HK2 17-18].....	118
Câu 3 [HK2 14-15].....	121
Câu 3 [HK1 22-23].....	123
Câu 3 [HK2 19-20].....	130

A. CÁC KHÁI NIỆM CƠ SỞ

I. TỔNG QUAN.

1. Các phương pháp lập trình.

a. Lập trình không có cấu trúc (Non-structured programming).

- Các ngôn ngữ như Assembly, Basic.
- Sử dụng các biến toàn cục.
- Lạm dụng lệnh GOTO.

Nhược điểm:

- Dài dòng, khó hiểu.
- Gần như không thể tái sử dụng.

→ Chỉ đáp ứng được một số nhu cầu lập trình đơn giản.

b. Lập trình hướng cấu trúc (lập trình thủ tục, hướng chức năng, Structure oriented programming, SOP).

- Tổ chức phân chia chương trình thành các chương trình con (hàm/thủ tục).
- Mỗi chương trình con đảm nhận xử lý một công việc nhỏ hay một nhóm công việc trong toàn bộ hệ thống.
- Việc trao đổi dữ liệu giữa các chương trình con được thực hiện thông qua các đối số của hàm và các biến toàn cục.

Ưu điểm:

- Dễ hiểu, dễ bảo trì vì chương trình được module hóa (phân tách chương trình thành nhiều phần).
- Dễ dàng tái sử dụng.

Nhược điểm:

- Chương trình dùng để xử lý dữ liệu nhưng lại tách rời dữ liệu.
→ Không mô tả được đầy đủ, trung thực hệ thống trong thực tế.
- Chương trình được xây dựng bởi các giải thuật, dựa trên cấu trúc dữ liệu (**Lấy cấu trúc dữ liệu làm nền tảng**) (Chương trình = Cấu trúc dữ liệu + Giải thuật).
→ Khi cấu trúc dữ liệu thay đổi thì giải thuật cũng thay đổi, đòi hỏi viết lại chương trình.

c. Lập trình hướng đối tượng (Object oriented programming, OOP).

- Được xây dựng trên nền tảng của SOP và sự trừu tượng hóa dữ liệu.

- Một chương trình hướng đối tượng được thiết kế xoay quanh dữ liệu mà chúng ta có thể làm việc trên đó (**Lấy đối tượng làm nền tảng**).
- Một chương trình OOP sẽ bao gồm tập các đối tượng có quan hệ với nhau.

Ưu điểm:

- **Tính tái sử dụng (Reusability):**
 - Thiết kế các thành phần có thể được sử dụng trong nhiều phần mềm khác nhau.
- **Tính mở rộng (Extensibility):**
 - Hỗ trợ các plug-in/add-n.
- **Tính mềm dẻo (Flexibility):**
 - Dễ dàng thay đổi khi thêm mới dữ liệu hay tính năng.
 - Các thay đổi không làm ảnh hưởng nhiều đến toàn bộ hệ thống.

II. TỔNG QUAN VỀ CÁC THÀNH PHẦN CƠ BẢN VÀ CÁC TÍNH CHẤT CỦA OOP.

1. Tổng quan các thành phần cơ bản của OOP.

a. Đối tượng (Object).

Là một thể hiện của lớp, bạn có thể hiểu đối tượng là một biến.

Mỗi đối tượng thuộc về 1 hoặc nhiều lớp. Lúc này, mỗi đối tượng được gọi là một thể hiện của lớp (Instance).

Mỗi đối tượng gồm có:

- Thuộc tính (Attribute).
- Hành vi / thao tác (Operation). Cài đặt hành vi được gọi là phương thức (Method).

b. Lớp (Class).

Là tập hợp các đối tượng có cùng bản chất (cùng thuộc tính và hành vi), hay là nơi định nghĩa thông tin về các đối tượng.

c. Thuộc tính (Attribute).

Là các đặc điểm của đối tượng, hay các biến (mức độ lập trình).

d. Hành vi (Operation) và Phương thức (Method).

Hành vi là hành động của đối tượng.

Phương thức là định nghĩa hành vi (mức độ lập trình), hay các hàm.

2. Quan hệ giữa các lớp.

- **Quan hệ kết hợp / tổng quát (Association):** Lớp B có mối quan hệ với lớp A.
- **Quan hệ thành phần (Composition):** Lớp B là thành phần của lớp A và không thể tách rời khỏi A.
VD: Xe máy - Động cơ (của xe máy).
- **Quan hệ bao hàm (Aggregation):** Lớp B là thành phần của lớp A và có thể tách rời khỏi A.
VD: Lớp học - Học sinh (của lớp học).
- **Quan hệ kế thừa (Inheritance):** Lớp B là một biểu hiện cụ thể của lớp A.
VD: Động vật – Con mèo (là một biểu hiện cụ thể của động vật).

3. Sơ đồ lớp (Class diagram) và Thiết kế lớp.

Sơ đồ lớp chính là sơ đồ mô tả của các lớp, mối quan hệ giữa các đối tượng trong hệ thống.

Ký hiệu mỗi lớp: Gồm có 3 ô:

- **Ô 1:** Tên lớp.
- **Ô 2:** Danh sách thuộc tính.
- **Ô 3:** Danh sách phương thức.

Ký hiệu mỗi thành phần: `access-modifier name(par...) : data-type`.

Phạm vi truy cập	Gồm có +, #, - ứng với public, protected, private
Tham số	<code>par : data-type</code>
Phương thức khởi tạo	<code>class-name(par...)</code>
Lớp bạn	friend class-name
Phương thức bạn	<code>access-modifier(par...) name : friend data-type</code>
Thuộc tính tĩnh	<code>attribute : static data-type</code>
Phương thức tĩnh	<code>access-modifier name(par...) : static data-type</code>
Phương thức ảo	<code>access-modifier name(par...) : virtual data-type</code>
Phương thức thuần ảo	<code>access-modifier name(par...) = 0 : virtual data-type</code>
Lớp B là thành phần không thể tách rời với lớp A	A —◆ B
Lớp B là thành phần có thể tách rời với lớp A	A —◇ B
Lớp B là lớp con của lớp A	A —▷ B

struct cũng có thể xuất hiện và được ký hiệu như class, nhưng chỉ có 2 ô là tên struct và danh sách thuộc tính (nên ký hiệu chữ struct ở cạnh để tránh nhầm lẫn).

Mẫu:

Tên lớp
-Thuộc tính 1 -Thuộc tính 2 -...
+Phương thức 1() +Phương thức 2() +...()

Ví dụ:

Student
-name: string -age: int -gender: string -gpa: double
+Student(name: string, age: int, gender: string, gpa: double) +display(): void

Quy tắc đặt tên:

- Tên lớp: **cDanhTu**.
- Tên thuộc tính: **iDanhTu**, trong đó **i** là tên viết tắt của kiểu dữ liệu.

- Tên phương thức: `dongTu`.

4. Tổng quan về các tính chất của OOP.

Có 4 tính chất:

a. Tính đóng gói (Encapsulation).

Là nhóm những gì có liên quan với nhau vào làm một để sau này có thể dùng một cái tên để gọi đến.

→ Dễ sử dụng.

→ Bảo vệ, che giấu các thành phần bên trong → Đảm bảo chương trình an toàn.

b. Tính kế thừa (Inheritance).

Kế thừa là thừa hưởng lại những thuộc tính và phương thức của một lớp khác. Có nghĩa là nếu lớp A kế thừa lớp B thì lớp A sẽ có những thuộc tính và phương thức của lớp B.

→ Tái sử dụng.

→ Dễ dàng xây dựng các lớp mới dựa trên một lớp đã có.

c. Tính trừu tượng (Abstraction).

Trừu tượng hóa là **cách nhìn khái quát hóa** về một tập các đối tượng có chung các đặc điểm được quan tâm (bỏ qua những chi tiết không cần thiết). Nó cho phép ta giao tiếp với các thành phần của đối tượng mà không cần phải biết về cách mà các thành phần này được xây dựng.

Trừu tượng hóa cũng có thể hiểu là cách chuyển một đối tượng trên thực tế thành các thành phần của OOP.

d. Tính đa hình (Polymorphism).

Tên một thao tác hoặc thuộc tính có thể được định nghĩa tại nhiều lớp và có thể có những cài đặt khác nhau tại các lớp đó.

Trong lập trình thì khi một đối tượng hay một phương thức có nhiều hơn một hình thái thì đó chính là đa hình. Tính đa hình được thể hiện dưới 3 hình thức:

- **Đa hình với nạp chồng phương thức:** Phương thức cộng sẽ có các hình thái là cộng 2 số nguyên, cộng 2 số thực, cộng 3 số nguyên, v/v.
- **Đa hình với ghi đè phương thức:** Các lớp khác nhau sở hữu 1 hoặc nhiều phương thức giống nhau.
- **Đa hình thông qua các đối tượng đa hình:** Phương thức `makeSound()` là âm thanh phát ra từ các loài vật. Mỗi loài lại có một âm thanh khác nhau.

B. CÚ PHÁP CỤ THỂ CỦA OOP C++

I. VẬN DỤNG OOP CƠ BẢN.

1. Các thành phần chính của OPP.

a. Khai báo.

Cú pháp khai báo:

```
class class {  
    // Khai báo danh sách các thuộc tính  
    data-type attribute;  
  
    // Khai báo danh sách các phương thức  
    data-type method(parameters) {  
        // nội dung  
    }  
};
```

- Thuộc tính của lớp có thể là đối tượng một lớp khác.
- Tham số của phương thức có thể là đối tượng của lớp chứa phương thức đó.
- Phương thức có truy xuất đến các thuộc tính (tùy vào phạm vi truy cập) của lớp mà không cần coi như thuộc tính đó như tham số.
- Giá trị của đối tượng kia có thể được gán thẳng cho đối tượng này bằng toán tử =, nếu 2 đối tượng đó chung lớp.

Chú ý: Không nên gán thẳng giá trị khi khai báo các thuộc tính (=). Giá trị của các thuộc tính này chỉ được cập nhật qua các phương thức.

Phương thức hằng và từ khóa const:

Từ khóa **const** được thêm vào phía sau nguyên mẫu hàm chỉ thị hàm này không được phép thay đổi dữ liệu các thuộc tính của lớp → Đảm bảo an toàn cho chương trình.

```
class Time {  
public:  
    void Set (int hours , int minutes , int seconds);  
    void Increment ( );  
    void Write ( ) const;  
    Time (int initHrs, int initMins, int initSecs );  
    Time ( );  
private:  
    int hrs;  
    int mins;  
    int secs;  
};
```

- Phương thức / đối tượng const chỉ có thể gọi phương thức const.
- Phương thức / đối tượng không const có thể gọi bất kỳ phương thức nào.

Khai báo phương thức bên ngoài lớp bằng cách sử dụng nguyên mẫu và toán tử phân giải phạm vi:

Ngoài cách khai báo phương thức bên trong lớp, có thể **khai báo phương thức bên ngoài lớp**:

- Cú pháp khai báo bên ngoài lớp:
data-type class :: method(parameters) {content}
- Trong lớp, phải có nguyên mẫu hàm (prototype):
data-type method(parameters);

Tham số của nguyên mẫu hàm không nhất thiết phải có tên, nhưng nhất thiết phải có kiểu dữ liệu.

VD:

```
class TamGiac{
private:
    int a,b,c;
public:
    TamGiac() {a = 1; b = 1; c = 1;}
    TamGiac(int a, int b, int c): a(a), b(b), c(c) {};

    bool kiểmTraTG(TamGiac tg);
};

bool TamGiac::kiểmTraTG() {
    if (a > 0 && b > 0 && c > 0
        && (a + b > c)
        && (a + c > b)
        && (b + c > a ))
        return true;
    return false;
};
```

Chú ý:

- Nếu lớp đơn giản → Nên khai báo phương thức bên trong lớp.
- Nếu lớp phức tạp → Nên khai báo phương thức bên ngoài lớp.

b. Truy xuất.

Cú pháp truy xuất:

- Gọi thuộc tính / phương thức (thành phần) của 1 tượng cụ thể:
 - Nếu đối tượng không phải một con trỏ → Dùng toán tử chấm (.): **object-name.member**.
 - Nếu đối tượng là một con trỏ → Dùng toán tử mũi tên (->): **object-name->member**.
- Thuộc tính và phương thức có thể được gọi ở trong class hoặc trong phương thức. Ở trong hàm main(), nó buộc phải được gọi kèm với tên đối tượng.

(Cú pháp này giống với struct).

VD:

- Có lớp:

```
class Rectangle {  
private:  
    int width, length;  
public:  
    void set (int w, int l); // Hàm này đã được khai báo bên  
    ngoài lớp  
};
```

- Gọi trong hàm main:

```
Rectangle r1; // Đối tượng không phải con trỏ  
r1.set(5, 8);  
  
Rectangle *r2; // Đối tượng là con trỏ  
r2 = &r1;  
r2->set(8,10);
```

c. Gán giá trị cho thuộc tính.

Các cách:

- Khởi tạo bằng cú pháp **cin >>** bên trong phương thức.
- Khởi tạo bằng toán tử **=** bên trong phương thức hoặc khởi tạo trực tiếp khi khai báo thuộc tính.
- Khởi tạo bằng hàm khai báo ngoài lớp.
- Khởi tạo bằng hàm trong lớp: **parameter(attribute)**.
- Khởi tạo bằng con trỏ **this** -> **attribute** bên trong phương thức để ám chỉ thuộc tính attribute của đối tượng.

Con trỏ this có 2 cách dùng cơ bản:

- 1. **this -> attribute = value;**
value là giá trị hoặc tham số nạp vào phương thức chứa con trỏ này.
- 2. **this -> method();**
method là phương thức kiểu void, chứa con trỏ this dạng 1.
- Khởi tạo bằng **tham số mặc định (Default agrument)**: Là gán giá trị ngay cho tham số của phương thức. Giá trị này được dùng làm giá trị mặc định cho thuộc tính.

Chú ý:

- Tham số mặc định phải được khai báo đằng sau các tham số khác.
- Hàm có toàn bộ tham số mặc định có thể được gọi với ít hơn tham số mặc định. Giá trị của biến sẽ được thay thế theo thứ tự.

Cú pháp: `method (data-type parameter1 = value)`.

VD. Tham số mặc định:

```
void Point(int a = 0, int a = 0){  
    x = a;  
    y = b;  
}
```

→

Cách gọi	Giá trị	
	x	y
Point();	0	0
Point(1);	1	0
Point(1, 1);	1	1

VD2: Gán giá trị qua hàm ngoài lớp:

```
class Rectangle {  
private:  
    int width, length;  
public:  
    void set (int w, int l);  
};  
  
void Rectangle :: set (int w, int l) {  
    width = w;  
    length = l;  
}
```

→ Khi gọi `myRectangle.set(1, 2);` thì `width` được gán là 1 và `length` được gán là 2.

d. Xây dựng header.

Header (.h) là file cung cấp phần định nghĩa lớp, có thể không chứa định nghĩa các hàm thành phần. Định nghĩa các hàm thành phần thường nằm ở một file .cpp riêng, .h chỉ chứa nguyên mẫu các hàm đó.

File A include file B tức là “sao chép” toàn bộ nội dung của file B vào file A. Khi định nghĩa lớp và định nghĩa hàm được tách bạch thành 2 file riêng biệt như trên, file .cpp phải include file .h chứa khai báo lớp chứa các hàm thành phần.

Cú pháp include: `#include "fileName.h"` hoặc `#include <fileName.h>` đều đúng. Nhưng thông thường, các lớp tự định nghĩa sẽ được include bằng `#include "fileName.h"`, và các lớp được cài đặt sẵn (built-in) bắt buộc phải được include bằng `#include <fileName.h>`.

Khi một phải được quá nhiều file khác include thì ở trên cùng file đó phải có chỉ thị `#pragma once`.

2. Phạm vi truy cập (Access modifier / Access control label).

Phạm vi truy cập là khái niệm để xác định xem một thuộc tính hay một phương thức có được truy xuất từ bên ngoài hay không. Trong C++ có 3 loại phạm vi truy cập là **private**, **protected**, **public**. Xem bảng minh họa dưới đây để có cái nhìn tổng quan về phạm vi truy cập:

Phạm vi	Ký hiệu trong sơ đồ lớp	Được truy cập trong class?	Được truy cập trong class con? (Kế thừa)	Được truy cập ngoài class?
private	-	Có	Không	Không
protected	#	Có	Có	Không
public	+	Có	Có	Có

Các thành phần có **protected** và **private** được gọi là các thành phần riêng tư của lớp.

Chú ý:

- Phạm vi truy cập được nhắc đến ngay trước thành phần của OPP.
- Nếu không khai báo phạm vi truy cập, **mặc định** là **private**.
- Các phương thức khai báo bên ngoài lớp nhưng thuộc lớp thì vẫn có quyền gọi các hàm thành phần dù nó là **protected** hay **private**.
- Hàm bạn, lớp bạn **vẫn có quyền truy cập** vào **protected** và **private**.

Thông thường:

- **private** → Dùng cho thuộc tính → Tránh giá trị thuộc tính bị thay đổi tùy tiện.
- **public** → Dùng cho phương thức → Chỉ được dùng các phương thức để thay đổi giá trị thuộc tính.

Ý nghĩa: Hiện thực **tính đóng gói**.

3. Hàm bạn (Friend function) và Lớp bạn (Friend class).

Bài toán:

Giả sử có lớp Vector, lớp Matrix. Cần viết hàm nhân Vector với một Matrix, nhưng hàm này:

- Không thể thuộc lớp Vector → Không lấy được dữ liệu private và protected.
- Không thể thuộc lớp Matrix → Không lấy được dữ liệu private và protected.

→ Việc lấy dữ liệu từ lớp Vector và Matrix làm tăng độ phức tạp và không đảm bảo các tính chất của OOP.

Giải pháp: Dùng hàm bạn (hàm toàn cục) và lớp bạn.

Hàm bạn / lớp bạn là hàm / lớp có quyền **truy cập vào các thành phần private và protected** của lớp đó, tương tự **như là một thành phần** của lớp đó, nhưng không phải thành phần của lớp. Điều ngược lại không xảy ra.

Do hàm bạn không thuộc lớp nên không thể kế thừa.

Tính chất:

- Một hàm / lớp có thể là bạn của nhiều lớp khác.
- A có thể là bạn của B, và B cũng có thể là bạn của A.
- **Không có tính đối xứng:** Nếu B là bạn của A thì B có quyền truy cập vào các thành phần riêng tư của A, điều ngược lại không xảy ra.
- **Không có tính bắc cầu:** Nếu C là bạn của B, B là bạn của A, C không thể truy cập vào các thành phần riêng tư của A, điều ngược lại không xảy ra.

Chú ý: Việc sử dụng hàm bạn và lớp bạn cần được thực hiện cẩn thận và hợp lý, vì nó có thể làm **giảm tính đóng gói và gây ra rối loạn trong thiết kế của lớp** → Nên dùng get và set thay thế.

Khi khai báo 2 lớp ở 2 file header riêng biệt là bạn, chỉ cần khai báo đúng cú pháp sau đây (dùng nguyên mẫu), không cần `#include`.

a. Hàm bạn.

Một hàm bạn có thể được khai báo bên trong khai báo lớp khác hoặc bên ngoài khai báo lớp.

Cú pháp khai báo nguyên mẫu hàm: `friend data-type friend-function(parameters);`.

VD.

```
class TamGiac{
private:
    int a,b,c;
public:
    friend bool kiemTraTG(TamGiac tg);
};
```

```

bool kiemTraTG(TamGiac tg){
    if(tg.a>0 && tg.b>0 && tg.c>0 && (tg.a+tg.b>tg.c)&& (tg.a+tg.c>tg.b)&&
(tg.b+tg.c>tg.a))
        return true;
    return false;
}

```

b. Lớp bạn.

Một lớp bạn có thể được khai báo bên trong khai báo lớp khác hoặc bên ngoài khai báo lớp.

Cú pháp khai báo nguyên mẫu lớp: `friend class friend-class;`.

VD.

```

class ClassA {
private:
int privateDataA;
protected:
int protectedDataA;
public:
    friend class ClassB; // Khai báo ClassB là lớp bạn của ClassA
};

class ClassB {
public:
    void modifyData(ClassA& obj) {
        obj.privateDataA = 10; // Truy cập vào thành phần private của ClassA
        obj.protectedDataA = 20; // Truy cập vào thành phần protected của ClassA
    }
};

int main() {
    ClassA objA;
    ClassB objB;
    objB.modifyData(objA); // Gọi phương thức modifyData từ objB để thay đổi dữ
    liệu của objA
    return 0;
}

```

4. Phương thức khởi tạo (Constructor) và bài toán gán giá trị cho các thuộc tính.

a. Vị trí & ý nghĩa của constructor.

- Constructor là phương thức **không có kiểu dữ liệu**, nên cũng **không có giá trị trả về**.
- Được dùng để **gán giá trị khởi tạo cho các thuộc tính**.
- Được khai báo **trong class**.
- Cần phải có phạm vi **public**.
- Một lớp có thể có nhiều constructor với số lượng tham số khác nhau (nạp chồng hàm).
- Khi truyền tham trị, một bản sao của đối tượng được truyền sẽ được tạo ra. Tuy nhiên, lúc đó, constructor sẽ không được gọi.
- Constructor có thể được gọi mà không cần thông qua đối tượng.
Cú pháp gọi (gọi trực tiếp, không cần thông qua đối tượng):
constructor(parameters)
- Cú pháp khai báo:
constructor(parameters){content}

b. Tham số của constructor và phân loại constructor.

- Có 2 loại constructor tùy vào tham số của nó:
 - Không có tham số: **Default constructor**.
 - Có tham số: **Parameter constructor**.
- Trong lớp bắt buộc phải có **default constructor**. Nếu người lập trình không khai báo constructor thì compiler sẽ tự động tạo một default constructor không có nội dung, có tên trùng với tên lớp ⇒ Khi người dùng khai báo parameter constructor mà không khai báo default constructor, default constructor sẽ không được tạo ra tự động.
- Default constructor được compiler tự động gọi khi khởi tạo đối tượng mà **không cần được người lập trình gọi thủ công**.
- Parameter constructor có **toàn bộ tham số là tham số mặc định** thì có vai trò như default constructor.

Chú ý: Khi khai báo con trỏ đối tượng mà chưa trỏ đến đâu, đối tượng chưa thật sự được khởi tạo (mà chỉ có địa chỉ được cấp) nên không nhất thiết phải có default constructor. Tương tự với con trỏ cấp cao và mảng (tĩnh / động) chứa các con trỏ đến đối tượng.

Copy constructor là:

- Một **parameter constructor** có các tham số là **hằng tham chiếu đến đối tượng thuộc lớp đang định nghĩa** (const **class-name** &**parameter-name**) được tự động khởi tạo bởi compiler.
- Được gọi khi người dùng sử dụng toán tử gán (=) trực tiếp đối tượng này vào đối tượng kia (2 đối tượng cùng lớp). Khi đó, phương thức này sẽ **sao chép dữ liệu** từ đối tượng này vào đối tượng kia cùng lớp.

- Người lập trình cần định nghĩa copy constructor thủ công khi:
 - Cần những cài đặt đặc biệt với hành vi sao chép, chẳng hạn như sao chép một phần thành viên.
 - Khi lớp có con trỏ (cần tự định nghĩa để tránh các lỗi liên quan đến bộ nhớ).

c. Initializer.

Initializer là một cách gán giá trị cho thuộc tính mà không sử dụng toán tử = hay con trỏ this->.

- Các initializer được khai báo bên cạnh tên constructor, được ngăn cách với nhau bằng dấu , và được ngăn cách với constructor bằng dấu : .
- Cú pháp trong trường hợp có 2 initializer:

constructor (parameters) : attribute1(value), attribute2(value) {content}.

Value ở đây có thể là tham số của constructor. Cũng có nghĩa, initializer đã gọi constructor.

Chú ý:

- Initializer không nhất thiết phải khai báo đầy đủ đối số cho các property.
- Initializer không thể khai báo trong nguyên mẫu constructor.

Ví dụ: Dùng constructor để gán giá trị vào các thuộc tính của đối tượng s.

Lúc này, s.name = "Bao", s.age = 2.

Constructor không có tham số.

Bên trong constructor thực hiện gán trực tiếp giá trị vào thuộc tính của mọi đối tượng thuộc lớp Student. Trong trường hợp này chỉ có 1 đối tượng là s.

Có thể dùng = hoặc this->.

Khởi tạo bằng tham số mặc định.

```
class Student {
public:
    string name;
    int age;
    Student() {
        name = "Bao";
        age = 2;
    }
};
```

```
int main() {
    Student s;
    return 0;
}
```

```
class Student {
public:
    string name;
    int age;
    Student(string name = "Bao", int age = 2) {
    }
};
```

```
int main() {
    Student s;
    return 0;
}
```

Tham số của constructor có tên khác với thuộc tính.

```
class Student {
public:
    string name;
    int age;
    Student(string n, int a) {
        name = n;
        age = a;
    }
};
```

Tham số của constructor có tên trùng với thuộc tính.

Sử dụng con trỏ this-> để gán giá trị vào thuộc tính.

```
int main() {
    Student s("Bao", 2);
    return 0;
}

#include <iostream>
using namespace std;
class Student {
public:
    string name;
    int age;
    Student(string name, int age) {
        this->name = name;
        this->age = age;
    }
};

int main() {
    Student s("Bao", 2);
    return 0;
}
```

Sử dụng danh sách các initializer.

```
#include <iostream>
using namespace std;
class Student {
public:
    string name;
    int age;
    Student(string name, int age) : name(name), age(age) {
    }
};

int main() {
    Student s("Bao", 2);
    return 0;
}
```

5. Phương thức hủy (Destructor) và bài toán tiết kiệm bộ nhớ.

- Destructor là phương thức **không có kiểu dữ liệu**.
- Destructor không có tham số.
- Mỗi lớp chỉ có 1 destructor.
- Được dùng để **giải phóng bộ nhớ** của đối tượng ở tất cả những nơi gọi thành phần của lớp đó, thay vì phải giải phóng thủ công ở mỗi nơi gọi / mỗi lần gọi.
- Được khai báo **trong class**.
- Destructor **chỉ được gọi tự động** bởi compiler.

- Destructor được gọi **ngay trước khi** kết thúc block code chứa đối tượng thuộc lớp có destructor hay **ngay trước khi** kết thúc chương trình.
- Mỗi lớp chỉ có một destruction.
- Destruction phải có phạm vi **public**.
- Cấu trúc: `~class-name() { content }`.

Ví dụ: Chương trình sau đã gọi destructor mấy lần?

```
#include <iostream>
#include <string>

using namespace std;

class point
{
private:
    float x, y;
public:
    point(int x = 0, int y = 0)
    {
        this->x = x;
        this->y = y;
    }
    ~point()
    {
        cout << "Destructor was called!\n";
    }
    void print()
    {
        cout << "(" << x << ", " << y << ")\n";
    }
    point vectorize(point B)
    {
        point A = {x, y};
        point res = {A.x - B.x, A.y - B.y};
        return res;
    }
};

int main() {
    point A(1, 1);
    point B(5, 5);
    A.vectorize(B).print();
    return 0;
}
```

Tạo & Hủy B

Tạo & Hủy A

Tạo & Hủy res

Tạo & Hủy A

Tạo & Hủy B

Như vậy, toàn bộ chương trình đã cấp phát vùng nhớ 5 lần, trong đó riêng việc vector hóa đã tốn 3 lần cấp phát, gây lãng phí bộ nhớ. Cách code sau giảm còn 4 lần cấp phát:

```
point vectorize(point B)
{
    return {x - B.x, y - B.y};
}
```

Tạo & Hủy B

Tạo & Hủy {x - B.x, y - B.y}

Việc sử dụng **tham chiếu** cũng sẽ tiết kiệm bộ nhớ:

```
point vectorize(const point &B)
{
    return {x - B.x, y - B.y};
}
```

Tạo & Hủy {x - B.x, y - B.y}

6. Thành phần tĩnh (Static member).

Từ khóa static được đặt trước tên biến hay phương thức (thành phần) dùng để chỉ rằng: Giá trị của thành phần đó được dùng chung cho tất cả các đối tượng của lớp đó (tương tự mối quan hệ giữa biến toàn cục và các hàm).

- Nếu static xuất hiện trước một dữ liệu hoặc một phương thức của lớp thì giá trị của nó vẫn được lưu lại và có ý nghĩa cho đối tượng khác của cùng lớp này.
- Các thành phần static có thể là public, private hoặc protected.
- Các thành phần static có thể được truy xuất mà không cần thông qua tên đối tượng.
 - Cách 1: Thông qua tên lớp:
class-name::static-element
 - Cách 2: Thông qua tên đối tượng (bất cứ đối tượng nào thuộc lớp đó):
object-name.static-element
- Các thành phần static tồn tại trước khi 1 đối tượng được khởi tạo.

Ý nghĩa thực tế: Tối ưu bộ nhớ.

Chú ý: Một thuộc tính hằng trong lớp phải luôn là **static const urType urProperty = urValue;**

Ví dụ: Kết quả của chương trình sau là gì:

```
#include <iostream>
#include <string>

using namespace std;

class student {
public:
    static string school;
};

int main() {
    student a, b, c;
    a.school = "US";
    b.school = "BKU";
    c.school = "UIT";

    cout << a.school << ' ' << b.school << ' ' << c.school;
    return 0;
}
```

→ Kết quả: UIT UIT UIT.

7. Nạp chồng phương thức (Method overloading).

Nạp chồng phương thức là một nhiều phương thức cùng tên như tham số khác nhau (khác về số lượng, kiểu dữ liệu và tên gọi của tham số).

8. Nạp chồng toán tử (Operator overloading) và ép kiểu.

a. Bản chất hàm của toán tử.

Toán tử thực chất là các hàm được viết theo cách khác.

VD.

Loại operator	Cách viết tắt	Cách viết đầy đủ	Ý nghĩa
Operator 2 ngôi	<code>A * B</code>	<code>A.operator*(B)</code>	Gọi method operator * từ lớp A.
Operator 1 ngôi	<code>- A</code>	<code>A.operator-()</code>	Gọi method operator - từ lớp A.

b. Cú pháp nạp chồng toán tử.

Nạp chồng toán tử là dùng phương thức để sử dụng toán tử theo một cách khác với cách mặc định.

Cú pháp khai báo nguyên mẫu: `data-type operator @ (parameters);`.

Chú ý:

- Chỉ có thể tái định nghĩa các operator đã có sẵn (không thể tạo toán tử mới), bao gồm `new`, `new[]`, `delete`, `delete[]`.
- Không thể nạp chồng toán tử 3 ngôi (`?:`), toán tử `., ::, .*`.
- Không thể thay đổi thứ tự ưu tiên của toán tử.
- Tất cả các hàm nạp chồng toán tử là hàm thành viên của lớp chỉ được phép có **tối đa 1 tham số**, vì các hàm này đã có sẵn 1 tham số chính là đối tượng gọi hàm. Riêng hàm nạp chồng là friend thì có thể có **tối đa 2 tham số**.

Ví dụ:

```
#include <iostream>

using namespace std;

class phanSo{
private:
    int tu, mau;
public:
    phanSo(int tu = 1, int mau = 1);
    phanSo operator*(phanSo);
    phanSo operator-();
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

// Operator 2 ngôi
phanSo phanSo::operator*(phanSo A) {
    return phanSo(tu * A.tu, mau * A.mau);
}

// Operator 1 ngôi
phanSo phanSo::operator-() {
    return phanSo(-tu, mau);
}

int main() {
    phanSo A, B;
    phanSo C = A * B; // C.tu = 1; C.mau = 1
    phanSo D = -A;    // D.tu = -1; D.mau = 1
    return 0;
}
```

c. Ép kiểu bằng constructor và Nạp chồng toán tử ép kiểu.

Bản chất của operator là hàm, nó chỉ nhận tham số khi đúng kiểu dữ liệu. Khi đó, người lập trình phải khai báo operator cho rất nhiều kiểu dữ liệu, gây phiền toái. Nên, các compiler hiện đại đã được phát triển tính năng ép kiểu ngầm định.

Trong trường hợp toán hạng thứ hai không chung kiểu với toán hạng thứ nhất, compiler sẽ dựa vào constructor để ép kiểu toán hạng thứ hai chung với toán hạng thứ nhất.

```
#include <iostream>

using namespace std;

class phanSo{
private:
    int tu, mau;
public:
    phanSo(int tu = 1, int mau = 1);
    phanSo operator*(phanSo);
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

phanSo phanSo::operator*(phanSo A) {
    return phanSo(tu * A.tu, mau * A.mau);
}

int main() {
    phanSo A, B;
    phanSo C = A * 5;
    return 0;
}
```

Ép kiểu bằng cách gọi constructor **phanSo(5)**

Chú ý: Lớp `int` không có constructor chuyển `phanSo` thành `int` nên đoạn code sau sẽ gây lỗi:

```
phanSo C = 5 * A;
```

Cách 1: Nạp chồng nhiều toán tử:

```
#include <iostream>
using namespace std;

class phanSo{
public:
    int tu, mau;
private:
    phanSo(int tu = 1, int mau = 1);
    phanSo operator*(phanSo A);
    friend phanSo operator*(int n, phanSo A);
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

phanSo phanSo::operator*(phanSo A) {
    return phanSo(tu * A.tu, mau * A.mau);
}

phanSo operator*(int n, phanSo A) {
    return phanSo(n * A.tu, A.mau);
}

int main() {
    phanSo A, B;
    phanSo C = 5 * A; // C.tu = 5, C.mau = 1
    phanSo D = A * 5; // D.tu = 5, D.mau = 1
    return 0;
}
```

Ép kiểu bằng cách gọi constructor `phanSo(5)`

Gọi hàm *bạn* (hàm toàn cục) do lớp `int` không có constructor chuyển đổi

Việc dùng hàm bạn toàn bộ sẽ giúp tiết kiệm chi phí khai báo:

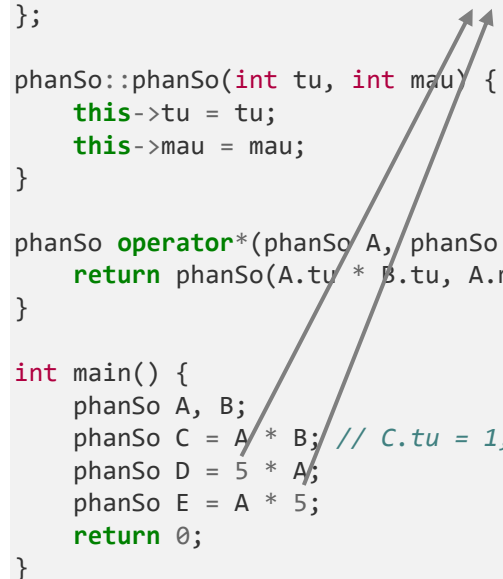
```
#include <iostream>
using namespace std;

class phanSo{
private:
    int tu, mau;
public:
    phanSo(int tu = 1, int mau = 1);
    friend phanSo operator*(phanSo A, phanSo B);
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

phanSo operator*(phanSo A, phanSo B) {
    return phanSo(A.tu * B.tu, A.mau * B.mau);
}

int main() {
    phanSo A, B;
    phanSo C = A * B; // C.tu = 1, C.mau = 1
    phanSo D = 5 * A;
    phanSo E = A * 5;
    return 0;
}
```



Cách 2: Gọi constructor để tạo object mới.

```
phanSo C = phanSo(5) * A;
```

Khi muốn chuyển object sang kiểu dữ liệu vốn có của C++, ta phải dùng ép kiểu tường minh với các toán tử ép kiểu:

Cú pháp khai báo nguyên mẫu: `operator datatype();`.

```
#include <iostream>
#include <math.h>
using namespace std;

class phanSo{
private:
    int tu, mau;
public:
    phanSo(int tu = 1, int mau = 1);
    operator double() const;
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

phanSo::operator double() const {
    return (double)tu / mau;
}

int main() {
    phanSo A;
    double a = sqrt(A); // a = 1
    return 0;
}
```

Hàm `sqrt` của `math.h` không nhận tham số `phanSo` nên compiler tự ép kiểu dựa trên nạp chồng `double`

Chú ý: Khi khai báo quá nhiều phương thức ép kiểu, compiler có thể báo lỗi vì không biết nên chọn phương pháp nào.

```
#include <iostream>
using namespace std;

class phanSo{
private:
    int tu, mau;
public:
    phanSo(int tu = 1, int mau = 1);
    operator double() const;
    phanSo operator * (phanSo);
};

phanSo::phanSo(int tu, int mau) {
    this->tu = tu;
    this->mau = mau;
}

phanSo::operator double() const {
    return (double)tu / mau;
}

phanSo phanSo::operator*(phanSo A) {
    return phanSo(tu * A.tu, mau * A.mau);
}

int main() {
    phanSo A;
    phanSo C = A * 5;
    return 0;
}
```

Compiler không biết nên chọn cách ép kiểu nào để thực hiện phép tính.

Cách khắc phục: Ép kiểu tường minh.

d. Nạp chồng toán tử gán (=).

Toán tử gán mặc định có chức năng sao chép từng thành phần của đối tượng kia cho đối tượng này. Trong trường hợp cần quản lý bộ nhớ động, tính năng này rất nguy hiểm vì nó có thể dẫn đến truy cập vùng nhớ đã bị hủy cấp phát.

VD: Đoạn chương trình sau có lỗi hay không:

```
class String {
    char *p;
public:
    String(char *s = "") { p = strdup(s); }
    String(const String &s) { p = strdup(s.p); }
    ~String() { delete [] p; }
    void Output() const { cout << p; }
};

void main() {
    String a("Nguyen Van A");
    String b = "Le van AA";

    b = a;

}
```

Tạo b → String a("Nguyen Van A"); ← Tạo a

b = a; → b trỏ đến địa chỉ mà a trỏ

Hủy a

Hủy b (Lỗi), do địa chỉ mà a trỏ đã bị hủy trước đó.

Giải pháp: Nạp chồng toán tử = có chức năng **hủy vùng nhớ cũ** và **sao chép mới**:

```
String& String::operator = (const String &s) {
    if (this != &s) { // Ko xử lý TH 1 objt được gán cho chính nó
        delete [] p; // Hủy vùng nhớ hiện tại
        p = strdup(s.p); // Tạo vùng nhớ mới bằng cách sao chép s
    }
    return *this; // Trả về vùng nhớ hiện tại sau khi cấp
}
```

Nạp chồng toán tử = nên trả về ***this** để phục vụ cho việc gán liên tiếp (VD: a = b = c ...).

f. Nạp chồng toán tử ++ và --.

Nguyên lý toán tử ++ (--) tiền tố:

- Tăng (giảm) toán hạng 1 đơn vị.
- Trả về tham chiếu đến chính toán hạng đó.

Nguyên lý toán tử ++ (--) hậu tố:

- Tăng (giảm) toán hạng 1 đơn vị.
- Trả về tham chiếu đến chính toán hạng đó trước khi tăng.

```
className &operator ++(); // Tiền tố
className operator ++(int); // Hậu tố (Cần tham số)

className &className::operator ++() {
    // Code
    return *this;
}

className className::operator ++(int) {
    className obj = *this;
    // Code
    return obj;
}
```

e. Xử lý luồng nhập - xuất, Nạp chồng toán tử >> và <<.

Có 2 lớp chỉ định luồng nhập - xuất trong C++:

- `std::istream::istream`: Luồng nhập.
- `std::ostream::ostream`: Luồng xuất.

Cả `istream` và `ostream` đều không thể được sao chép vì chúng không có copy constructor. Do đó, chỉ có làm việc với chúng thông qua *tham chiếu*.

Có 2 đối tượng chỉ định nơi xuất phát luồng nhập - xuất:

Đối với nhập - xuất với thiết bị chuẩn:

- `std::istream::cin`
: Bàn phím.
- `std::ostream::cout`
: Màn hình.

Đối với nhập - xuất file:

- `std::fstream::ifstream`
: File để đọc.
- `std::fstream::ofstream`
: File để ghi.
- `std::fstream::fstream`
: File vừa để đọc, vừa để ghi.

Ghi chú:

- `.is_open()`
: Trả về true nếu file đã được mở.
- `.close()`
: Chủ động đóng file.
- `ifstream fileObject("filePath")`
: Mở file tại địa chỉ `filePath` bằng `ifstream`.

Ví dụ: Xét luồng sau:

```
cin >> a >> b;
```

→ Từ `cin` (bàn phím), đọc vào biến `a`, con trỏ được dịch chuyển liền sau vị trí của đối số `a` và đưa vào `cin`.

→ Từ `cin`, đọc vào biến `b`, con trỏ được dịch chuyển liền sau vị trí của đối số `b` và đưa vào `cin`.

Như vậy, luồng nhập - xuất được thực hiện tuần tự từ trái sang phải. Cứ mỗi lần nhập / xuất xong một giá trị, vị trí con trỏ được dịch chuyển liền sau vị trí tương ứng, đưa vào `cin` / `cout` và trả về đối tượng `cin` / `cout`. Sau đó, lại dựa trên `cin` / `cout` này để nhập / xuất tiếp.

Ta có thể khai báo hàm nhập và xuất với đối tượng chỉ định nguồn nhập - xuất bất kỳ:

Ví dụ:

```
void readFrom(istream& is) { is >> x; };  
void writeTo(ostream& os) { os << x; };
```

Hai hàm trên có thể hoạt động với bất kỳ đối tượng `istream` và `ostream` nào một cách liên tiếp mà vị trí con trỏ không bị đứt quãng.

Ví dụ:

```
friend istream& operator >> (istream &is, PhanSo &p);
friend ostream& operator << (ostream &os, PhanSo p);

istream& operator >> (istream &is, PhanSo &p) {
    is >> p.tu >> p.mau;

    while (!p.mau){
        cout << "Mau so khong hop le. Nhap lai: ";
        is >> p.mau;
    }

    p.UocLuoc();
    return is;
}

ostream& operator << (ostream &os, PhanSo p) {
    os << p.tu;
    if (p.tu != 0 && p.mau != 1)
        os << '/' << p.mau;
    return os;
}
```

Chú ý: Trong định nghĩa nạp chồng, nếu dùng cin (>>) thay vì is hoặc cout (<<) thay vì os, compiler vẫn biên dịch được nhưng sẽ chỉ đúng khi cin hoặc cout trong phần thân chương trình. Mục tiêu của tham chiếu is và os dùng để tổng quát hóa đối tượng, giúp hàm hoạt động với cả những đối tượng không phải cin và cout.

II. VẬN DỤNG CÁC TÍNH CHẤT CỦA OPP.

1. Tính đóng gói (Encapsulation).

a. Tính đóng gói.

Đây là kỹ thuật giúp bạn che giấu đi những thông tin bên trong đối tượng bằng cách sử dụng phạm vi truy cập **private** cho các thuộc tính. Muốn giao tiếp hay lấy ra các thông tin của đối tượng thì phải thông qua các phương thức **public**, từ đó sẽ hạn chế được các lỗi khi phát triển chương trình.

Lợi ích:

- Hạn chế được các truy xuất không hợp lệ tới các thuộc tính của đối tượng.
- Ổn định trạng thái của đối tượng.
- Ẩn đi những thông tin không cần thiết về đối tượng.
- Cho phép bạn thay đổi cấu trúc bên trong lớp mà không ảnh hưởng tới lớp khác.

b. Setter và getter.

- Setter là phương thức nạp dữ liệu vào thuộc tính, trong tên thường có tiền tố “set”, thường sử dụng con trỏ `this` -> `attribute` = `attribute`, kiểu **void**, **tham số** attribute.
- Getter là phương thức thu dữ liệu từ đối tượng, trong tên thường có tiền tố “get”, thường sử dụng từ khóa `return` `value`, không có tham số.
- Các thuộc tính có phạm vi truy cập là **private**.
- Các getter và setter có phạm vi truy cập là **public**.
- Do có phạm vi `private`, các thuộc tính chỉ có thể được thay đổi thông qua setter và được lấy ra bởi getter.

2. Tính kế thừa (Inheritance).

a. Quan hệ giữa các lớp đối tượng.

Giữa các lớp đối tượng có những loại quan hệ sau:

- **Quan hệ một một (1-1):** Một đối tượng thuộc lớp này có quan hệ duy nhất với một đối tượng thuộc lớp kia.
- **Quan hệ một nhiều (1-n):** Một đối tượng thuộc lớp này quan hệ với nhiều đối tượng thuộc lớp kia và một đối tượng lớp kia có quan hệ duy nhất với một đối tượng thuộc lớp này.
- **Quan hệ nhiều nhiều (n-n):** Một đối tượng thuộc lớp này có quan hệ với nhiều đối tượng thuộc lớp kia và một đối tượng lớp kia cũng có quan hệ với nhiều đối tượng thuộc lớp này.
- **Quan hệ đặc biệt hóa - tổng quát hóa:** Đối tượng này là trường hợp đặc biệt của lớp đối tượng kia và lớp đối tượng kia là trường hợp tổng quát của lớp đối tượng này.

b. Tính kế thừa.

Ở mức độ lập trình: Kế thừa là thừa hưởng lại những thuộc tính và phương thức của một lớp. Có nghĩa là nếu lớp A kế thừa lớp B thì lớp A sẽ có những thuộc tính và phương thức của lớp B.

Ở mức độ trừu tượng:

- Kế thừa là một đặc điểm của ngôn ngữ dùng để biểu diễn mối quan hệ đặc biệt hóa – tổng quát hóa giữa các lớp. Các lớp được trừu tượng hóa và được tổ chức thành một sơ đồ phân cấp lớp.
- Sự kế thừa là một mức cao hơn của trừu tượng hóa, cung cấp một cơ chế gom chung các lớp có liên quan với nhau thành một mức khái quát hóa đặc trưng cho toàn bộ các lớp nói trên.

Đa kế thừa: Là một lớp kế thừa nhiều lớp khác.

c. Lớp con (Subclass) và lớp cha (Parent class).

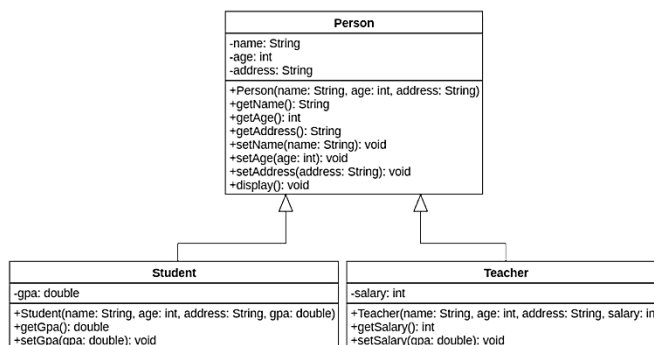
Lớp được thừa hưởng những thuộc tính và phương thức từ lớp khác được gọi là **dẫn xuất** (Derived Class) hay **lớp con** (Subclass) và lớp bị lớp khác kế thừa được gọi là **lớp cơ sở** (Base Class) hoặc **lớp cha** (Parent Class).

VD.

Lớp cha: Person.

Lớp con: Student, Teacher.

Các lớp con sẽ bao gồm những thành phần của cha (kế thừa) và những thành phần riêng của riêng con.



Cú pháp khai báo lớp con:

```
class subclass : access-modifier parent-class {
    access-modifier:
        // Nội dung giống như một class thông thường
};
```

Chú ý: Lớp con có thể:

- **Sử dụng** các phương thức và thuộc tính của lớp cha (tùy thuộc vào phạm vi kế thừa).
- **Định nghĩa lại** các thành phần của lớp cha (các thành phần của cha đã bị con ghi đè). Trong trường hợp đó, lớp con có thể gọi các thành phần cùng tên của lớp cha bằng toán tử phạm vi: `parent-class::member` hoặc dùng từ khóa `virtual` (giới thiệu sau).
- **Định nghĩa thêm** các thành phần mà lớp cha không có.

d. Phạm vi kế thừa.

Bạn có thể thay đổi phạm vi truy cập của các thành phần được kế thừa từ cha bằng cách kế thừa các với từ khóa phạm vi kế thừa: **public**, **protected**, **private** (mặc định).

Các thành phần có phạm vi **private** thì **được kế thừa nhưng không thể được sử dụng**. Trong trường đó, lớp con vẫn có thể vẫn sử dụng các thành phần **private** bằng cách:

- Sử dụng **get**, **set** (không thuận tiện).
- Khai báo lớp con là bạn của lớp cha (**friend**) (không đảm bảo tính đóng gói và nguyên tắc lập trình OOP).
- Đổi các thành phần **private** của cha thành **protected**.

Các phương thức **friend** không thể được kế thừa do không là thành phần của lớp.

Phạm vi truy cập của cha (Của lớp cha)	Phạm vi kế thừa	Phạm vi truy cập vào cha (Của lớp con)
public	public	public
protected		protected
private		private
public	protected	protected
protected		protected
private		private
public	private	private
protected		private
private		private

→ Phạm vi truy cập vào các thành phần của cha khi được kế thừa luôn có xu hướng bị thu hẹp lại chứ không mở rộng ra.

VD:

```
class A {
    public: int x;
    protected: int y;
    private: int z;
};

class B : public A {
    // x: public
    // y: protected
    // z là private nên không thể được truy cập từ Lớp B
};

class C : protected A {
    // x: protected
    // y: protected
    // z là private nên không thể được truy cập từ Lớp C
};

class D : private A {
    // x: private
    // y: private
    // z là private nên không thể được truy cập từ Lớp D
};
```

e. Kế thừa constructor.

Constructor của lớp cha sẽ luôn được tự động gọi **trước tiên** khi một đối tượng của lớp con được khởi tạo. Mặc định, compiler luôn gọi default constructor.

Bạn có thể chủ động gọi parameter constructor thay cho default constructor bằng cú pháp:

```
sub-constructor(parameters) : parent-constructor(parameters) {  
    //Nội dung  
}
```

→ **parent-constructor** được thực thi trước, sau đó mới thực thi **sub-constructor**.

f. Kế thừa destructor.

Destructor của lớp cha sẽ luôn được tự động gọi **sau khi** gọi destructor của lớp con.

Do đó, lớp cần **không cần** (và cũng **không được**) chủ động dọn dẹp tài nguyên thành phần của cha. Việc xử lý dọn dẹp tài nguyên của cha có thể xảy ra lỗi liên quan đến vùng nhớ.

g. Con trỏ trong kế thừa, Ép kiểu con về cha.

Một đối tượng con trỏ thuộc lớp cha có thể được cấp phát dữ liệu lớp con, nhưng điều ngược lại không nên xảy ra vì dữ liệu của lớp cha có thể rộng hơn lớp con và việc cố ép dữ liệu có thể gây ra lỗi.

```
parentClass* parent = new subClass; // Hợp Lệ  
subClass* sub = new parentClass;    // Có thể dẫn đến Lỗi
```

Giả sử có class Parent chứa các property a, b; class Sub là kế thừa Parent và chứa c, d. Để nạp chồng toán tử >> đọc vào dữ liệu cho cả a, b, c, d, bạn cần **ép kiểu của đối tượng con về cha** thông qua cú pháp: **static_cast<Sub*>(obj)** (Static cast) hoặc **(Sub)(obj)** (C-style cast).

h. Đa kế thừa (Multiple inheritance), Vấn đề hình thoi (Diamond problem) và Kế thừa ảo (Virtual inheritance).

[Đọc thêm]

Đa kế thừa là hiện tượng một lớp kế thừa nhiều lớp khác.

```
class sub : public parentA, private parentB { ... };
```

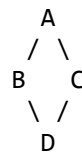
Khi đó: Constructor của các lớp cha được gọi theo thứ tự *từ trái sang phải* (theo thứ tự liệt kê). Còn destructor thì ngược lại.

Diamond problem trong là một vấn đề thường gặp khi dùng kế thừa nhiều lớp và đa kế thừa, đặc biệt khi một lớp con kế thừa từ hai lớp cha - thứ cùng kế thừa chung một lớp cha nữa.

Giả sử có các lớp:

- A là lớp cha lớn nhất.
- B và C đều kế thừa từ A.
- D kế thừa từ cả B và C.

Mô hình:



Vấn đề: Khi truy cập đến một thành phần chung giữa B, C từ D (*do các thành phần chung giữa cha và con sẽ bị con ghi đè*), compiler sẽ không thể nhận biết được nên truy cập cụ thể đến thành phần từ lớp nào (*Ambiguous*).

```

#include <iostream>
using namespace std;

class A {
public:
    void show() {
        cout << "Class A" << endl;
    }
};

class B : public A {
};

class C : public A {
};

class D : public B, public C {
};

int main() {
    D obj;
    // obj.show(); // Ambiguous
    obj.B::show(); // OK
    obj.C::show(); // OK
    return 0;
}

```

Cách khắc phục: Dùng **kế thừa ảo**: Thêm từ khóa **virtual** vào trước các lớp trung gian.

```

class B : virtual public A {
};

class C : virtual public A {
};

```

→ Từ khóa **virtual** có chức năng “che” các thành phần của lớp B và C. Khi đó, các thành phần của lớp A sẽ được gọi thay vì bị ghi đè bởi B và C, và rồi bị compiler trả về Ambiguous.

3. Tính đa hình (Polymorphism) và Tính trừu tượng (Abstraction).

a. Tính đa hình.

Đa hình là hiện tượng các đối tượng thuộc các lớp khác nhau có khả năng hiểu cùng một hàm theo các cách khác nhau.

Ví dụ: Nhận được cùng một thông điệp “nhảy”, nhưng *một con kangaroo và một con cóc nhảy theo hai kiểu khác nhau*. Chúng cùng có hành vi “nhảy” nhưng các hành vi này có nội dung khác nhau.

b. Phương thức ảo (Virtual method), Liên kết tĩnh (Static binding) và Liên kết động (Dynamic binding).

Giả sử có bài toán *lưu trữ danh sách các đối tượng thuộc các lớp khác nhau*. Do một danh sách thông thường chỉ lưu được một lớp nên ta phải dùng cách khác.

Vận dụng đặc điểm kế thừa. Giả sử *các đối tượng cần lưu trữ thuộc các lớp con cùng kế thừa một lớp cha*. Do một đối tượng con trở thuộc lớp cha có thể được cấp dữ liệu lớp con, nên ta có thể tạo danh sách con trở lớp cha để thay thế:

```
vector <parent-class*> list;
```

Lúc này phát sinh bài toán mới. Giả sử *các lớp con có các phương thức tương tự nhau nhưng khác nhau về nội dung* thì làm sao phân biệt được: Phần tử nào trong danh sách thuộc lớp con nào?

Có 2 cách giải quyết:

- **Dùng vùng chọn kiểu (liên kết tĩnh):** Thêm một thuộc tính quyết định kiểu dữ liệu của lớp cha hay khai báo riêng biệt các phương thức ở lớp con → Gọi trực tiếp phương thức cần gọi (đã được biết trước khi biên dịch chương trình).
- **Sử dụng phương thức ảo (liên kết động):** Thêm từ khóa `virtual` vào trước phương thức của lớp cha. Khi đó, các phương thức của cha sẽ bị các phương thức của con ghi đè → Khi chương trình chạy (runtime), compiler mới xác định phương thức nào cần được gọi.

c. Tính trừu tượng, Phương thức thuần ảo (Pure virtual method), Lớp cơ sở trừu tượng (Abstract class).

Bạn chỉ có thể truy cập đến các phương thức của con mà có *phương thức cha tương ứng* → Có những phương thức mà nội dung của nó được định nghĩa tại các lớp con (*phương thức tổng quát*).

VD:

```
class Shape
{
public:
};

class Circle : public Shape {
public:
    void print() { cout << "I am a circle" << endl;}
};

class Rectangle : public Shape {
public:
    void print() { cout << "I am a rectangle" << endl;}
};

int main() {
    Rectangle obj;
    obj.draw();    // Lỗi
}
```

→ Lỗi do draw không có ở Shape.

Sửa chữa:

```
class Shape
{
public:
    virtual void print() { cout << "I am a shape" << endl;}
};

class Circle : public Shape {
public:
    void print() { cout << "I am a circle" << endl;}
};

class Rectangle : public Shape {
public:
    void print() { cout << "I am a rectangle" << endl;}
};

int main() {
    Rectangle obj;
    obj.print();    // I am a rectangle
}
```

Tuy vấn đề đã được giải quyết, nhưng lớp `Shape::print()` không được dùng đến, dẫn đến dư thừa, thiếu tường minh. Để khắc phục vấn đề này, C++ hỗ trợ người dùng khái niệm phương thức thuần ảo.

Phương thức thuần ảo là **phương thức ảo** có giá trị là **0** và **không có định nghĩa**.

```
class Shape
{
public:
    virtual void print() = 0;
};
```

→ Có vai trò tương tự cách giải quyết cũ.

Lớp có ít nhất 1 phương thức thuần ảo được gọi là lớp **cơ sở trừu tượng**.

Chú ý: Không thể khởi tạo 1 đối tượng cụ thể thuộc lớp cơ sở trừu tượng, ngoại trừ con trỏ vì con trỏ chưa có giá trị.

MỘT SỐ THUẬT TOÁN THÊM

Nếu không có yêu cầu cụ thể, sinh viên có thể sử dụng các cấu trúc dữ liệu và thuật toán sẵn có của STL (như `vector`, không nhất thiết phải sử dụng con trỏ).

I. LỚP PHÂN SỐ (cPhanSo).

Lớp phân số có 2 thuộc tính riêng tư như sau:

```
float tu, mau;
```

Có thể xây dựng một thuộc tính nữa để lưu ước chung lớn nhất (gcd) nếu cần.

```
unsigned int gcd;
```

C++17 có một hàm có sẵn để tính gcd, thuộc namespace `std` và thư viện `numeric`.

```
std::gcd(a, b) // Trả về gcd(a, b)
```

Trong trường hợp bạn sử dụng compiler `GCC` hoặc `Clang` thì có thể dùng hàm sau:

```
__gcd(a, b) // Trả về gcd(a, b)
```

Định nghĩa hàm tính gcd:

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}
```

1. Kiểm tra phân số.

```
void cPhanSo::Nhap()  
{  
    cout << "Nhap phan so:\n";  
    cout << "- Nhap tu: ";  
    cin >> tu;  
    while (true) {  
        cout << "- Nhap mau: ";  
        cin >> mau;  
        if (mau == 0)  
            cout << "Mau so khong hop le. Hay nhap lai.\n";  
        else  
            break;  
    }  
}
```

2. Xuất phân số.

```
void cPhanSo::Xuat()  
{  
    if (mau == 1 || tu == mau)  
        cout << tu;  
    else  
        cout << tu << '/' << mau;  
}
```

3. Rút gọn phân số.

Phân số được rút gọn bằng cách chia cả tử và mẫu cho gcd của chúng (lấy trị tuyệt đối).

4. Quy đồng phân số.

Cho 2 phân số:

$$\frac{t_1}{m_1}, \quad \frac{t_2}{m_2}$$

2 phân số này có thể được đưa về chung mẫu số là $m_1 \cdot m_2$ nhờ phép quy đồng:

$$\frac{t_1}{m_1} = \frac{t_1}{m_1} \cdot \frac{m_2}{m_2}, \quad \frac{t_2}{m_2} = \frac{t_2}{m_2} \cdot \frac{m_1}{m_1}$$

Phép quy đồng chỉ cần thiết trong phép cộng và phép trừ. Nên ta không cần thiết phải lập phương thức riêng để tính toán.

II. LỚP ĐIỂM VÀ ĐA GIÁC (cDiem, cDaGiac).

Lớp điểm có 2 thuộc tính riêng tư:

```
float x, y;
```

Lớp đa giác chỉ có 1 vector để lưu trữ danh sách các điểm.

```
vector <cDiem> diems;
```

1. Các phép toán với điểm.

Tịnh tiến 1 điểm thêm d_x và d_y :

$$A' = (x_A + d_x, \quad y_A + d_y)$$

Kéo dài đoạn thẳng BA k lần về phía A :

$$A' = (x_B + k \cdot (x_A - x_B), \quad y_B + k \cdot (y_A - y_B))$$

Xoay điểm A theo điểm B r rad ngược chiều kim đồng hồ:

1. Tịnh tiến B về gốc tọa độ.

2. Thay đổi tọa độ của A :

$$A' = (x_A \cdot \cos r - y_A \cdot \sin r, \quad x_A \cdot \sin r + y_A \cdot \cos r)$$

3. Tịnh tiến B về vị trí cũ.

Khoảng cách giữa 2 điểm AB :

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

2. Các phép toán với đa giác có n đỉnh.

Trọng tâm của đa giác:

$$G = \left(\frac{\sum x_i}{n}, \quad \frac{\sum y_i}{n} \right)$$

Chu vi đa giác:

$$c = \sum d(A_i, A_j), \quad j = (i + 1) \% n$$

Diện tích đa giác:

$$s = \frac{|\sum (x_i \cdot y_j - x_j \cdot y_i)|}{2}, \quad j = (i + 1) \% n$$

Tịnh tiến đa giác: Là tịnh tiến từng đỉnh của đa giác.

Phóng to đa giác: Là phóng to các cạnh GA , A là các đỉnh của đa giác.

IV. LỚP GIỜ-PHÚT-GIÂY (cTime).

Các thành phần riêng tư:

- **Cách 1:** Tách biệt giờ, phút, giây:

```
int gio, phut, giay;
```

- **Cách 2:** Chỉ khai báo tổng số giờ, hoặc tổng số phút, hoặc tổng số giây.

```
int giay;
```

Đối với cách 2. Gọi s là tổng số giây, số giờ, số phút, số giây được tính như sau:

$$g = \left\lfloor \frac{s}{3600} \right\rfloor, \quad p = \left\lfloor \frac{s \% 3600}{60} \right\rfloor, \quad g = s \% 60$$

V. LỚP NGÀY-THÁNG-NĂM (cDate).

Các thành phần riêng tư:

```
int iDay, iMonth, iYear;
```

Trong trường hợp đặc biệt có thể thêm `sDayInWeek` để lưu trữ thứ (dưới dạng chuỗi).

1. Kiểm tra năm nhuận.

Năm nhuận là năm chia hết cho bốn và không chia hết cho 100, hoặc là năm chia hết cho 400.

```
bool cDateTime::isLeapYear()
{
    return (iYear % 4 == 0 && iYear % 100 != 0) || (iYear % 400 == 0);
}
```

2. Kiểm tra ngày-tháng-năm hợp lệ.

- Các tháng 4, 6, 9, 11 có 30 ngày.
- Các tháng 1, 3, 5, 7, 8, 10, 12 có 31 ngày.
- Nếu năm nhuận, tháng 2 có 29 ngày. Nếu không thì tháng 2 có 28 ngày.

```
vector<char> cDate::GetDaysInMonth()
{
    vector<char> daysInMonth = { 0, 31, 0, 31, 30, 31, 30, 31, 31, 30,
31, 30, 31 };
    daysInMonth[2] = IsLeapYear() ? 29 : 28;
    return daysInMonth;
}
```

```
bool cDate::IsValid()
{
    if (year <= 0)
        return false;

    if ((month < 1) || (month > 12))
        return false;

    if ((day < 1) || (day > GetDaysInMonth()[month]))
        return false;

    return true;
}
```

Hàm `GetDaysInMonth` có thể được tái sử dụng để xét trong các bài toán sau.

3. Kiểm tra thứ trong tuần.

Thứ trong tuần được tính theo công thức Zeller:

Đặt:

$$Y = y \% 100, \quad C = \left\lfloor \frac{y}{100} \right\rfloor$$

Thứ trong tuần được xác định bằng hệ số h như sau:

$$h = \left(d + \left\lfloor \frac{13 \cdot (m + 1)}{5} \right\rfloor + Y + \left\lfloor \frac{Y}{4} \right\rfloor + \left\lfloor \frac{C}{4} \right\rfloor - 2C \right) \% 7$$

$(h + 7) \% 7$ cho biết thứ trong tuần dưới dạng số nguyên, với $(h + 7) \% 7 = 0$ là Chủ nhật.

```
string cDateTime::getDayOfWeek()
{
    if (iMonth < 3) {
        iMonth += 12;
        iYear -= 1;
    }

    int Y = iYear % 100;
    int C = iYear / 100;

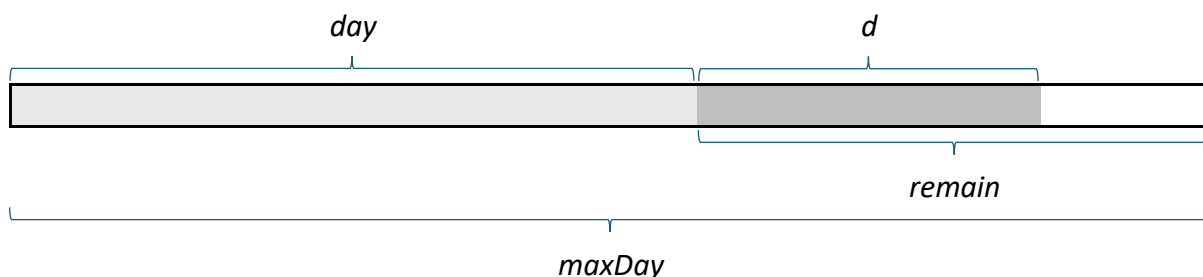
    int h = (iDay + (13 * (iMonth + 1)) / 5 + Y + (Y / 4) + (C / 4) - (2 *
C)) % 7;

    string days[] = { "Chu Nhat", "Thu Hai", "Thu Ba", "Thu Tu",
        "Thu Nam", "Thu Sau", "Thu Bay" };

    return days[(h + 7) % 7];
}
```

4. Thêm ngày-tháng-năm theo số ngày.

Giả sử ta có d là số ngày lệch cần thêm vào, day là số ngày hiện tại, $maxDay$ là số ngày tối đa trong tháng:



- Nếu $d < remain$, ta cứ thế cộng d vào day thôi.
- Nếu $d \geq remain$:
 - Ngày quay về 1, tháng tăng lên 1. Nếu tháng quá 12 thì cài đặt lại.
 - Giảm d trên đoạn $remain$.
- Thực hiện lại các bước trên cho đến khi $d \leq 0$.

```
cDate cDate::operator+(int d)
{
    cDate temp = *this;

    while (d > 0) {
        auto daysInMonth = temp.GetDaysInMonth();
        int remain = daysInMonth[temp.month] - temp.day + 1;

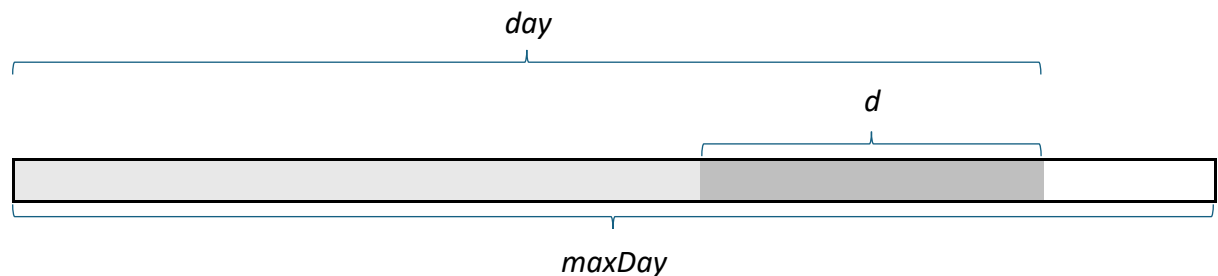
        if (d <= remain) {
            temp.day += d;
            return temp;
        }

        d -= remain;
        temp.day = 1;
        temp.month++;

        if (temp.month > 12) {
            temp.month = 1;
            temp.year++;
        }
    }
    return temp;
}
```

4. Bớt ngày-tháng-năm theo số ngày.

Giả sử ta có d là số ngày lịch cần bớt đi, day là số ngày hiện tại, $maxDay$ là số ngày tối đa trong tháng:



Ý tưởng của thuật toán này cũng tương tự như thêm ngày-tháng-năm:

- Nếu $d < day$: Trừ thẳng day cho d .
- Nếu $d \geq day$:
 - Giảm tháng. Nếu tháng quá 12 thì cài đặt lại.
 - Đặt $day = maxDay$.
- Thực hiện lại các bước trên cho đến khi $d \leq 0$.

```
cDate cDate::operator-(int d)
{
    cDate temp = *this;
    while (d > 0) {
        if (d < temp.day) {
            temp.day -= d;
            return temp;
        }
        d -= temp.day;
        temp.month--;
        if (temp.month < 1) {
            temp.month = 12;
            temp.year--;
        }
        auto daysInMonth = temp.GetDaysInMonth();
        temp.day = daysInMonth[temp.month];
    }
    return temp;
}
```

6. Tìm khoảng cách giữa 2 ngày-tháng-năm.

Các bước:

1. Quy đổi cả 2 về số ngày (tính từ 0).
2. Lấy hiệu 2 số ngày trên.

Để đổi cả 2 về số ngày, ta chỉ cần duyệt tuần tự từng năm / từng tháng và cộng số ngày ứng với mỗi năm / tháng:

```
int cDate::CountDaysFromStart()
{
    int count = 0;

    cDate temp = *this;

    for (int year = 1; year < temp.year; year++) {
        count += IsLeapYear(year) ? 366 : 365;
    }

    auto DaysInMonth = temp.GetDaysInMonth();
    for (int month = 1; month < temp.month; month++) {
        count += DaysInMonth[month];
    }
    count += temp.day;

    return count;
}
```

```
int cDate::operator-(cDate& date)
{
    return CountDaysFromStart() - date.CountDaysFromStart();
}
```

Việc lấy trị tuyệt đối `CountDaysFromStart() - date.CountDaysFromStart()` có thể thực hiện ngoài lớp để đảm bảo định nghĩa của operator -.

VI. LỚP ĐƠN THỨC VÀ ĐA THỨC (cDonThuc, cDaThuc).

Lớp đơn thức có 2 thuộc tính riêng tư:

```
float heSo;  
int bac;
```

Trong một số trường hợp, bac có thể là float.

Lớp đa thức thực chất là danh sách các đơn thức:

```
vector<cDonThuc>dt;
```

Lớp đơn thức chỉ được sử dụng để thuận tiện cài đặt lớp đa thức. Không sử dụng riêng lớp này trên thực tế.

1. Kiểm tra đa thức.

Đa thức là tập hợp các đơn thức, mỗi đơn thức có dạng:

$$ax^k, \quad a \in \mathbb{R}, \quad k \in \mathbb{N}$$

```
istream& operator>>(istream& is, cDonThuc& dt)  
{  
    cout << "- Nhập hệ số và số mũ: ";  
    is >> dt.heSo >> dt.bac;  
    while (dt.bac < 0) {  
        cout << "Bac không được < 0. Hay nhập lại bac: ";  
        cin >> dt.bac;  
    }  
    return is;  
}
```

2. Xuất đa thức dạng chuẩn.

Xuất đơn thức:

Yêu cầu:

- Hệ số = 1 $\Rightarrow$ Không cần xuất hệ số.
- Mũ = 1 $\Rightarrow$ Không cần xuất số mũ.
- Đơn thức đầu tiên của đa thức dương thì không cần xuất dấu.

```
ostream& operator<<(ostream& os, cDonThuc& dt)
{
    if (dt.bac == 0)
        os << dt.heSo;
    else {
        if (dt.heSo == 1)
            os << "x";
        else if (dt.heSo == -1)
            os << "-x";
        else
            os << dt.heSo << "x";
        if (dt.bac != 1)
            os << "^" << dt.bac;
    }
    return os;
}
```

Xuất đa thức:

Yêu cầu:

- Nếu đa thức chỉ có 0 thì xuất số 0.
- Nếu đa thức có nhiều đơn thức thì không xuất số 0.

```
ostream& operator<<(ostream& os, cDaThuc dt)
{
    if (dt.dt.size() > 0) {
        os << dt.dt[0];
        for (int i = 1; i < dt.dt.size(); i++) {
            if (dt.dt[i].getHeSo() != 0) {
                os << " ";
                if (dt.dt[i].getHeSo() > 0)
                    os << "+";
                os << dt.dt[i];
            }
        }
    }
    return os;
}
```

3. Chuẩn hóa đa thức để tính toán.

Để thuận tiện cho tính toán, ta nên đưa các đa thức về dạng chuẩn:

$$P = a_1x^k + a_2x^{k-1} + a_3x^{k-2} + \dots + a_nx^0$$

Kể cả $a_i = 0$ cũng phải được liệt kê vào dạng chuẩn này.

Các bước:

1. Tìm bậc lớn nhất.
2. Lần lượt duyệt các bậc từ lớn nhất về 0, sau đó cộng các đơn thức có bậc bằng bậc đang xét và đưa vào danh sách.

```
cDaThuc cDaThuc::chuanHoa()
{
    if (dt.size() == 0)
        return *this;

    int maxBac = dt[0].getBac();
    for (auto donThuc : dt) {
        maxBac = max(maxBac, donThuc.getBac());
    }

    cDaThuc ketQua;
    for (int i = maxBac; i >= 0; i--) {
        cDonThuc tam(0, i);

        for (auto donThuc : dt)
            if (donThuc.getBac() == i)
                tam.setHeSo(tam.getHeSo() + donThuc.getHeSo());

        ketQua.dt.push_back(tam);
    }

    return ketQua;
}
```

4. Cộng, trừ hai đa thức.

Các bước:

1. Đưa 2 đa thức về dạng chuẩn.
2. Lần lượt cộng / trừ từng đơn thức từ cuối lên đầu, sau đó lấy dạng chuẩn của nó.

```
cDaThuc cDaThuc::congKlan(cDaThuc B, const int& k)
{
    cDaThuc ketQua;

    cDaThuc A = chuanHoa();
    B = B.chuanHoa();

    int i = A.dt.size() - 1, j = B.dt.size() - 1;

    while (i >= 0 && j >= 0) {
        int bacA = A.dt[i].getBac();
        int bacB = B.dt[j].getBac();

        if (bacA > bacB) {
            ketQua.dt.push_back(A.dt[i]);
            --i;
        }
        else if (bacA < bacB) {
            ketQua.dt.push_back(cDonThuc(k * B.dt[j].getHeSo(), bacB));
            --j;
        }
        else {
            double heSo = A.dt[i].getHeSo() + k * B.dt[j].getHeSo();
            ketQua.dt.push_back(cDonThuc(heSo, bacA));
            --i;
            --j;
        }
    }

    while (i >= 0) {
        ketQua.dt.push_back(A.dt[i--]);
    }

    while (j >= 0) {
        B.dt[j].setHeSo(k * B.dt[j].getHeSo());
        ketQua.dt.push_back(B.dt[j--]);
    }

    return ketQua.chuanHoa();
}

cDaThuc cDaThuc::operator+(cDaThuc& dt)
{
    return congKlan(dt, 1);
}

cDaThuc cDaThuc::operator-(cDaThuc& dt)
{

```

```
    return congKLan(dt, -1);  
}
```

LUYỆN TẬP KẾT THÚC MÔN

Phương thức hằng:

`type name(par...) const {...};`

- Pt / đối tượng hằng chỉ có thể gọi pt hằng.

Thành phần bạn:

`friend class-name;`

`friend method-name(par...) {...};`

- B là bạn của A thì B không phải thành phần của A.

- B có thể truy cập vào private và protected của A.

Thành phần tĩnh:

`static type att-name;`

`static type method-name(par... {...});`

- Là những thành phần có giá trị chung cho toàn bộ các đối tượng của lớp.

- Được khởi tạo trước constructor.

- Có thể được truy xuất mà không thông qua tên đối tượng (. hoặc ->).

- Thuộc tính tĩnh là const thì luôn phải được khởi tạo giá trị trước.

Phương thức khởi tạo:

`class-name(par...) {...};`

`class-name(par...) : att(par)..., {...};`

- *Default constructor*: Không có tham số hoặc đều là tham số mặc định. Được compiler tự động khởi tạo khi dev không khởi tạo constructor.

- *Parameter constructor*: Có tham số.

- *Copy constructor*:

`class-name(const class-name& par) {...};`

được compiler tự động khởi tạo.

Phương thức hủy:

`~ class-name(par...) {...};`

- Được compiler tự động khởi tạo.

- Khi có thành phần con trở thì dev cần khởi tạo destructor để tránh sai sót quản lý bộ nhớ.

- Khi truyền tham số là đối tượng thì cần truyền theo cú pháp sau để tránh lãng phí bộ nhớ: `const class-name& parname`.

Nạp chồng toán tử ép kiểu:

Xét 2 toán hạng A (có kiểu cls) và toán hạng a (có kiểu std là tiêu chuẩn của C++). Cần thực hiện phép oprt giữa 2 toán hạng này.

Cách 1: Nạp chồng toán tử:

- Xử lý A + a:

`cls operator oprt(cls A);`

- Xử lý a + A:

`cls operator oprt(std a, cls A);`

- Xử lý A + a và a + A:

`friend cls operator oprt(cls A, cls B);`

Cách 2: Ép kiểu tường minh:

Gọi `cls(a) oprt A;`

Nạp chồng toán tử = đối với lớp có thành phần con trở:

```
String& String::operator = (const String &s) {  
    if (this != &s) {  
        delete[] p;  
        p = strdup(s.p); // Create new mem  
    }  
    return *this; // Return new mem  
}
```

Nạp chồng toán tử ++, --: <pre> className &operator ++(); // Tiền tố className operator ++(className); // Hậu tố (Cần tham số giả) className& className::operator ++() { // ... return *this; } className className::operator ++(className) { className obj = *this; // ... return obj; } </pre>	Nạp chồng toán tử >>, <<: <pre> friend istream& operator >> (istream &is, PhanSo &p); friend ostream& operator << (ostream &os, PhanSo p); istream& operator >> (istream &is, PhanSo &p) { is >> p.tu >> p.mau; return is; } ostream& operator << (ostream &os, PhanSo p) { os << p.tu; return os; } </pre>
Kế thừa: <pre> class sub : access-modifier parent {...}; </pre>	Liên kết động: Lưu trữ danh sách nhiều đối tượng <pre> vector <parent*> list; </pre> Cấp phát đối tượng: <pre> list[i] = new sub; </pre> Phương thức thuần ảo làm khuôn mẫu: <pre> type method (par...) = 0; </pre>

Đề thi cuối kỳ gồm có 3 câu tự luận:

1. **Câu 1 (2 điểm):** Câu hỏi lý thuyết.
2. **Câu 2 (3 điểm):** Hoàn thiện cài đặt các lớp đơn giản.
3. **Câu 3 (5 điểm):** Thiết kế và cài đặt lớp phức tạp qua đa hình và kế thừa.

I. Câu hỏi lý thuyết.

1. Giới thiệu OOP.

Câu 1:

- OOP là gì? Ưu điểm của OOP?

Giải:

- Là phong cách lập trình lấy đối tượng làm nền tảng. Mỗi đối tượng gồm có thuộc tính (đặc điểm) và hành động (phương thức).
- Ưu điểm: Dễ dàng tái sử dụng, mở rộng hệ thống.

Câu 1 [HK1 22-23]:

- Anh/Chị trình bày sự hiểu biết và cho ví dụ minh họa về khái niệm class (lớp) và object (đối tượng) trong lập trình hướng đối tượng. (2.0đ)

Giải:

- **Lớp** là một khuôn mẫu và **đối tượng** là một thể hiện dựa trên khuôn mẫu đó.
- Hay nói cách khác, lớp là một mô tả trừu tượng của một nhóm các đối tượng có cùng bản chất, ngược lại, mỗi đối tượng là một thể hiện cụ thể cho những mô tả trừu tượng đó.
- Mỗi lớp đều có 2 thành phần chính:
 - **Thuộc tính**: Thể hiện đặc điểm của đối tượng.
 - **Phương thức**: Thể hiện hành động của đối tượng.
- VD: `cSinhVien A` là một đối tượng của lớp `cSinhVien`.

Câu 1:

- Hàm bạn, lớp bạn là gì?

Giải:

- Hàm bạn / lớp bạn là hàm / lớp có quyền truy cập vào các thành phần private và protected của lớp đó, tương tự như là một thành phần của lớp đó, nhưng không phải thành phần của lớp.

Câu 1:

- Thành phần tĩnh là gì?

Giải:

- Giá trị của thành phần tĩnh được dùng chung cho tất cả các đối tượng của lớp đó (tương tự mối quan hệ giữa biến toàn cục và các hàm).

2. Phương thức khởi tạo và phương thức hủy.

Câu 1 a [HK2 23-24]:

- Khi định nghĩa một lớp đối tượng trong ngôn ngữ C++, trình biên dịch tự tạo lập những phương thức thiết lập hay còn gọi là phương thức khởi tạo (Constructor) nào? (1.0đ)
- Hãy trình bày đặc điểm của các loại phương thức thiết lập nêu trên (1.0đ).

Giải:

- Compiler tự động tạo ra *default constructor* và *copy constructor* khi người dùng không tự định nghĩa chúng.
- Đặc điểm:
 - **Default constructor**: Là constructor không tham số hoặc toàn bộ tham số đều là tham số mặc định, dùng để khởi tạo giá trị ban đầu các thuộc tính của đối tượng.
 - **Copy constructor**: Là constructor có tham số là một hằng tham chiếu, dùng để sao chép dữ liệu giữa các đối tượng cùng lớp.

Câu 1 [HK2 22-23 / HK1 23-24]: Trình bày về:

- Phương thức thiết lập (Constructor) và các dạng phương thức thiết lập.
- Phương thức hủy bỏ (Destructor) trong lập trình hướng đối tượng.
- Hãy nêu 3 đặc điểm khác nhau giữa phương thức thiết lập và phương thức hủy.

Giải:

- **Constructor**: Là phương thức có tên trùng với tên lớp, không có kiểu trả về, dùng để khởi tạo giá trị ban đầu các thuộc tính của đối tượng.
- Có 3 loại constructor:

Default constructor	• Là constructor không tham số. • Được compiler sinh nếu người dùng không định nghĩa bất kỳ constructor nào.
Parameter constructor	• Là constructor có tham số do người dùng định nghĩa.
Copy constructor	• Là constructor có tham số là một hằng tham chiếu. • Được compiler sinh mặc định nếu người dùng không tự định nghĩa. • Được gọi khi sao chép đối tượng này vào 1 đối tượng khác.

- **Destructor**: Là phương thức có tên trùng với tên lớp, không có kiểu trả về, ký hiệu có dấu ~ phía trước, dùng để giải phóng tài nguyên đã cấp phát khi đối tượng bị hủy (*nếu không dùng destructor thì bộ nhớ sẽ bị rò rỉ, bị tràn...*).
- **3 đặc điểm phân biệt constructor và destructor**:

Constructor	Destructor
Dùng để khởi tạo giá trị.	Dùng để giải phóng tài nguyên.
Được gọi khi đối tượng được tạo ra.	Được gọi khi đối tượng bị hủy.
Có thể có tham số.	Không thể có tham số.

3. Các tính chất của OOP.

Câu 1 b [HK2 22-23]:

- Phân biệt chuyển kiểu bằng phương thức thiết lập và chuyển kiểu bằng phép toán chuyển kiểu (1.5đ).
- Cho ví dụ minh họa (1.5đ).

Giải:

- Chuyển kiểu bằng constructor là gọi constructor để chuyển từ các kiểu cơ bản sang lớp.
- Chuyển kiểu bằng phép toán chuyển kiểu là ép các đối tượng thuộc lớp về các kiểu cơ bản.

VD: Có lớp:

```
class cPhanSo {  
    int a, b;  
public:  
    phanSo(int a = 1, int b = 1) : a(a), b(b) {};  
    operator float() {return float(a) / b;}  
}
```

Cho một cPhanSo A và int b.

- Nếu gọi A + phanSo(b) : Ép b từ int sang cPhanSo (chuyển kiểu bằng constructor).
- Nếu gọi float(A) + b : Ép A từ cPhanSo về float (chuyển kiểu bằng phép toán chuyển kiểu).

Câu 1 a b [HK2 21-22 / 17-18 / 15-16]:

- Hãy giải thích và cho ví dụ minh họa các khái niệm tính trừu tượng (Abstraction), Tính bao bọc (Encapsulation), tính kế thừa (Inheritance), tính đa hình (Polymorphism) của lập trình hướng đối tượng.

Giải:

- **Tính đóng gói:**
 - Là kết hợp các thành phần lại với nhau một đơn vị độc lập.
→ Giúp lớp dễ sử dụng và mở rộng.
→ Bảo vệ, che giấu các thành phần bên trong → Đảm bảo chương trình an toàn.
 - VD: Sử dụng các hàm getter, setter để lấy giá trị và sửa giá trị các thuộc tính của lớp thay vì lấy và sửa trực tiếp thuộc tính.
- **Tính kế thừa:**
 - Kế thừa là thừa hưởng lại các thành phần của một lớp khác.
→ Giúp lớp dễ tái sử dụng và mở rộng.
→ Dễ dàng xây dựng các lớp mới dựa trên một lớp đã có.
 - VD: Lớp hìnhVuong có thể kế thừa các thuộc tính và phương thức của lớp hìnhHoc.
- **Tính đa hình:**
 - Một thành phần có thể được định nghĩa tại nhiều lớp và có thể có những cài đặt khác nhau tại các lớp đó.

→ Gọi cùng một phương thức và có kết quả khác nhau tùy thuộc vào đối tượng gọi phương thức đó

- VD: Cùng là phương thức `keu()` nhưng mỗi loài động vật sẽ có một cách kêu khác nhau.

- **Tính trừu tượng:**

- Là tạo ra các lớp trừu tượng để *mô hình hóa* các đối tượng trong thế giới thực.
→ Giúp tập trung vào các khía cạnh quan trọng và ẩn đi các chi tiết không cần thiết.
- VD: Các lớp `cho`, `meo`, `cuu`,... có thể có một lớp trừu tượng không mẫu là `dongVat`.

Câu 1 a [HK2 19-20]:

- Hãy trình bày những đặc điểm của tính Đóng gói (encapsulation) trong lập trình hướng đối tượng. Trường hợp nào thì có thể vi phạm tính đóng gói? Cho ví dụ minh họa (1đ)

Giải:

- Là kết hợp các thành phần lại với nhau một đơn vị độc lập.
→ Giúp lớp dễ sử dụng và mở rộng.
→ Bảo vệ, che giấu các thành phần bên trong → Đảm bảo chương trình an toàn.
- Tính đóng gói có thể bị vi phạm khi người dùng cố gắng truy cập các thành phần `private` từ bên ngoài lớp hoặc các thành phần `protected` từ ngoài các lớp kế thừa.
- VD:

```
class Diem {
    private: int x, y;
};

int main() {
    Diem P;
    P.x = 0; // Sai
    return 0;
}
```

Câu 1 [HK1 17-18]:

- Cho biết ý nghĩa và mục đích của các hàm `get/set` trong một lớp (1đ)

Giải:

- **get:** Dùng để lấy giá trị thuộc tính.
- **set:** Dùng để thiết lập giá trị thuộc tính.
- Cả `get` và `set` đều thể hiện tính đóng gói.
→ Giúp lớp dễ sử dụng và mở rộng.
→ Bảo vệ, che giấu các thành phần bên trong → Đảm bảo chương trình an toàn.

Câu 1 b [HK2 19-20 / 16-17]:

- Hãy trình bày những ưu điểm của kế thừa trong việc lập trình hướng đối tượng và cho ví dụ minh họa (1đ)

Giải:

- Kế thừa là thừa hưởng lại các thành phần của một lớp khác.
 - Giúp lớp dễ tái sử dụng và mở rộng.
 - Dễ dàng xây dựng các lớp mới dựa trên một lớp đã có.
- VD:

```
class dongVat {...}

class cuu : public dongVat {...}
class meo : public dongVat {...}
```

→ Các lớp cuu, meo đều kế thừa từ lớp dongVat nên có thể sử dụng lại các thành phần protected và public (đối với kế thừa public) mà không cần phải định nghĩa lại từ đầu.

Câu 1 a [HK2 17-18]:

- Hàm thuần ảo là gì? Lớp trừu tượng là gì? Cho ví dụ minh họa. (1đ)

Giải:

- **Hàm thuần ảo:** Là hàm ảo có giá trị là 0, không có nội dung, dùng làm khuôn mẫu cho các lớp con kế thừa ghi đè phương thức.
- **Lớp trừu tượng:** Là lớp có ít nhất 1 hàm thuần ảo, dùng làm khuôn mẫu xây dựng các lớp con kế thừa.
- VD:

```
class dongVat {
public: virtual void keu() = 0;
};

class meo : public dongVat {
public: void keu();
};
```

→ Phương thức keu() ở lớp dongVat là hàm thuần ảo, và lớp dongVat là lớp trừu tượng.

→ Phương thức keu() ở lớp meo (kế thừa lớp dongVat) đã ghi đè lớp keu() thuần ảo.

Câu 1 b [HK2 23-24 / 16-17]:

- Phân biệt Lớp thông thường (class) và Lớp cơ sở trừu tượng (abstract class)? (1.0đ)
- Lớp cơ sở trừu tượng được cài đặt trong C++ như thế nào? (1.0đ)

Giải:

- **Lớp cơ sở trừu tượng:**
 - Được dùng làm khuôn mẫu khai báo các lớp con kế thừa nó, không thể khai báo trực tiếp đối tượng thuộc nó.
 - Có ít nhất 1 phương thức thuần ảo để được các lớp con ghi đè.
- **Lớp thông thường:**
 - Được dùng làm khuôn mẫu khai báo các đối tượng thuộc nó.
 - Có thể có đầy đủ các phương thức và thực hiện được đầy đủ chức năng.
- Lớp cơ sở trừu tượng có ít nhất 1 phương thức thuần ảo (*là phương thức ảo không có nội dung và được gán là 0*):
`virtual data-type method-name (par...) = 0;`

Câu 1 a [HK2 18-19]:

- Phân biệt khái niệm overload (tải chồng) và override (ghi đè) (1đ)

Giải:

- **Nạp chồng:** Là định nghĩa nhiều phương thức cùng tên nhưng khác nhau về số lượng tham số hoặc kiểu dữ liệu.
- **Ghi đè:** Là định nghĩa lại (giống nhau về tên, danh sách tham số, kiểu dữ liệu) các phương thức đã được định nghĩa ở lớp cha (trong kế thừa). Một lớp cha có thể có nhiều lớp con kế thừa, tuy nhiên các phương thức ở lớp cha có thể không phù hợp với một số lớp con nên các lớp con đó cần ghi đè lại phương thức cho phù hợp.

Câu 1 b [HK2 18-19 / 15-16 / 14-15]:

- Phân biệt các kiểu kế thừa private, protected và public. (1đ)

Giải:

- **Kế thừa private:** Các thành phần protected và public của cha sẽ trở thành private đối với con.
- **Kế thừa protected:** Các thành phần protected và public của cha sẽ trở thành protected đối với con.
- **Kế thừa public:** Các thành phần protected và public của cha sẽ trở thành protected và public tương ứng đối với con.
- Các thành phần private ở cha thì vẫn là private đối với con (không truy xuất được).

Câu 1:

- Phân biệt liên kết động và liên kết tĩnh.

Giải:

- **Liên kết tĩnh** là gọi một phương thức được xác định tại thời điểm biên dịch, không dùng phương thức ảo.
- **Liên kết động** là gọi một phương thức được xác định khi chạy chương trình (runtime), cần dùng phương thức ảo.

II. Hoàn thiện cài đặt lớp đơn giản.

Câu 2 [HK1 22-23].

Xây dựng class IntArr để hàm main hoạt động đúng như mong đợi.

```
class IntArr {
private:
    int count; //tổng số lượng phần tử có trong values
    int* values; //mảng các số nguyên đang có trong đối tượng hiện tại
public:
    //Sinh viên bổ sung đầy đủ các thành phần cần thiết để hàm main hoạt động như
    mong đợi
};

int main() {
    IntArr l1; //tạo mảng không chứa bất kì phần tử nào

    IntArr l2(3,2); // tạo một mảng với 3 phần tử, tất cả phần tử đều có giá trị là
    2
    IntArr l3(2); //tạo một mảng với 2 phần tử, tất cả phần tử đều có giá trị là 0

    IntArr l4 = l2.concat(l3); //tạo ra một IntArr mới có nội dung là kết quả của
    việc nối các phần tử l3 vào cuối các phần tử của l2 theo thứ tự

    l2.push(3); //thêm số 3 vào cuối danh sách trong đối tượng l2

    cin >> l2; //Xoá các giá trị hiện có trong l2 và cho phép người dùng nhập số
    Lượng phần tử mới và giá trị các phần tử mới vào l2 (cần xoá các vùng nhớ không sử
    dụng nếu có)

    cout << l2; //in ra các số nguyên có trong danh sách

    //Khi vượt quá phạm vi sử dụng cần huỷ tất cả các vùng nhớ được cấp phát cho
    các values của IntArr

    return 0;
}
```

Giải:

Xây dựng constructor:

```
IntArr(int count = 0, int value = 0) : count(count), values(nullptr) {
```

```

    if (count > 0) {
        values = new int[count];
        for (int i = 0; i < count; i++)
            values[i] = value;
    }
}

```

Các phương thức mở rộng danh sách:

```

IntArr concat(IntArr B) {
    IntArr ans = IntArr(this->count + B.count);

    for (int i = 0; i < this->count; i++)
        ans.values[i] = this->values[i];

    for (int i = 0; i < B.count; i++)
        ans.values[this->count + i] = B.values[i];

    return ans;
}

void push(int x) {
    *this = this->concat(IntArr(1, x));
}

```

Các phương thức nhập - xuất:

```

IntArr(int count, int* values) : count(count), values(values) {}

IntArr& operator = (const IntArr& A) {
    if (this != &A) {
        delete[] values;

        count = A.count;
        if (count > 0) {
            values = new int[count];
            for (int i = 0; i < count; ++i)
                values[i] = A.values[i];
        } else
            values = nullptr;
    }

    return *this;
}

friend istream& operator >> (istream& is, IntArr& A) {
    int* temp = nullptr;

    int n;
    is >> n;

    if (n > 0) {
        temp = new int[n];
        for (int i = 0; i < n; i++)
            is >> temp[i];
    }

    A = IntArr(n, temp);
}

```

```

        delete[] temp;

        return is;
    }

    friend ostream& operator << (ostream& os, IntArr A) {
        for (int i = 0; i < A.count; i++)
            os << A.values[i] << ' ';
        return os;
    }
}

```

Chú ý: Không nên gán thẳng 2 con trỏ cho nhau mà nên sao chép từng phần tử một.

Câu 2 [HK2 23-24].

Cho đoạn chương trình tính toán với lớp đối tượng ngày tháng năm (cNgày) như sau. Sinh viên định nghĩa lớp cNgày thích hợp để chương trình không bị lỗi biên dịch và chạy đúng. Lưu ý rằng không được chỉnh sửa hàm main.

```

void main()
{
    cNgày ng1;                // ng1 sẽ có giá trị ngày 1 tháng 1 năm 1
    cNgày ng2(2017, 1, 7);    // ng2 sẽ có giá trị ngày 7 tháng 1 năm 2017
    cin>>ng1;
    int kq = ng1.SoSanh(ng2);
    if(kq == -1)
        cout << "Ngày 1 trước ngày 2" << endl;
    else if (kq == 0)
        cout << "Ngày 1 và ngày 2 là cùng 1 ngày" << endl;
    else
        cout << "Ngày 1 sau ngày 2" << endl;
}

```

Giải:

Chú ý:

- Constructor nhận tham số theo thứ tự năm, tháng, ngày.
- Khi so sánh 2 cNgày, ưu tiên so sánh theo thứ tự năm, tháng, ngày.

```

#include <iostream>

using namespace std;

class cNgày {
    int ngay, thang, nam;
public:
    cNgày(int nam = 1, int thang = 1, int ngay = 1) : ngay(ngay),
        thang(thang), nam(nam) {}

    friend istream& operator >> (istream& is, cNgày& x) {
        is >> x.ngay >> x.thang >> x.nam;
        return is;
    }
}

```

```

int SoSanh(cNgay x) {
    if (this->nam < x.nam || this->thang < x.thang || this->ngay < x.ngay)
        return 2;
    else if (this->nam == x.nam && this->thang == x.thang && this->ngay == x.ngay)
        return 0;
    else
        return -1;
}
};

```

Câu 2 [HK2 22-23].

Định nghĩa class Money để mô tả thông tin về tiền lưu hành ở châu Âu như sau:

```

class Money
{
private:
    int iEuro;
    int iCent;
};

```

Cho biết 1 euro = 100 cents. Trong lớp Money, hãy định nghĩa các phương thức:

- Toán tử >> (1đ)
- Toán tử ++ (tiếp đầu ngữ hoặc tiếp vị ngữ đều được). Ví dụ: Số tiền có 100 euros 98 cents, sau khi thực hiện toán tử ++ thì số tiền là 100 euros 99 cents (1đ)

Giải:

a.

```

istream& operator >> (istream& is, Money& m) {
    is >> m.iEuro >> m.iCent;
    if (m.iCent >= 100) {
        m.iEuro += m.iCent / 100;
        m.iCent = m.iCent % 100;
    }
    return is;
}

```

b.

```

Money operator ++ () {
    this->iCent++;
    if (this->iCent == 100) {
        this->iEuro++;
        this->iCent = 0;
    }
    return *this;
}

Money operator ++ (Money) {
    Money temp = *this;
    ++(*this);
    return temp;
}

```

Câu 2 [HK2 21-22].

Một đơn thức theo biến x được xác định bởi hệ số và số mũ, $P(x) = a \cdot x^b$. Hãy xây dựng lớp biểu diễn khái niệm đơn thức cho phép thực hiện các yêu cầu sau:

- Khởi tạo đơn thức khi biết hệ số và số mũ.
- Xác định giá trị của đơn thức khi biết giá trị của x .
- Phép toán cộng (+) để cộng hai đơn thức cùng bậc.

Giải:

```
#pragma once
#include <iostream>
#include <math.h>

using namespace std;

class cDonThuc {
private:
    float fHeSo, fMu;
public:
    cDonThuc(float fHeSo = 1, float fMu = 1) : fHeSo(fHeSo), fMu(fMu) {}

    float tinhGT(float x) {
        return fHeSo * pow(x, fSoMu);
    }

    friend cDonThuc operator + (cDonThuc x) {
        if (this->fMu != x.fMu)
            cout << "Khong cong duoc.";
        else
            return cDonThuc(this->fHeSo + x.fHeSo, this->fMu + x.fMu);
    }
};
```

III. Thiết kế và cài đặt lớp phức tạp qua đa hình và kế thừa.

Hướng dẫn làm bài:

- Đọc yêu cầu trước, mô tả sau để tìm kiếm những thông tin cần thiết.
- Chỉ thiết kế các thuộc tính và phương thức như đề yêu cầu, không cần thiết kế toàn bộ phương thức như nhập, xuất,...
- Không cần thiết kế các tính năng gọi nhắc khi nhập dữ liệu.

Quy tắc thiết kế:

- Những thành phần chung giữa các lớp thì ở lớp cha.
- Luôn dọn dẹp trước khi khởi tạo / khai báo mới.
- Luôn kiểm tra tất cả mọi trường hợp.

Mẹo tiết kiệm thời gian cài đặt:

- Không cần khai báo nguyên mẫu hàm / tách file.
- Sử dụng vector và các cấu trúc / thuật toán sẵn có.
- Sử dụng `#include <bits/stdc++.h>`.
- Sử dụng cấu trúc `for (auto elm : list)` để xuất dữ liệu.
- Sử dụng cấu trúc `for (auto& elm : list)` để nhập dữ liệu (đối với con trỏ thì không dùng cách này).
- Sử dụng cấu trúc toán tử ba ngôi `condition ? value-if-true : value-if-false`.

Câu 3 [HK1 24-25].

Chiến đấu (battle) là một thành phần không thể thiếu với các thể loại game RPG. Trong một trận chiến với boss (boss battle – hay thường được gọi nôm na là đánh trùm), một nhóm nhân vật do người chơi điều khiển sẽ chiến đấu với trùm – là một nhân vật do máy tính điều khiển. Cả trùm (boss) và nhân vật do người chơi điều khiển (player character) đều được xem là một nhân vật trong game với một số thuộc tính chung và những đặc trưng riêng.

Mọi nhân vật (kể cả boss) đều có những đặc tính chung sau:

- Cấp độ (level): từ 1 đến 5
- Chỉ số tấn công cơ bản (base attack)
- Chỉ số phòng thủ cơ bản (base defense)
- Chỉ số sát thương (damage)
- Máu hiện tại (hay được gọi tắt là HP - health point): nếu HP bằng 0 thì nhân vật xem như bị loại bỏ khỏi trận chiến
- Máu tối đa (max HP): cho biết lượng máu tối đa nhân vật có thể có tại một thời điểm, cũng phụ thuộc vào lớp nhân vật và cấp độ nhân vật
- Một hoặc nhiều các hiệu ứng bất lợi: đầu độc, choáng, v.v., tùy thuộc vào diễn biến của trận chiến.

Mọi nhân vật (kể cả boss) đều thuộc 1 trong 3 lớp sau:

1. **Ranger:** thường đứng tuyến sau, chủ yếu để rửa máu đối phương, chỉ số sát thương dưới trung bình, máu và phòng thủ cũng dưới trung bình. Các chỉ số cơ bản của Ranger theo cấp độ từ 1 đến 5 như bảng sau:

Cấp độ	1	2	3	4	5
Base Attack	60	75	90	120	140
Base Defense	30	40	50	60	70
Damage	8	15	30	45	70
Max Health	50	65	80	95	110

2. **Tanker:** thường là nhân vật chịu đòn đánh của đối phương để đồng đội rảnh tay thi triển chiêu thức, lớp nhân vật này có chỉ số phòng thủ cao, máu cao nhưng tấn công và sát thương thì ở mức trung bình hoặc dưới trung bình. Các chỉ số cơ bản của Tanker theo cấp độ từ 1 đến 5 như bảng sau:

Cấp độ	1	2	3	4	5
Base Attack	70	92	104	130	150
Base Defense	20	30	40	60	80
Damage	4	7	11	19	27
Max Health	30	40	50	60	70

3. **Fighter:** đứng giữa, chỉ số tấn công, sát thương và phòng thủ ở mức trung bình. Các chỉ số cơ bản của Fighter theo cấp độ từ 1 đến 5 như bảng sau:

Cấp độ	1	2	3	4	5
Base Attack	70	92	104	130	150
Base Defense	20	30	40	60	80
Damage	4	7	11	19	27
Max Health	30	40	50	60	70

Luật tấn công:

- Khi một nhân vật A tấn công nhân vật B, hệ thống sẽ lấy ngẫu nhiên chỉ số tấn công của đòn đánh của A trong khoảng từ **$0.8 * BaseAttack$ đến $1.1 * BaseAttack$** .
- Tương tự, hệ thống sẽ lấy ngẫu nhiên chỉ số phòng thủ của B trong khoảng từ **$0.8 * BaseDefense$ đến $1.3 * BaseDefense$** .
- Nếu chỉ số tấn công của A > chỉ số phòng thủ của B thì đòn đánh được xem là đánh trúng. Máu (HP) của B sẽ bị trừ bằng chỉ số sát thương hiện tại của A. Nếu máu của B về 0 thì B bị chết (loại bỏ khỏi trận chiến).
- Nếu chỉ số tấn công của A nằm đúng trong khoảng từ **$1.05 * BaseAttack$ đến $1.10 * BaseAttack$** , đòn đánh được xem là chí mạng (critical).
- Đòn đánh chí mạng sẽ gây ngẫu nhiên một trong số 5 hiệu ứng bất lợi sau:
 - **Choáng**: nhân vật sẽ mất hoàn toàn lượt.
 - **Đầu độc**: trong suốt 5 lượt, mỗi lần bắt đầu lượt là nhân vật bị mất 10% max HP.
 - **Chậm**: trong lượt tiếp theo, nhân vật bị giảm 25% BaseAttack và BaseDefense.
 - **Yếu**: trong lượt tiếp theo, nhân vật bị giảm 25% sát thương.
 - **Mất trí**: trong lượt tiếp theo, nhân vật sẽ tấn công ngẫu nhiên 1 nhân vật cùng phe của mình. Nếu là boss thì boss tự đánh vào chính nó.

Trận đánh diễn ra trong nhiều vòng. Trong một vòng thì các nhân vật của người chơi sẽ được đi trước, rồi sau đó tới lượt boss đi.

Khi một nhân vật tới lượt của mình, nhân vật đó có thể chọn 1 trong 3 hành động:

- **Tấn công**: chọn 1 nhân vật phía đối phương (hoặc nhân vật ngẫu nhiên phía mình nếu bị vào trạng thái mất trí).
- **Phòng thủ**: để tăng chỉ số BaseDefense lên 25%. Nếu không nhận sát thương từ đòn đánh tới đối phương, sẽ được phục hồi 10% lượng máu tối đa ở cấp độ hiện tại của lớp nhân vật.

Quy luật của boss:

- Tới lượt của boss, boss sẽ chọn hành động ngẫu nhiên: 70% tấn công và 30% phòng thủ.
- Nếu boss chọn tấn công thì boss sẽ tiếp tục chọn ngẫu nhiên mục tiêu: 50% tanker, 30% fighter và 20% ranger. Nếu không có nhân vật tanker thì thay vì chọn tanker sẽ chọn fighter, và ngược lại.

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình), thiết kế sơ đồ chi tiết các lớp đối tượng (lý thuyết) và xây dựng chương trình thực hiện các yêu cầu sau:

- (1.0đ) Cho phép người dùng khởi tạo một trận chiến gồm tối đa 5 nhân vật người chơi và boss. Với mỗi nhân vật, người chơi chỉ cần chọn cấp độ và lớp nhân vật. Khi mới vào trận chiến, mặc định mọi nhân vật đều có HP = max HP ứng với cấp độ của nhân vật.
- (0.5đ) In thông tin chi tiết các nhân vật có trong trận đánh bao gồm: cấp độ, BaseAttack, BaseDefense, Damage, max HP và HP.

- c) (2.5đ) Cài đặt logic cho diễn biến 1 trận đánh như mô tả ở trên cho đến khi kết thúc. Trận chiến kết thúc khi boss chết hoặc tất cả nhân vật của người chơi đều chết.

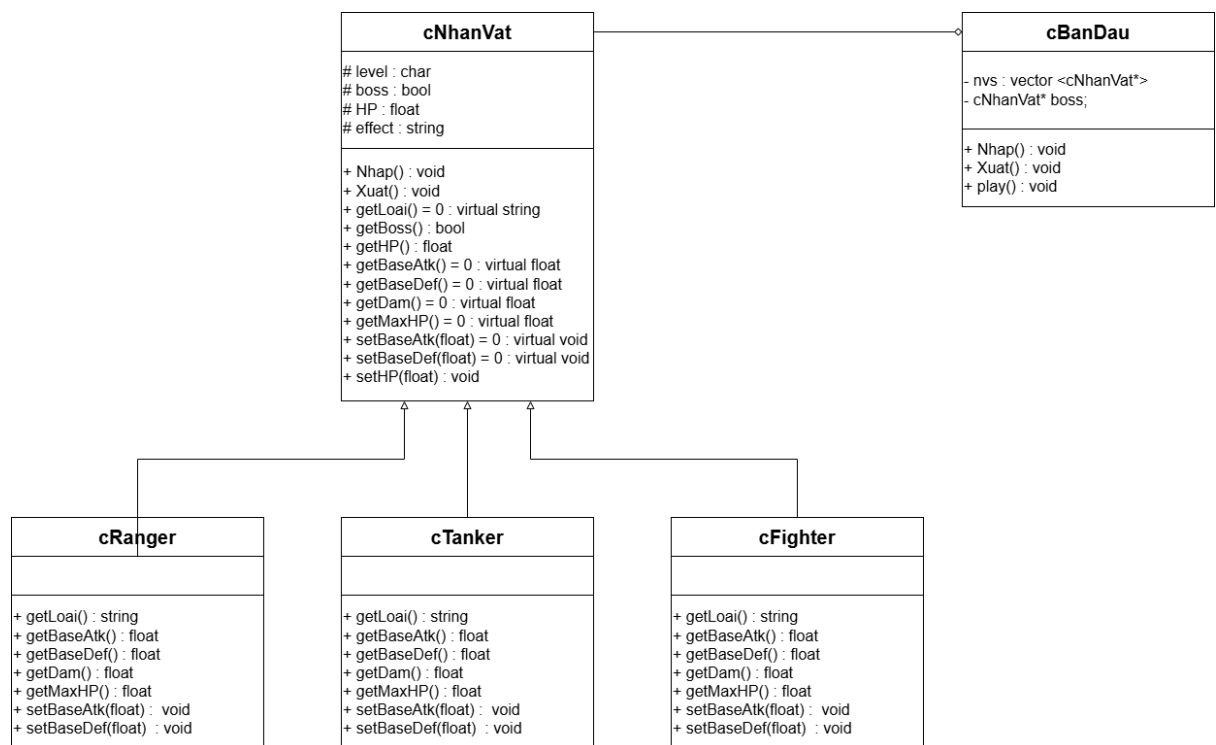
Giả định: Hệ thống đã có sẵn hàm `randFloat()` để trả ra một số double trong khoảng từ 0 đến 1 và `randInt(from, to)` để trả ra một số nguyên ngẫu nhiên trong khoảng từ `from` đến `to`.

Bài làm:

Tóm tắt yêu cầu chương trình:

- Nhập danh sách n nhân vật. Mỗi nhân vật:
 - Có 2 phân loại chia theo phe: Boss, nhân vật người chơi.
 - Có 3 phân loại chia theo yêu cầu: Ranger, Tanker, Fighter.
 - Có các thuộc tính chung: level, HP, effect.
 - Có các thuộc tính riêng theo từng phân loại chia theo yêu cầu: Base attack, base defense, damage, max HP.
- Khi nhập: Chỉ cần nhập level và phân loại chia theo yêu cầu.
- Khi xuất: Chỉ cần xuất level, Base Attack, Base Defense, Damage, max HP và HP.
- Cài đặt logic diễn biến trận đấu.

Sơ đồ lớp:



Câu 3 [HK2 23-24].

LƯU Ý: Các thông tin trong đề chỉ **mô phỏng** các thông tin y học với mục tiêu để sinh viên vận dụng kiến thức lập trình hướng đối tượng. Do vậy, các thông tin trong đề **KHÔNG** nhất thiết phải đúng với các thông tin y học hiện tại. Sinh viên cần bám sát các mô tả trong đề thi để làm bài, không vận dụng thêm bất kỳ kiến thức y khoa, phổ thông, thời sự nào khác để làm bài.

Có rất nhiều chủng **virus** sống trên các vật chủ là các sinh vật sống khác kể cả loài người. Khi nhiễm lên vật chủ, một loại virus có thể gây hậu quả tử tử vong, các triệu chứng nặng, các triệu chứng nhẹ hoặc không triệu chứng gì tùy thuộc vào **khả năng miễn dịch** của vật chủ.

- **Khả năng miễn dịch** của vật chủ đối với các loại virus ở ba mức: thấp, trung bình và cao. Nó sẽ ảnh hưởng đến xác suất gặp các triệu chứng nặng, nhẹ, không có triệu chứng hoặc xác suất tử vong như bảng sau:

	Không triệu chứng	Triệu chứng nhẹ	Triệu chứng nặng	Tử vong
Cao	50%	35%	15%	50%
Trung bình	10%	40%	50%	70%
Thấp	5%	15%	80%	100%

- Xác suất tử vong ở bảng trên được hiểu là xác suất dựa trên xác suất tử vong trung bình của từng loại virus. Chẳng hạn có một virus có xác suất tử vong trung bình là 50% và nhiễm lên vật chủ có khả năng miễn dịch trung bình thì xác suất tử vong cuối cùng sẽ là : $70\% \times 50\% = 35\%$.

Trong những năm 2019 đến nay, dòng virus Corona mới - tên chính thức là **SARS-CoV-2** xuất hiện đã gây ra bao hậu quả tàn khốc cho khắp thế giới với hơn chục triệu người nhiễm bệnh và hơn nửa triệu người tử vong. Triệu chứng nhẹ của Covid-19 giống như cảm cúm thông thường như sốt, ho, mất vị giác trong vài ngày rồi khỏi. Triệu chứng nặng của Covid-19 bao gồm sốt cao, ho khan, khó thở và đôi lúc kèm theo đau đầu dữ dội. Tuy nhiên, Covid-19 gây tử vong với xác suất trung bình khá thấp là **3-5%**.

Một dòng virus chết chóc khác là **Ebola**, có khả năng gây tử vong với tỷ lệ trung bình lên đến **50%**. Triệu chứng nhẹ của Ebola trong vòng 2-3 tuần đầu bao gồm sốt, đau họng, đau cơ và đau đầu. Sau đó, nếu không may, người bệnh có thể gặp các triệu chứng nặng như bị nôn mửa, tiêu chảy và nặng nhất là xuất huyết cả ngoài lẫn bên trong.

Virus **HIV** tấn công hệ miễn dịch của con người, làm suy yếu nó để các virus/bệnh khác có cơ hội trở dậy. HIV chỉ gây các triệu chứng nhẹ giống cảm sốt trong thời gian ủ bệnh (từ 3 đến 5 năm). Nếu không may mắn được miễn dịch, đến giai đoạn AIDS, giai đoạn hệ miễn dịch đã bị suy yếu đáng kể, người bệnh sẽ gặp các triệu chứng nặng như mệt mỏi cực độ không giải thích được, sưng hạch kéo dài, lở loét, viêm phổi, tiêu chảy nặng và sau đó là khả năng tử vong trung bình cao đến **90%**.

Hậu quả do virus gây ra (bao gồm cả tử vong) khi bị nhiễm có thể được giảm bớt bằng cách tiêm vaccine phòng ngừa. Nếu tiêm vaccine sẽ làm thay đổi xác suất gặp triệu chứng nặng/nhẹ/không triệu chứng/tử vong như sau (theo chiều hướng có lợi):

	Không triệu chứng	Triệu chứng nhẹ	Triệu chứng nặng	Tử vong
Cao	70%	25%	5%	40%
Trung bình	20%	50%	30%	60%
Thấp	10%	40%	50%	80%

1. Áp dụng kiến thức Lập trình hướng đối tượng (kế thừa, đa hình), vẽ sơ đồ chi tiết các lớp đối tượng được mô tả trong đề bài (1.5đ).

Viết chương trình mô phỏng thực nghiệm dịch tễ học thực hiện các yêu cầu sau:

2. Nhập danh sách N vật chủ ($N \leq 10000$). Mỗi vật chủ được gán mã số (là một chuỗi) để nhận diện theo dõi. Sau đó, cho từng vật chủ trong danh sách nhiễm tất cả 3 chủng virus kể trên (Covid-19, Ebola, HIV). (1.5đ)
3. In ra mã các vật chủ và mô tả các triệu chứng của từng vật chủ (nếu có), cũng như cho biết vật chủ có bị tử vong hay không. (1.0đ)
4. Nhập danh sách M vật chủ mới (trong đó $M = N$ trước đó). Tiêm vaccine cho cả 3 loại virus. Cho từng vật chủ nhiễm cả 3 loại virus. In ra số lượng vật chủ gặp triệu chứng nặng (của bất kỳ virus nào bị nhiễm), số lượng vật chủ tử vong. (1.0đ)

LƯU Ý: Giả định đã có hàm `double RandomDouble()` để phát sinh một số double ngẫu nhiên trong khoảng từ **0 đến 1.0**.

Bài làm:

Tóm tắt yêu cầu chương trình:

- Nhập N vật chủ (mỗi vật chủ có ID riêng là chuỗi) bị nhiễm tất cả 3 chủng virus.
- Nhập thêm N vật chủ, tiêm vaccine trước rồi mới cho nhiễm 3 loại virus.
- Kiểm tra và mô tả triệu chứng từng vật chủ, bao gồm xác nhận vật chủ có tử vong không.

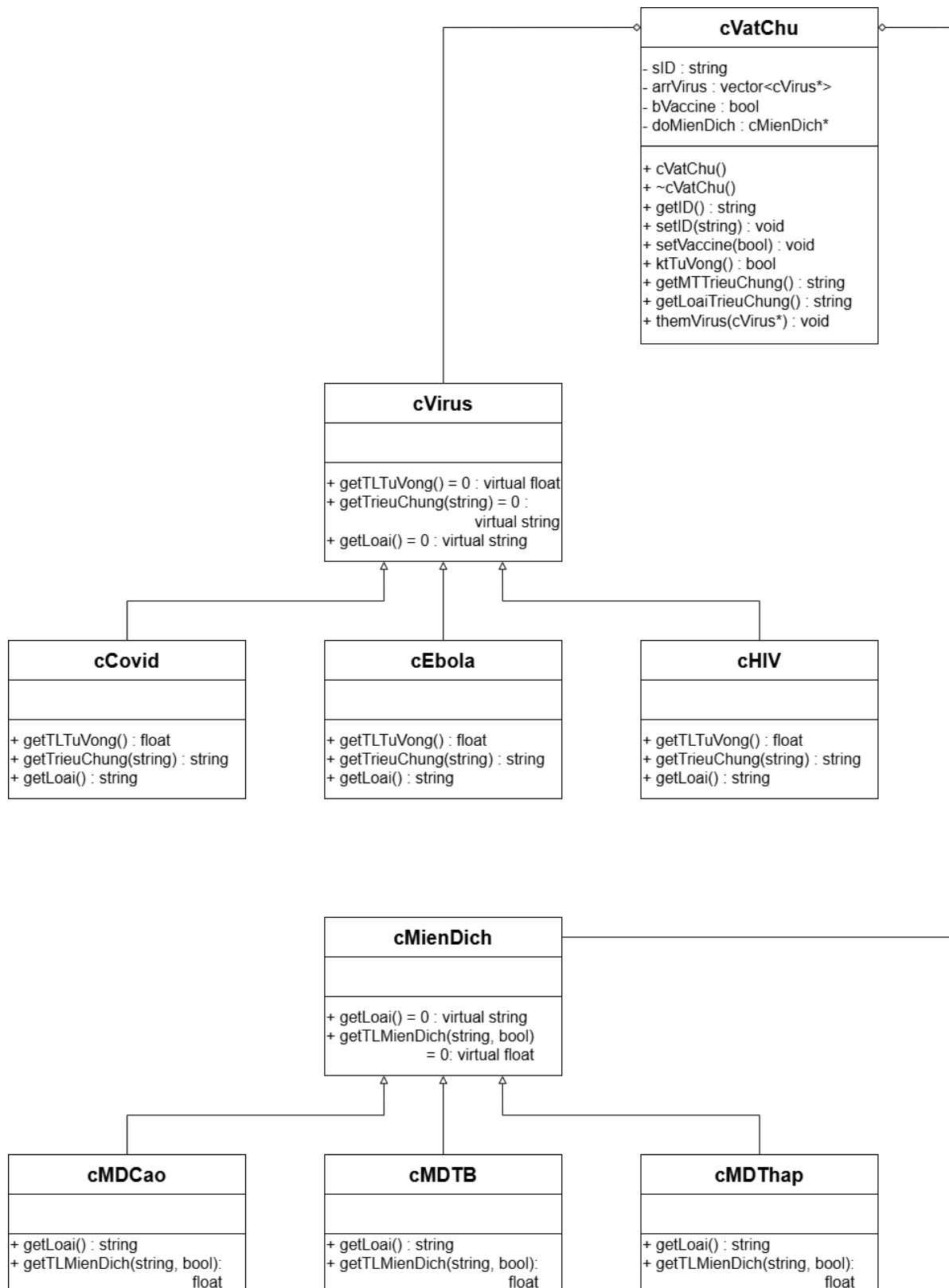
Một số nhận xét:

- Sinh một số thực ngẫu nhiên từ a đến b :

$$a + \text{RandomDouble}().(b - a)$$
- Cho một sự kiện A có xác suất xảy ra là k . Sự kiện A sẽ thật sự xảy ra khi:

$$\text{RandomDouble}() \leq k$$

Sơ đồ lớp:



Khai báo các lớp:

```
cVirus.h
#pragma once
#include <string>

using std::string;

class cVirus {
public:
    virtual float getTLTuVong() = 0;
    virtual string
getTrieuChung(string) = 0;
    virtual string getLoai() = 0;
};
```

```
cCovid.h
#pragma once
#include "cVirus.h"

class cCovid : public cVirus {
public:
    float getTLTuVong();
    string getTrieuChung(string);
    string getLoai();
};
```

```
cEbola.h
#pragma once
#include "cVirus.h"

class cEbola : public cVirus {
public:
    float getTLTuVong();
    string getTrieuChung(string);
    string getLoai();
};
```

```
cHIV.h
#pragma once
#include "cVirus.h"

class cHIV : public cVirus {
public:
    float getTLTuVong();
    string getTrieuChung(string);
    string getLoai();
};
```

```
cMienDich.h
#pragma once
#include <string>

using namespace std;

class cMienDich {
public:
    virtual ~cMienDich() = default;
    virtual string getLoai() = 0;
    virtual float getTLMienDich(string,
bool) = 0;
};
```

```
cMDCao.h
#pragma once
#include "cMienDich.h"

class cMDCao : public cMienDich {
public:
    string getLoai();
    float getTLMienDich(string, bool);
};
```

```
cMDTB.h
#pragma once
#include "cMienDich.h"

class cMDTB : public cMienDich {
public:
    string getLoai();
    float getTLMienDich(string, bool);
};
```

```

cMDThap.h
#pragma once
#include "cMienDich.h"

class cMDThap : public cMienDich {
public:
    string getLoai();
    float getTLMienDich(string, bool);
};

```

```

cVatChu.cpp
#pragma once
#include <vector>
#include <string>
#include "cVirus.h"
#include "cMienDich.h"
#include "cMDCao.h"
#include "cMDTB.h"
#include "cMDThap.h"

using namespace std;

class cVatChu {
private:
    string sID;
    vector<cVirus*> arrVirus;
    bool bVaccine;
    cMienDich* doMienDich;
public:
    cVatChu();
    ~cVatChu();
    string getID();
    void setID(const string&);
    void setVaccine(bool);
    bool ktTuVong();
    string getMTTrieuChung();
    string getLoaiTrieuChung();
    void themVirus(cVirus*);
};

```

Định nghĩa các phương thức:

```
cCovid.cpp
#include "cCovid.h"

float cCovid::getTLTuVong() {
    return 3 + RandomDouble() * 2;
}

string cCovid::getTrieuChung(string mucDo) {
    if (mucDo == "Nhe")
        return "Sot, ho, mat vi giac";
    else if (mucDo == "Nang")
        return "Sot, ho khan, kho tho";
    else
        return "Khong co";
}

string cCovid::getLoai() {
    return "Covid";
}
```

```
cEbola.cpp
#include "cEbola.h"

float cEbola::getTLTuVong() {
    return 0.5;
}

string cEbola::getTrieuChung(string mucDo) {
    if (mucDo == "Nhe")
        return "Sot, dau hong, dau co, dau dau";
    else if (mucDo == "Nang")
        return "Non mua, tieu chay, sot xuất huyết";
    else
        return "Khong co";
}

string cEbola::getLoai() {
    return "Ebola";
}
```

```
cHIV.cpp
#include "cHIV.h"

float cHIV::getTLTuVong() {
    return 0.9;
}

string cHIV::getTrieuChung(string mucDo) {
    if (mucDo == "Nhe")
        return "Cam sot";
}
```

```
cMDCao.cpp
#include "cMDCao.h"

string cMDCao::getLoai() {
    return "Cao";
}

float cMDCao::getTLMienDich(string mucDo, bool vaccine) {
    if (mucDo == "Khong")
        return vaccine ? 0.7 : 0.5;
    else if (mucDo == "Nhe")
        return vaccine ? 0.25 : 0.35;
    else if (mucDo == "Nang")
        return vaccine ? 0.05 : 0.15;
    else
        return vaccine ? 0.4 : 0.5;
}
```

```
cMDTB.cpp
#include "cMDTB.h"

string cMDTB::getLoai() {
    return "Trung binh";
}

float cMDTB::getTLMienDich(string mucDo, bool vaccine) {
    if (mucDo == "Khong")
        return vaccine ? 0.2 : 0.1;
    else if (mucDo == "Nhe")
        return vaccine ? 0.5 : 0.4;
    else if (mucDo == "Nang")
        return vaccine ? 0.3 : 0.5;
    else
        return vaccine ? 0.6 : 0.7;
}
```

```
cMDThap.cpp
#include "cMDThap.h"

string cMDThap::getLoai() {
    return "Thap";
}

float cMDThap::getTLMienDich(string mucDo, bool vaccine) {
    if (mucDo == "Khong")
        return vaccine ? 0.1 : 0.05;
    else if (mucDo == "Nhe")
        return vaccine ? 0.4 : 0.15;
    else if (mucDo == "Nang")
        return vaccine ? 0.5 : 0.8;
}
```

```

else if (mucDo == "Nang")
    return "Met moi, sung hach, lo loet, viem phoi";
else
    return "Khong co";
}

string cHIV::getLoai() {
    return "HIV";
}

```

```

else
    return vaccine ? 0.8 : 1.0;
}

```

```

cVatChu.cpp
#include "cVatChu.h"

cVatChu::cVatChu() {
    int vt = RandomDouble() * 3;

    if (vt == 0)
        doMienDich = new cMDCao;
    else if (vt == 1)
        doMienDich = new cMDTB;
    else
        doMienDich = new cMDThap;

    bVaccine = false;
}

cVatChu::~cVatChu() {
    delete doMienDich;
    for (auto v : arrVirus)
        delete v;
}

string cVatChu::getID() {
    return sID;
}

void cVatChu::setID(const string& id) {
    sID = id;
}

void cVatChu::setVaccine(bool vaccine) {
    bVaccine = vaccine;
}

bool cVatChu::ktTuVong() {
    for (auto vr : arrVirus)
        if (RandomDouble() <=
            doMienDich->getTLMienDich("Tu vong", bVaccine) * vr->getTLTuVong())
            return true;
    return false;
}

string cVatChu::getLoaiTrieuChung() {
    float r = RandomDouble();
    float p0 = ;
    if (r < doMienDich->getTLMienDich("Khong", bVaccine))
        return "Khong";
}

```

```

    r -= p0;
    float p1 = doMienDich->getTLMienDich("Nhe", bVaccine);
    if (r < p1)
        return "Nhe";

    r -= p1;
    float p2 = doMienDich->getTLMienDich("Nang", bVaccine);
    if (r < p2)
        return "Nang";

    return ktTuVong() ? "Da tu vong" : "Khong";
}

string cVatChu::getMTTrieuChung() {
    string MTTrieuChung;
    string loai;

    for (auto vr : arrVirus) {
        loai = getLoaiTrieuChung();
        if (loai == "Nhe")
            MTTrieuChung += vr->getTrieuChung("Nhe") + "\n";
        else if (loai == "Nang")
            MTTrieuChung += vr->getTrieuChung("Nang") + "\n";
        else if (loai == "Khong")
            MTTrieuChung += "Khong co trieu chung\n";
        Else
            MTTrieuChung += "Da tu vong\n";
    }
    return MTTrieuChung;
}

void cVatChu::themVirus(cVirus* vr) {
    arrVirus.push_back(vr);
}

```

Thân chương trình:

```
Bai03.cpp
#include "cVatChu.h"
#include <iostream>

using namespace std;

int main() {
    // Cau 1

    int n;
    cin >> n;
    vector<cVatChu*> vatChus;

    for (int i = 0; i < n; ++i) {
        string id;
        cin >> id;

        cVatChu* tam;

        tam->setID(id);
        tam->setVaccine(false);

        tam->themVirus(new cCovid());
        tam->themVirus(new cEbola());
        tam->themVirus(new cHIV());

        vatChus.push_back(tam);
    }

    // Cau 3

    for (auto vc : vatChus) {
        cout << vc->getID() << "\n";
        if (vc->ktTuVong()) cout << "Da tu vong\n";
        else cout << vc->getMTTrieuChung();
    }

    // Cau 4

    for (auto vc : vatChus)
        delete vc;
    vatChus.clear();

    cin >> n;

    for (int i = 0; i < n; ++i) {
        string id;
        cin >> id;

        cVatChu* tam = new cVatChu();

        tam->setID(id);
        tam->setVaccine(true);
    }
}
```

```

        tam->themVirus(new cCovid());
        tam->themVirus(new cEbola());
        tam->themVirus(new cHIV());

        vatChus.push_back(tam);
    }

    int demNang = 0, demTuVong = 0;
    for (auto vc : vatChus) {
        string loai = vc->getLoaiTrieuChung();
        if (loai == "Nang")
            ++demNang;
        else if (loai == "Da tu vong")
            ++demTuVong;
    }

    cout << "Trieu chung nang: " << demNang << endl;
    cout << "Da tu vong: " << demTuVong;

    return 0;
}

```

Câu 3 [HK2 22-23].

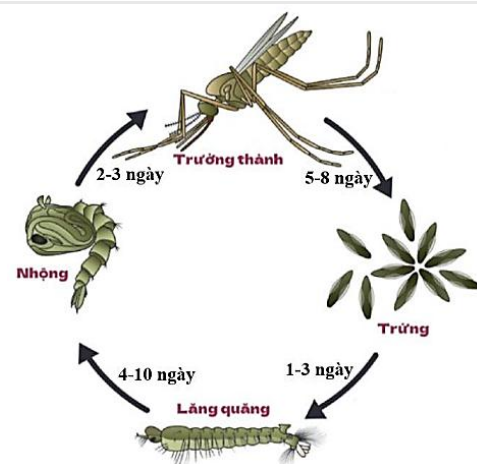
Vòng đời của côn trùng hoặc một số động vật lưỡng cư là khoảng thời gian tính từ khi chúng còn là trứng tới khi trưởng thành và lại đẻ trứng (lần đầu tiên). Chúng có thể trải qua quá trình biến đổi dần dần hoặc biến đổi hoàn toàn (trong đó mỗi giai đoạn của vòng đời có một diện mạo khác biệt rõ ràng so với giai đoạn trước và giai đoạn sau).

Có khoảng 88% côn trùng trên thế giới và ếch trải qua biến đổi hình thái hoàn toàn (Holometabolous). Sau đây là thông tin về vòng đời của muỗi, ếch và bướm sống trong môi trường tự nhiên.

Muỗi cái nhận nhiệm vụ đẻ trứng khi có đủ lượng máu. Ngoài việc sinh sản trên nước, một số loài muỗi còn chọn những vùng đất ẩm ướt để đẻ trứng. Và nước chính là thành phần thiết yếu giúp cho trứng muỗi phát triển.

Sau 1-3 ngày, trứng muỗi sẽ nở thành lăng quăng. Lăng quăng sẽ ăn vi sinh và trôi lên mặt nước để hít thở. Chúng sẽ trải qua khoảng 4 lần lột xác và lần lột xác cuối cùng, chúng sẽ tiến hóa thành nhộng. Giai đoạn này tổng cộng mất khoảng 4-10 ngày.

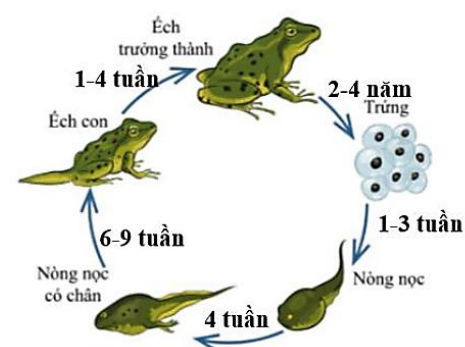
Ở giai đoạn nhộng, chúng tập trung vào việc nghỉ ngơi và không ăn. Loài nhộng mất khoảng 2-3 ngày để phát triển từ nhộng sang muỗi trưởng thành. Sau 5-8 ngày, muỗi trưởng thành lại đẻ trứng và một vòng đời mới lại bắt đầu để sinh ra một thế hệ muỗi tiếp theo.



Ếch có thể đẻ lên đến vài nghìn trứng. Từ 1-3 tuần, trứng đã sẵn sàng để nở và một con nòng nọc nhỏ bé sẽ tự thoát ra ngoài. Nòng nọc lúc này có mang để hô hấp, miệng và đuôi dài. Nòng nọc sẽ hấp thụ phần lòng đỏ còn sót lại từ trứng để cung cấp chất dinh dưỡng cần thiết cho quá trình phát triển. Sau 4 tuần, chúng bắt đầu hít thở bằng hai lỗ mũi nhỏ xíu đã hình thành trên mặt và hô hấp bằng phổi. Cái đuôi dần dần ngắn lại.

Nòng nọc bắt đầu mọc hai chi trước và sau để phát triển thành ếch con. Giai đoạn để nòng nọc phát triển thành ếch con trải qua 6-9 tuần.

Từ 1-4 tuần sau đó, ếch con phát triển thành ếch trưởng thành.



Sau 2-4 năm tiếp theo, ếch mới có khả năng sinh sản và đẻ trứng.

Trứng thường được bướm mẹ đẻ trên lá.

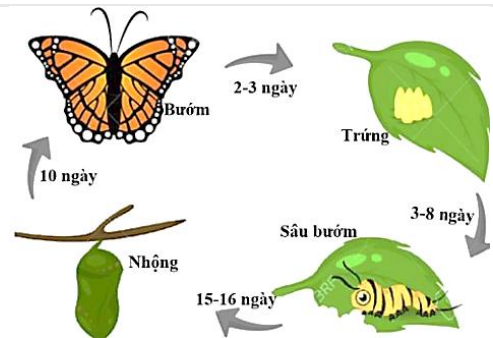
Trứng sẽ nở thành sâu bướm sau 3-8 ngày tùy vào nhiệt độ. Sau khi nở, sâu bướm sẽ ăn vỏ trứng vì chúng chứa nhiều chất dinh dưỡng cần thiết để sâu bướm phát triển, tiếp theo chúng sẽ ăn lá cây để phát triển.

Chúng phải trải qua nhiều lần lột xác. Sau mỗi lần lột xác, sâu bướm sẽ to hơn. Sau khoảng 15-16 ngày, sâu bướm sẽ dệt một chiếc kén và dính nó vào mặt dưới của lá hoặc thân cây, nó treo mình ở đó và lột xác lần cuối cùng để tạo thành nhộng.

Giai đoạn nhộng là thời điểm sự chuyển đổi bên trong xảy ra mạnh mẽ nhất mặc dù nó không thể di chuyển và ăn trong suốt thời gian này.

Khi sự biến đổi đã hoàn thành, con bướm sẽ nghỉ ngơi trong kén, giai đoạn này kéo dài khoảng 10 ngày.

Đến thời điểm phù hợp con bướm trưởng thành sẽ tự chui ra khỏi chiếc kén. Sau 2-3 ngày, bướm trưởng thành lại đẻ trứng và một vòng đời mới lại bắt đầu để sinh ra một thế hệ bướm tiếp theo.



Các nhà khoa học quan sát, nghiên cứu và ghi nhận thông tin về thời gian phát triển (thể hiện bằng số nguyên) của các con vật trên qua các giai đoạn. Ví dụ như thông tin về 1 con muỗi như sau: 2, 9, 3, 5. Có nghĩa là con muỗi này mất 2 ngày để phát triển từ trứng thành lăng quăng, mất 9 ngày để phát triển từ lăng quăng thành nhộng, mất 3 ngày từ nhộng thành muỗi và mất 5 ngày từ lúc thành muỗi trưởng thành để đẻ trứng.

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình), thiết kế sơ đồ chi tiết các lớp đối tượng (1đ) và xây dựng chương trình thực hiện các yêu cầu sau:

1. Nhập vào một danh sách N ($1 \leq N \leq 20$) con vật (bao gồm muỗi, bướm và ếch) và các thông tin về thời gian phát triển của chúng qua các giai đoạn (1.5đ).
2. Xuất ra những con vật (chỉ cần xuất thứ tự vị trí của con vật đó trong danh sách các con vật) có thời gian phát triển giữa các giai đoạn không tuân theo qui luật thông thường như trong hình minh họa (1.5đ). Ví dụ: Thông tin về 1 con bướm là 12, 15, 10, 2 và không tuân theo qui luật thông thường vì mất đến 12 ngày để chuyển hóa từ trứng thành sâu, trong khi đó bình thường một con bướm chỉ mất 3-8 ngày cho giai đoạn này.
3. Tìm con muỗi có tổng thời gian phát triển từ trứng thành nhộng dài nhất (1đ).

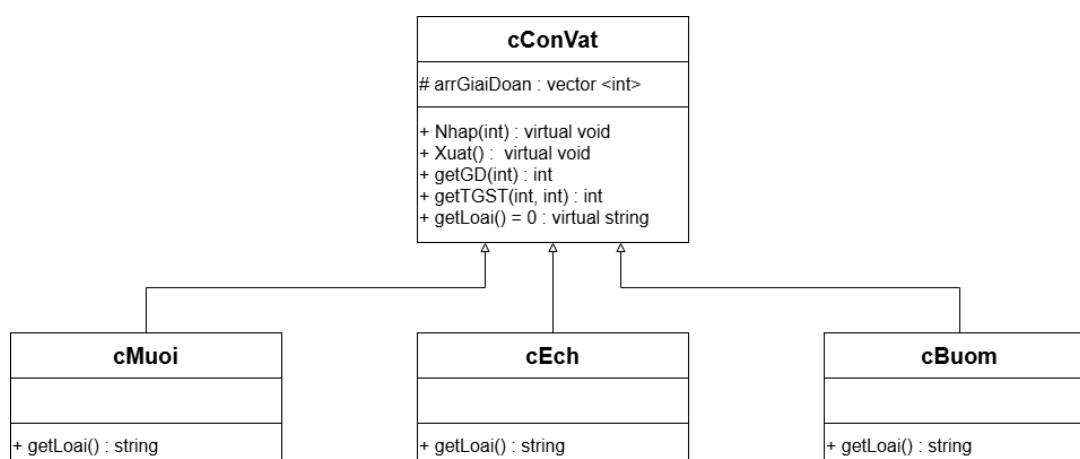
Lưu ý: Trong trường hợp sinh viên không có kiến thức về vòng đời và các giai đoạn phát triển của con vật trước đây thì phải đọc kỹ thông tin trên (các thông tin trên đủ để sinh viên thực hiện các yêu cầu) và nghiêm túc làm bài.

Bài làm:

Tóm tắt yêu cầu chương trình:

- Nhập vào N con vật, mỗi con vật có thông tin về các giai đoạn phát triển là một mảng số nguyên có 4 (muỗi, bướm) hoặc 5 giai đoạn (ếch).
- So sánh và tìm những con vật có giai đoạn phát triển khác với đề cung cấp.
- Tìm tổng thời gian phát triển của con vật.

Sơ đồ lớp:



Khai báo các lớp:

```
cConVat.h
#pragma once
#include <iostream>
#include <vector>
using namespace std;

class cConVat {
protected:
    vector<int> arrGiaiDoan;
public:
    virtual void Nhap(int);
    virtual void Xuat();
    int getGD(int);
    int getTGST(int, int);
    virtual string getLoai() = 0;
};
```

```
cMuoi.h
#pragma once
#include "cConVat.h"

class cMuoi : public cConVat {
public:
    string getLoai();
};
```

```
cEch.h
#pragma once
#include "cConVat.h"

class cEch : public cConVat {
public:
    string getLoai();
};
```

```
cBuom.h
#pragma once
#include "cConVat.h"

class cBuom : public cConVat {
public:
    string getLoai();
};
```

Định nghĩa các phương thức:

```
cConVat.cpp
#include "cConVat.h"

void cConVat::Nhap(int n) {
    for (int i = 0; i < n; i++) {
        int temp;
        cin >> temp;
        arrGiaiDoan.push_back(temp);
    }
}

void cConVat::Xuat() {
    for (auto gd : arrGiaiDoan)
        cout << gd << ' ';
    cout << endl;
}

int cConVat::getGD(int i) {
    return arrGiaiDoan[i];
}

int cConVat::getTGST(int s, int e) {
```

```

    int sum = 0;
    for (int i = s; i <= e; i++)
        sum += arrGiaiDoan[i];
    return sum;
}

```

cMuoi.cpp

```
#include "cMuoi.h"
```

```

string cMuoi::getLoai() {
    return "Muoi";
}

```

cEch.cpp

```
#include "cEch.h"
```

```

string cEch::getLoai() {
    return "Ech";
}

```

cBuom.cpp

```
#include "cBuom.h"
```

```

string cBuom::getLoai() {
    return "Buom";
}

```

Thân chương trình:

Bai03.cpp

```

#include <iostream>
#include <vector>
#include "cConVat.h"
#include "cMuoi.h"
#include "cEch.h"
#include "cBuom.h"

```

```
using namespace std;
```

```
int main() {
```

```
    // Cau 1
```

```

    int n;
    cin >> n;
    vector<cConVat*> conVats;

```

```

    for (int i = 0; i < n; i++) {
        cout << "Nhap loai con vat (1 = muoi, 2 = ech, 3 = buom): ";
        int luaChon;
        cin >> luaChon;

```

```
        cConVat* tam;
```

```

        switch (luaChon) {
            case 1:

```

```

        tam = new cMuoi;
        tam->Nhap(4);
        break;
    case 2:
        tam = new cEch;
        tam->Nhap(5);
        break;
    default:
        tam = new cBuum;
        tam->Nhap(4);
    }

    conVats.push_back(tam);
}

// Cau 2

for (int i = 0; i < conVats.size(); i++) {
    cConVat* conVat = conVats[i];
    if (conVat->getLoai() == "Muoi") {
        if (!(conVat->getGD(0) >= 1 && conVat->getGD(0) <= 3 &&
            conVat->getGD(1) >= 4 && conVat->getGD(1) <= 10 &&
            conVat->getGD(2) >= 2 && conVat->getGD(2) <= 3 &&
            conVat->getGD(3) >= 5 && conVat->getGD(3) <= 8)) {
            cout << "Vi tri con vat khac thuong: " << i << endl;
        }
    } else if (conVat->getLoai() == "Buum") {
        if (!(conVat->getGD(0) >= 3 && conVat->getGD(0) <= 8 &&
            conVat->getGD(1) >= 15 && conVat->getGD(1) <= 16 &&
            conVat->getGD(2) == 10 &&
            conVat->getGD(3) >= 2 && conVat->getGD(3) <= 3)) {
            cout << "Vi tri con vat khac thuong: " << i << endl;
        }
    } else {
        if (!(conVat->getGD(0) >= 7 && conVat->getGD(0) <= 21 &&
            conVat->getGD(1) == 4 &&
            conVat->getGD(2) >= 6 && conVat->getGD(2) <= 9 &&
            conVat->getGD(3) >= 1 && conVat->getGD(3) <= 4 &&
            conVat->getGD(4) >= 730 && conVat->getGD(4) <= 1460)) {
            cout << "Vi tri con vat khac thuong: " << i << endl;
        }
    }
}

// Cau 3

int tgst = 0;
for (auto conVat : conVats) {
    if (conVat->getLoai() == "Muoi") {
        int tg = conVat->getTGST(0, 1);
        if (tg > tgst) tgst = tg;
    }
}

for (auto conVat : conVats) {
    if (conVat->getLoai() == "Muoi" && conVat->getTGST(0, 1) == tgst) {
        conVat->Xuat();
    }
}

```

```
    }  
}  
  
return 0;  
}
```

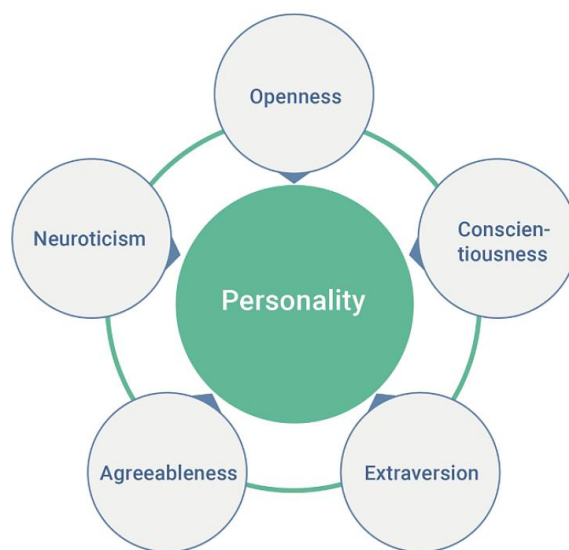
Câu 3 [HK2 18-19].

Big Five Model là mô hình tâm lý được nghiên cứu và phát triển bởi nhiều nhà khoa học trên thế giới. Mô hình này cho rằng trong tính cách của một người đều bao gồm 5 yếu tố và nhiệm vụ của trắc nghiệm tâm lý là xác định mức độ biểu hiện của từng yếu tố này. Big Five Model được đánh giá cao về độ tin cậy và tính khoa học.

Mô hình đặc điểm tính cách Big Five đo lường 5 nét tính cách của con người:

- Tự chủ tận tâm (Conscientiousness).
- Hướng ngoại (Extraversion).
- Hòa đồng (Agreeableness).
- Sẵn sàng trải nghiệm (Openness to Experience).
- Bất ổn cảm xúc (Neuroticism).

Nên còn được gọi là mô hình OCEAN.



Yếu tố	Chỉ số cao	Chỉ số thấp
Sẵn sàng trải nghiệm (O) Yếu tố Sẵn sàng trải nghiệm chỉ ra mức độ sẵn sàng tiếp nhận những cái mới, khả năng tuân thủ các quy định, luật lệ chung của một người. Yếu tố Sẵn sàng trải nghiệm có 6 tiêu chí gồm: thích nghệ thuật, trải nghiệm cảm xúc, trải nghiệm hoạt động, ham hiểu biết, tự do, trí tưởng tượng.	Người có điểm cao ở yếu tố này thường là người thích những ý tưởng mới, thích hiểu biết nhiều lĩnh vực nhưng đồng thời cũng thích tự do, không thích bị ràng buộc...	Người có điểm thấp ở yếu tố này thường khá bảo thủ, khó tiếp nhận những ý tưởng mới, lạ. Họ thích sự ổn định, quen thuộc và thực tế.
Tự chủ tận tâm (C) Yếu tố Tự chủ chỉ ra khả năng chịu áp lực, sự nỗ lực, kiên trì của một người. Yếu tố Tự chủ được chia ra thành 6 tiêu chí nhỏ gồm: tính trật tự, kỷ luật, tự tin, trách nhiệm, nỗ lực, thận trọng.	Người có điểm cao ở yếu tố này thường là người chăm chỉ, có khả năng chịu áp lực tốt. Họ thường là người gắn bó, trung thành với tổ chức.	Người có điểm thấp ở yếu tố này thường dễ bỏ cuộc, khả năng chịu áp lực, tuân thủ kỷ luật của tổ chức kém.
Hướng ngoại (E)	Người có điểm cao ở yếu tố này thường là người nhiệt tình,	Người có điểm thấp ở yếu tố này thường ngại giao tiếp, không

Yếu tố Hướng ngoại chỉ ra khả năng giao tiếp, thái độ nhiệt tình trong công việc cũng như mức độ thích tạo sự ảnh hưởng của một người. Yếu tố Hướng ngoại có 6 tiêu chí nhỏ gồm: thích tạo ảnh hưởng, quảng giao, tích cực vận động, tìm kiếm sự hưng phấn, thân thiện, cảm xúc tích cực.	năng động, giao tiếp tốt, thích thể hiện bản thân.	thích sự nổi bật, thích được làm việc độc lập.
Hòa đồng dễ chịu (A) Yếu tố Hoà đồng chỉ ra mức độ hoà hợp, sự quan tâm và chấp nhận sự khác biệt của một người. Hòa đồng có 6 tiêu chí nhỏ gồm: tin tưởng, hợp tác, thành thực, vị tha, khiêm tốn, nhân hậu.	Người có điểm cao ở yếu tố này thường thân thiện, cởi mở, đồng cảm với mọi người nhưng nhiều khi "thiếu chính kiến".	Người có điểm thấp thường đặt lợi ích của bản thân lên trên, ít đồng cảm, chia sẻ với đồng nghiệp, có tính cạnh tranh cao.
Bất Ổn cảm xúc (N) Yếu tố Bất Ổn cảm xúc chỉ ra khả năng kiểm soát cảm xúc, chịu áp lực, ứng phó với căng thẳng của một người. Yếu tố Bất Ổn cảm xúc có 6 tiêu chí tâm lý gồm: trầm cảm, tự ti, sống bản năng, dễ bị tổn thương, lo âu, giận dữ	Người có điểm cao ở yếu tố này thường có các cảm xúc tiêu cực như: lo lắng, bức bối, tự ti, yếu đuối và khả năng chịu áp lực kém.	Người có điểm thấp ở yếu tố này thường kiểm soát được cảm xúc, ứng phó với căng thẳng tốt, ít bị bên ngoài ảnh hưởng đến tâm trạng của bản thân.

Bài kiểm tra tâm lý theo Big Five Model có kết quả tương tự như sau: **O93-C74-E31-A96-N5**. Các chữ cái đại diện cho mỗi nét tính cách, và những con số đại diện cho tỷ lệ % những người ghi được điểm thấp hơn bạn so với từng nét tính cách. Ở đây, O93 có nghĩa là 93% của những người đã thử nghiệm đạt được thấp hơn bạn trong tính cởi mở. Vì vậy, so với những người khác, bạn rất cởi mở để có những trải nghiệm mới và sáng tạo. C74 có nghĩa là 74% của những người đã thử nghiệm đạt điểm thấp hơn so với bạn trong sự tận tâm. Vì vậy, bạn đang khá có tổ chức và kỷ luật tự giác, so với những người khác đã thử nghiệm.

Hiệu quả của trắc nghiệm Big Five Model trong tuyển chọn đánh giá nhân sự đã được các nhà nghiên cứu liên tục đưa ra để hỗ trợ những nhà tuyển dụng, giúp cho nhà tuyển dụng phát hiện và tuyển chọn những nhân sự phù hợp với các vị trí công việc, giúp nhà tuyển dụng lường trước được những nguy cơ có thể xảy đến đối với một nhân sự của mình.

Một số thông tin đề xuất như sau:

- Người có chỉ số C cao sẽ là người tuân thủ nguyên tắc và chuẩn mực của tổ chức, làm việc chăm chỉ và kiên trì trong các kế hoạch công việc.

- b. Người có chỉ số C thấp có những biểu hiện vô tổ chức, bỏ việc giữa chừng, thiếu trách nhiệm, bất cẩn, cầu thả và bốc đồng trong công việc.
- c. Người có chỉ số N cao sẽ có các cảm xúc tiêu cực, cảm giác căng thẳng trở nên lớn hơn và giảm đi sự chắc chắn trong việc cam kết lâu dài với một công việc.
- d. Đặc biệt người có chỉ số E thấp và N cao ảnh hưởng đến việc tìm kiếm và kết nối với thông tin, hệ quả sẽ khiến một cá nhân có xu hướng tránh tiếp cận với nguồn thông tin mới, trở nên thiếu hụt kỹ năng và hạn chế việc tiếp cận các thông tin nghề nghiệp quan trọng.

Trong đó các trường hợp b, c, d có nguy cơ cao mà nhà tuyển dụng/ tổ chức/ doanh nghiệp cần lưu ý.

Giả sử chọn con số 70 là ngưỡng xác định chỉ số cao và 30 là ngưỡng xác định chỉ số thấp, khoảng còn lại, chúng ta không đưa ra nhận định chính xác về khuynh hướng tính cách của yếu tố này.

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.5đ) và xây dựng chương trình thực hiện các yêu cầu sau:

1. Nhập vào thông tin kết quả đánh giá tâm lý của một người. (1đ)
2. Nhập vào kết quả đánh giá tâm lý của n người trong danh sách. (0.5đ)
3. Chọn một người trong danh sách và cho biết các thông tin về tính cách của người đó (1đ) Ví dụ: Một người có đánh giá tâm lý Big Five như sau: O70-C30-E60-A96-N10 Sẽ xuất kết quả là:

70	Sẵn sàng trải nghiệm (O) Người có điểm cao ở yếu tố này thường là người thích những ý tưởng mới, thích hiểu biết nhiều lĩnh vực nhưng đồng thời cũng thích tự do, không thích bị ràng buộc...
30	Tự chủ tận tâm (C) Người có điểm thấp ở yếu tố này thường dễ bỏ cuộc, khả năng chịu áp lực, tuân thủ kỷ luật của tổ chức kém.
60	Hướng ngoại (E) Không xác định rõ.
96	Hòa đồng dễ chịu (A) Người có điểm cao ở yếu tố này thường thân thiện, cởi mở, đồng cảm với mọi người nhưng nhiều khi "thiếu chính kiến".
10	Bất ổn cảm xúc (N) Người có điểm thấp ở yếu tố này thường kiểm soát được cảm xúc, ứng phó với căng thẳng tốt, ít bị bên ngoài ảnh hưởng đến tâm trạng của bản thân.

4. Hãy cho biết những người có nguy cơ cao mà nhà tuyển dụng/ tổ chức/ doanh nghiệp cần lưu ý. (1đ)

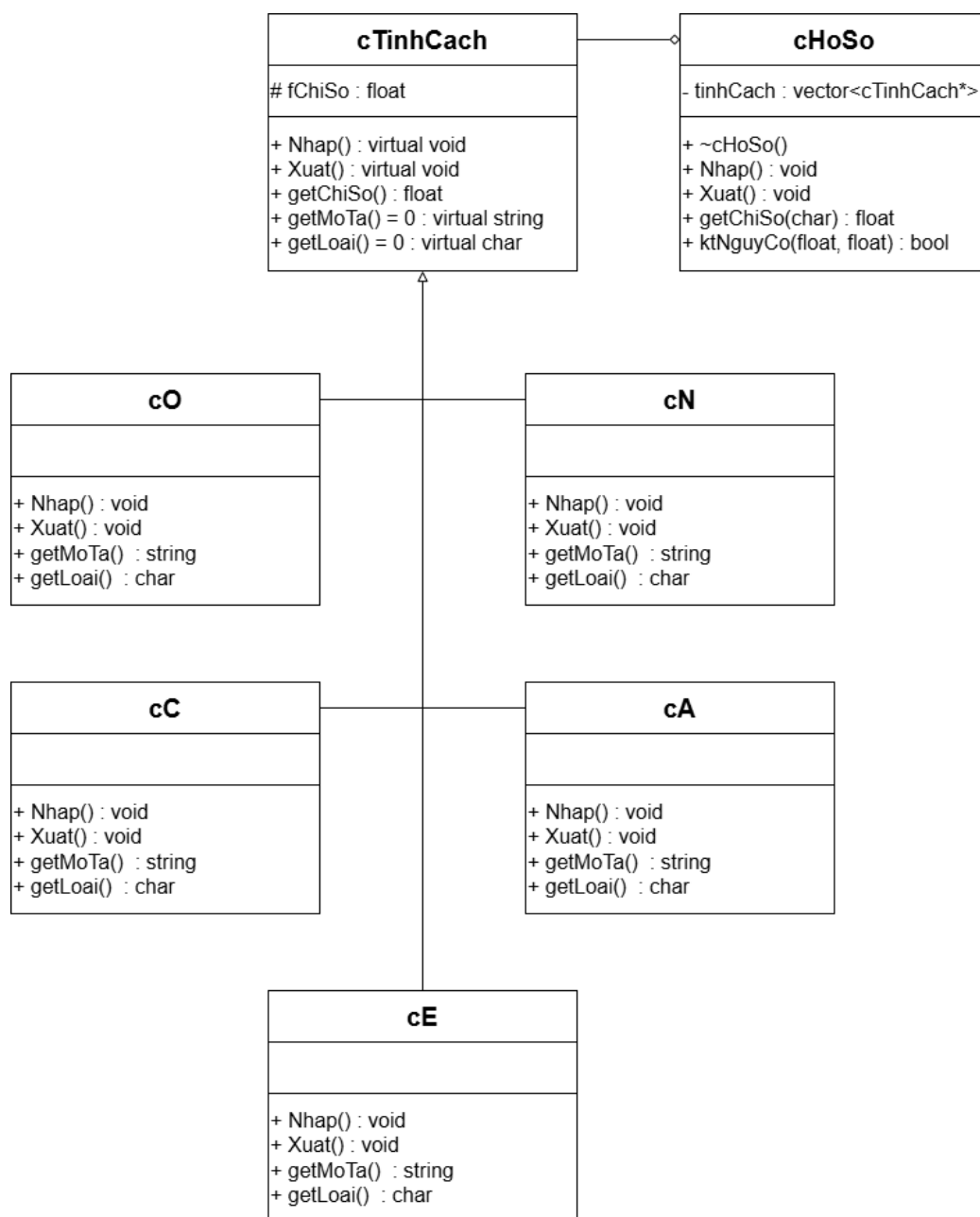
Lưu ý: Trong trường hợp sinh viên không biết khái niệm về tâm lý học và mô hình 5 yếu tố trước đây thì phải đọc kỹ thông tin trên (các thông tin trên đủ để sinh viên thực hiện các yêu cầu của đề thi) và nghiêm túc làm bài. Giám thị coi thi không giải thích gì thêm.

Bài làm:

Tóm tắt yêu cầu chương trình:

1. Nhập vào thông tin kết quả đánh giá tâm lý của 1 và n người. Mỗi thông tin kết quả gồm có các chỉ số của O, C, E, A, N.
2. Mô tả tính cách của 1 người.
3. Cho biết những người có nguy cơ cao: Là nhóm người có C thấp, N cao, E thấp – N cao. Trong đó, ≥ 70 là cao và ≤ 30 là thấp.

Sơ đồ lớp:



Khai báo các lớp:

```
cTinhCach.h
#pragma once
#include <string>
using namespace std;

class cTinhCach {
protected:
    float fChiSo;
public:
    virtual void Nhap();
    virtual void Xuat();
    float getChiSo();
    virtual string getMoTa() = 0;
    virtual char getLoai() = 0;
};
```

```
c0.h
#pragma once
#include "cTinhCach.h"

class c0 : public cTinhCach {
public:
    string getMoTa();
    char getLoai();
    void Nhap();
    void Xuat();
};
```

Tương tự cC, cE, cA, cN.

```
cHoSo.h
#pragma once
#include <vector>
#include "cTinhCach.h"
#include "c0.h"
#include "cC.h"
#include "cE.h"
#include "cA.h"
#include "cN.h"

using namespace std;

class cHoSo {
private:
    vector<cTinhCach*> tinhCach;
public:
    ~cHoSo();
    void Nhap();
    void Xuat();
    float getChiSo(char);
    bool ktNguyCo(float, float);
};
```

Định nghĩa các phương thức:

```
cTinhCach.cpp
#include "cTinhCach.h"
#include <iostream>

using namespace std;

void cTinhCach::Nhap() {
    cout << "- Chi so: ";
    cin >> fChiSo;
}

void cTinhCach::Xuat() {
    cout << "- Chi so: " << fChiSo << endl;
}

float cTinhCach::getChiSo() {
    return fChiSo;
}
```

```
c0.cpp
#include "c0.h"
#include <iostream>

using namespace std;

void c0::Nhap() {
    cout << "- Chi so 0: ";
    cTinhCach::Nhap();
}

void c0::Xuat() {
    cout << "- Chi so 0: ";
    cTinhCach::Xuat();
}

string c0::getMoTa() {
    return "San sang trai nghiem";
}

char c0::getLoai() {
    return '0';
}
```

Tương tự cC, cE, cA, cN.

```
cHoSo.cpp
#include "cHoSo.h"
#include <iostream>

using namespace std;

cHoSo::~cHoSo() {
    for (auto& tc : tinhCach)
        delete tc;
}

void cHoSo::Nhap() {
    tinhCach.clear();
    tinhCach.resize(5);
    for (auto& tc : tinhCach)
        tc->Nhap();
}

void cHoSo::Xuat() {
    for (auto tc : tinhCach)
        tc->Xuat();
}

float cHoSo::getChiSo(char loai) {
    switch (loai) {
        case '0': return tinhCach[0]->getChiSo();
        case 'C': return tinhCach[1]->getChiSo();
    }
}
```

```

        case 'E': return tinhCach[2]->getChiSo();
        case 'A': return tinhCach[3]->getChiSo();
        default : return tinhCach[4]->getChiSo();
    }
}

bool cHoSo::ktNguyCo(float thap, float cao) {
    if (getChiSo('C') <= thap)
        return true;
    else if (getChiSo('N') >= cao)
        return true;
    else if (getChiSo('E') <= thap && getChiSo('N') >= cao)
        return true;
    return false;
}

```

Thân chương trình:

```
Bai03.cpp
#include "cHoSo.h"
#include <iostream>
#include <vector>

using namespace std;

int main() {

    // Cau 1

    int n;
    cout << "Nhap so luong ho so: ";
    cin >> n;
    vector<cHoSo*> ds;
    ds.reserve(n);

    for (int i = 0; i < n; ++i) {
        cHoSo* hs = new cHoSo();
        hs->Nhap();
        ds.push_back(hs);
    }

    // Cau 2

    cout << "Danh sach nguoi: \n";
    for (auto hs : ds) {
        hs->Xuat();
    }

    //Cau 3

    cout << "Nhưng doi tuong nguy co:\n";
    for (auto hs : ds) {
        if (hs->ktNguyCo(30,70))
            hs->Xuat();
    }

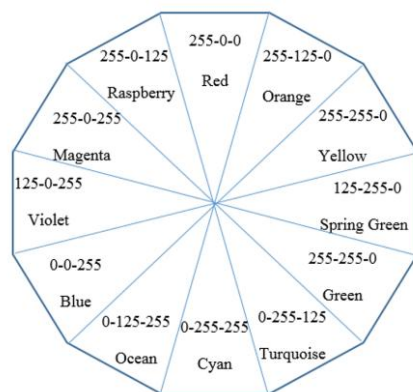
    return 0;
}
```


Câu 3 [HK2 16-17].

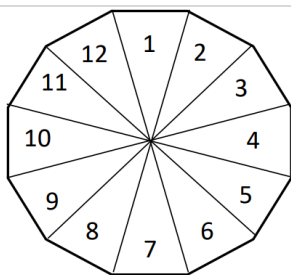
Giao diện website gồm các thành phần cơ bản đặc trưng chung bởi các yếu tố về tọa độ (hoành độ, tung độ), kích thước (dài, rộng). Website có 2 thành phần chính:

- **Label** có thêm nội dung text hiển thị, màu chữ và màu nền.
- **Button** có thể hiển thị một hình ảnh hoặc text (màu chữ, màu nền).

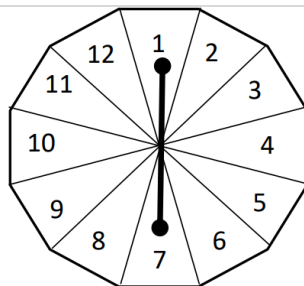
Màu sắc trên web được thực hiện bằng cách kết hợp pha trộn của màu đỏ, xanh lá và xanh dương; đây là hình thức phối màu có tên gọi là RGB. Mỗi màu sắc đại diện cho một giá trị số học từ 0 đến 255 và mỗi màu sắc có giá trị tương ứng với màu đỏ, xanh lá và xanh dương.



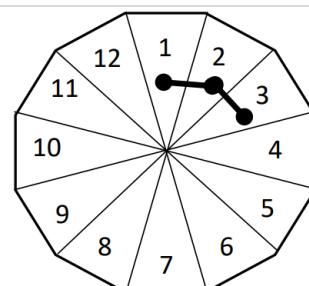
Phối màu web sẽ giúp chọn các màu khác phù hợp với màu cơ bản, để từ đó có các màu dùng chung cho 1 thiết kế mà đảm bảo tính hài hòa giữa màu sắc. Có rất nhiều phương pháp phối màu, và hầu hết đều dựa trên Bánh xe màu để phối. Trong đó, đơn giản nhất là 3 cách phối màu sau:



Phối màu đơn sắc: Tất cả các thành phần đều có cùng màu nền.



Phối màu bổ túc trực tiếp: là những cặp màu đối xứng nhau trên bánh xe màu. Vd: 1-7, 2-8, 3-9, 4-10, 5-11, 6-12.



Phối màu tương đồng (thường là 3 màu) các màu liền kề nhau trên bánh xe màu. Vd: 1-2-3, 2-3-4, 11-12-1,...

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.5đ) và xây dựng chương trình thực hiện các yêu cầu sau:

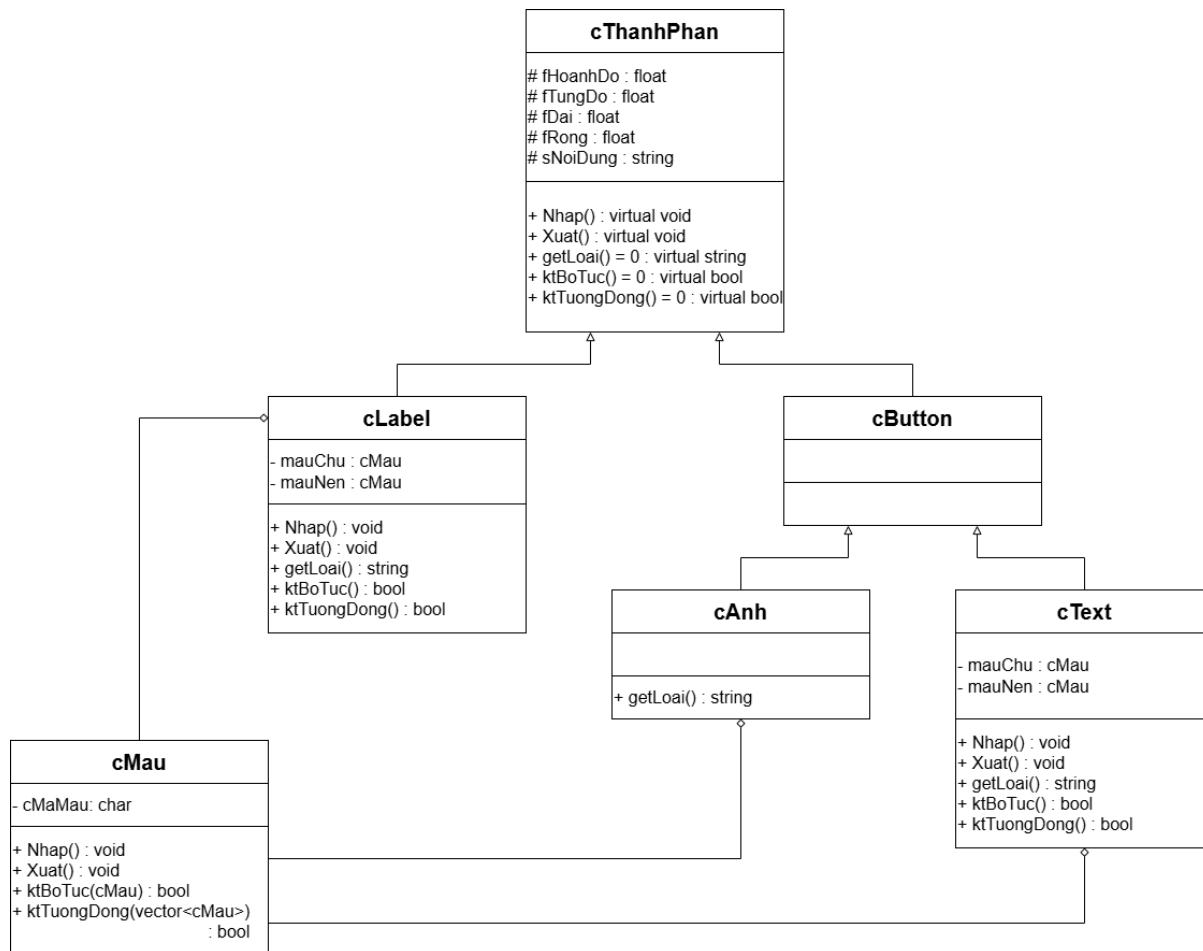
1. Nhập danh sách các thành phần có màu trên trang web. (1.5đ).
2. Kiểm tra màu nền và màu chữ của thành phần đầu tiên trong danh sách có phù hợp với phối màu bổ túc trực tiếp hay không? (1đ).
3. Kiểm tra màu nền của các thành phần xem phù hợp với quy tắc phối màu nào hay không? (1đ)

Bài làm:

Tóm tắt yêu cầu chương trình:

1. Nhập danh sách các thành phần. Mỗi thành phần gồm:
 - a. Hoành độ, tung độ.
 - b. Dài, rộng.
 - c. Label: Có thêm:
 - i. Nội dung.
 - ii. Màu chữ.
 - iii. Màu nền.
 - d. Button: Có 2 loại:
 - i. Hình ảnh.
 - ii. Text: Màu chữ, màu nền.
2. Kiểm tra quy tắc phối màu.

Sơ đồ lớp:



Khai báo các lớp:

```

cMau.h
#pragma once
#include <string>
#include <vector>

using namespace std;

class cMau {
private:
    char cMaMau;
public:
    void Nhap();
    void Xuat();
    bool ktBoTuc(cMau);
    bool ktTuongDong(vector <cMau>);
};
    
```

```

cThanhPhan.h
#pragma once
#include <string>

using namespace std;

class cThanhPhan {
protected:
    float fHoanhDo;
    float fTungDo;
    float fDai;
    float fRong;
    string sNoiDung;
public:
    virtual void Nhap();
    virtual void Xuat();
    virtual string getLoai() = 0;
    virtual bool ktBoTuc() = 0;
    virtual bool ktTuongDong() = 0;
};
    
```

```
cLabel.h
#pragma once
#include "cThanhPhan.h"
#include "cMau.h"

class cLabel : public cThanhPhan {
private:
    cMau mauChu, mauNen;
public:
    void Nhap();
    void Xuat();
    string getLoai();
    bool ktBoTuc();
    bool ktTuongDong();
};
```

```
cButton.h
#pragma once
#include "cThanhPhan.h"

class cButton : public cThanhPhan {
};
```

```
cAnh.h
#pragma once
#include "cButton.h"

class cAnh : public cButton {
public:
    string getLoai();
};
```

```
cText.h
#pragma once
#include "cButton.h"
#include "cMau.h"

class cText : public cButton {
private:
    cMau mauChu;
    cMau mauNen;
public:
    void Nhap();
    void Xuat();
    string getLoai();
    bool ktBoTuc();
    bool ktTuongDong();
};
```

Định nghĩa các phương thức:

```
cMau.cpp
#include "cThanhPhan.h"
#include <iostream>

void cMau::Nhap() {
    cout << "- Ma mau: ";
    int tam;
    cin >> tam;
    cMaMau = tam;
}

void cMau::Xuat() {
    cout << "- Ma mau: " << (int)cMaMau
<< endl;
}

bool cMau::ktBoTuc(cMau x) {
```

```
cThanhPhan.cpp
#include "cThanhPhan.h"
#include <iostream>

void cThanhPhan::Nhap() {
    cout << "- Toa do: ";
    cin >> fHoanhDo >> fTungDo;
    cout << "- Kich thuoc: ";
    cin >> fDai >> fRong;
    cout << "- Noi dung: ";
    cin.ignore();
    getline(cin, sNoiDung);
}

void cThanhPhan::Xuat() {
    cout << "- Toa do: " << fHoanhDo <<
' ' << fTungDo << endl;
```

```

        return (x.cMaMau + 6) % 12 == this-
>cMaMau;
    }

    bool cMau::ktTuongDong(vector <cMau>
ds) {
    ds.push_back(cMaMau);
    for (int i = 0; i < ds.size() - 1;
++i)
        if (ds[i] + 1 != ds[i+1])
            return false;
    return true;
}

```

```

    cout << "- Kich thuoc: " << fDai <<
' ' << fRong << endl;
    cout << "- Noi dung: " << sNoiDung
<< endl;
}

```

cLabel.cpp

```

#include "cLabel.h"
#include <iostream>

void cLabel::Nhap() {
    cThanhPhan::Nhap();
    mauChu.Nhap();
    mauNen.Nhap();
}

void cLabel::Xuat() {
    cThanhPhan::Xuat();
    mauChu.Xuat();
    mauNen.Xuat();
}

string cLabel::getLoai() {
    return "Label";
}

bool cLabel::ktBoTuc() {
    return mauChu.ktBoTuc(mauNen);
}

bool cLabel::ktTuongDong() {
    vector <cMau> ds = {mauNen};
    return mauChu.ktTuongDong(ds);
}

```

cButton.cpp

cAnh.h

```

#include "cAnh.h"
#include <iostream>

string cAnh::getLoai() {
    return "Anh";
}

```

cText.h

```

#include "cText.h"
#include <iostream>

void cText::Nhap() {
    cThanhPhan::Nhap();
    mauChu.Nhap();
    mauNen.Nhap();
}

void cText::Xuat() {
    cThanhPhan::Xuat();
    mauChu.Xuat();
    mauNen.Xuat();
}

```

```

}

string cText::getLoai() {
    return "Text";
}

bool cText::ktBoTuc() {
    return mauChu.ktBoTuc(mauNen);
}

bool cText::ktTuongDong() {
    vector <cMau> ds = {mauNen};
    return mauChu.ktTuongDong(ds);
}

```

Thân chương trình:

```

Bai03.cpp
#include "cThanhPhan.h"
#include "cLabel.h"
#include "cButton.h"
#include "cAnh.h"
#include "cText.h"
#include <iostream>
#include <vector>

using namespace std;

int main() {

    // Cau 1

    int n;
    cin >> n;

    vector <cThanhPhan*> ds;
    for (int i = 0; i < n; i++) {
        cThanhPhan* tam;
        cout << "- Nhap loai (0 = label, 1 = anh, 2 = text): ";
        int luaChon;
        cin >> luaChon;

        switch (luaChon) {
            case 0:
                tam = new cLabel; break;
            case 1:
                tam = new cAnh; break;
            default:
                tam = new cText;
        }

        tam->Nhap();
        ds.push_back(tam);
    }
}

```

```

// Cau 2

if (!ds.empty())
    cout << ds[0]->ktBoTuc();

// Cau 3

for (auto tp : ds) {
    cout << (tp->ktBoTuc() || tp->ktTuongDong());
}

return 0;
}

```

Câu 3 [HK2 15-16].

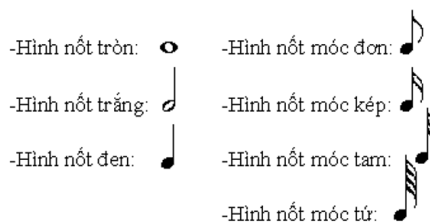
Xây dựng chương trình mô phỏng biên soạn nhạc với các mô tả ký hiệu âm nhạc như sau:

Nốt nhạc: là ký hiệu trong bản nhạc dùng để xác định cao độ (độ cao), trường độ (độ dài, độ ngân vang) của từng âm thanh được vang lên trong bản nhạc.

Có 7 ký hiệu nốt nhạc dùng để xác định cao độ theo thứ tự từ thấp đến cao, đó là Đô (C), Rê (D), Mi (E), Fa (F), Sol (G), La (A), và Si (B).



Để xác định trường độ của nốt nhạc có cao độ kể trên, người ta cũng dùng 7 hình nốt để thể hiện, đó là:



- **Nốt tròn** có trường độ tương đương với trường độ của 4 nốt đen.
- **Nốt trắng** có trường độ bằng 2 nốt đen.
- **Nốt đen** có trường độ bằng 1 phách (đơn vị thời gian trong âm nhạc - vd như 1 bước chân người đi trong không gian).
- **Nốt móc đơn** có trường độ bằng 1/2 nốt đen.
- **Nốt móc đôi** có trường độ bằng 1/4 nốt đen.
- **Nốt móc tam** có trường độ bằng 1/8 nốt đen.
- **Nốt móc tứ** có trường độ bằng 1/16 nốt đen.

Dấu lặng (Z - Zero) là ký hiệu cho biết phải ngưng, không diễn tấu âm thanh (không có cao độ) trong một thời gian nào đó. Các dấu lặng trong thời gian tương ứng (giá trị trường độ) với dạng dấu nhạc nào, thì cũng có tên gọi tương tự.



Ví dụ: Ký hiệu một bản nhạc:



C1 - C1/2 - A1/2 - G1/2 - Z1 - D1/2 - C1 - C1 - F2

Trong đó:

Trường độ	1	1/2	1/2	1/2	1	1/2	1	1	2
Cao độ	C	C	A	G	Z	D	C	C	F
Nốt	Đô đen	Đô móc đơn	La móc đơn	Sol móc đơn	Dấu lặng đen	Rê móc đơn	Rê đen	Đô đen	Fa trắng

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.5đ) và xây dựng chương trình thực hiện các yêu cầu sau:

1. Soạn một bản nhạc (1.5đ)
2. Tìm và đếm có bao nhiêu dấu lặng đen trong bản nhạc (1đ)
3. Cho biết nốt nhạc có cao độ cao nhất trong bản nhạc (1đ)

Lưu ý: Trong trường hợp sinh viên không biết về nhạc lý trước đây thì phải đọc kỹ thông tin trên (các thông tin trên đủ để sinh viên thực hiện các yêu cầu của đề thi) và nghiêm túc làm bài. Giám thị coi thi không giải thích gì thêm.

Bài làm:

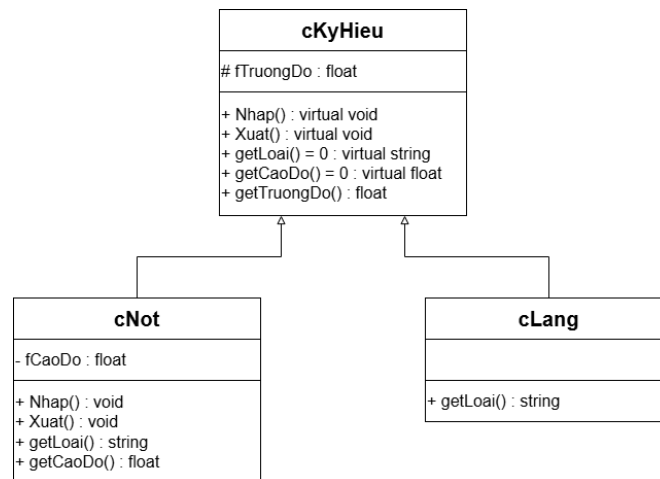
Tóm tắt yêu cầu chương trình:

1. Nhập / xuất danh sách các nốt nhạc.
2. Tìm nốt nhạc theo loại (cao độ).
3. Tìm nốt nhạc theo cao độ.

Quy ước riêng của bài làm:

- Dùng trường độ để phân biệt giữa các nốt nhạc và giữa các dấu lặng.
- Dùng cao độ để phân biệt các nốt nhạc với nhau (lấy mốc là nốt đen có cao độ là 1).

Sơ đồ lớp:



Khai báo các lớp:

```
cKyHieu.h
#pragma once
#include <string>

using namespace std;

class cKyHieu {
protected:
    float fTruongDo;
public:
    virtual void Nhap();
    virtual void Xuat();
    virtual string getLoai() = 0;
    virtual float getCaoDo() = 0;
    float getTruongDo();
};
```

```
cNot.h
#pragma once
#include "cKyHieu.h"

class cNot : public cKyHieu {
private:
    float fCaoDo;
public:
    void Nhap();
    void Xuat();
    string getLoai();
    float getCaoDo();
};

cLang.h
#pragma once
#include "cKyHieu.h"

class cLang : public cKyHieu {
public:
```

```
string getLoai();
};
```

Định nghĩa các phương thức:

cKyHieu.cpp

```
#include "cKyHieu.h"
#include <iostream>

void cKyHieu::Nhap() {
    cout << "- Truong do (so voi hinh
not den): ";
    cin >> fTruongDo;
}

void cKyHieu::Xuat() {
    cout << "- Truong do: " <<
fTruongDo << endl;
}

float cKyHieu::getTruongDo() {
    return fTruongDo;
}
```

cNot.cpp

```
#include "cNot.h"
#include <iostream>

void cNot::Nhap() {
    cKyHieu::Nhap();
    cout << "- Cao do: ";
    cin >> fCaoDo;
}

void cNot::Xuat() {
    cKyHieu::Xuat();
    cout << "- Cao do: " << fCaoDo <<
endl;
}

string cNot::getLoai() {
    return "Not";
}

float cNot::getCaoDo() {
    return fCaoDo;
}

cLang.cpp
#include "cLang.h"
#include <iostream>

string cLang::getLoai() {
    return "Lang";
}
```

Thân chương trình:

Bai03.cpp

```
#include "cKyHieu.h"
#include "cNot.h"
#include "cLang.h"
#include <vector>
#include <iostream>

using namespace std;
```

```

int main() {
    // Cau 1

    int n;
    cout << "Nhap so ky hieu: ";
    cin >> n;

    vector<cKyHieu*> banNhac;

    for (int i = 0; i < n; i++) {
        cKyHieu* tam;
        cout << "- Nhap loai (0 = not nhac, 1 = dau lang): ";
        int loai;
        cin >> loai;
        if (loai == 1)
            tam = new cLang;
        else
            tam = new cNot;

        tam->Nhap();
        banNhac.push_back(tam);
    }

    // Cau 2

    int dem = 0;
    for (auto kh : banNhac) {
        if (kh->getLoai() == "Lang" && kh->getTruongDo() == 1)
            dem++;
    }
    cout << "So dau lang den: " << dem << endl;

    // Cau 3

    int maxCD = -1;
    for (auto kh : banNhac) {
        if (kh->getLoai() == "Not" && kh->getCaoDo() > maxCD)
            maxCD = kh->getCaoDo();
    }
    cout << "Cac not cao do lon nhat: " << maxCD << "\n";
    for (int i = 0; i < banNhac.size(); i++) {
        if (banNhac[i]->getLoai() == "Not" && banNhac[i]->getCaoDo() == maxCD) {
            cout << "Vi tri: " << i+1 << ": ";
            banNhac[i]->Xuat();
        }
    }

    return 0;
}

```

Câu 3 [HK2 17-18].

Đầu những năm 1900, dựa trên sự hiện diện của các kháng nguyên trên màng hồng cầu, các nhà khoa học đã xác định rằng con người có 4 **nhóm máu** khác nhau: **O, A, B** và **AB**. Hệ thống phân loại nhóm máu này (gọi là hệ thống nhóm máu ABO) cung cấp cho bác sĩ các thông tin quan trọng để lựa chọn nhóm máu phù hợp trong việc truyền máu. Và đồng thời có thể tiên đoán được nhóm máu tương đối của người con dựa trên nhóm máu của cha mẹ theo cơ chế di truyền học.

Nhóm máu của người con khi biết được nhóm máu của cha và mẹ:

		Nhóm máu người cha				
		A	B	AB	O	
Nhóm máu người mẹ	A	A hoặc O	A, B, AB hoặc O	A, B hoặc AB	A hoặc O	Dự đoán khả năng nhóm máu người con
	B	A, B, AB hoặc O	B hoặc O	A, B hoặc AB	B hoặc O	
	AB	A, B hoặc AB	A, B hoặc AB	A, B hoặc AB	A hoặc B	
	O	A hoặc O	B hoặc O	A hoặc B	O	

Ngoài ra còn có thêm hệ thống phân loại **Rh (Rhesus)**

Căn cứ vào sự khác biệt khi nghiên cứu về sự vận chuyển oxy của hồng cầu thì các hồng cầu có thể mang ở mặt ngoài một protein gọi là Rhesus. Nếu có kháng nguyên D thì là nhóm Rh+ (dương tính), nếu không có là Rh- (âm tính). Các nhóm máu A, B, O, AB mà Rh- thì được gọi là âm tính A-, B-, O-, AB-. Nhóm máu Rh- chỉ chiếm 0,04% dân số thế giới. Đặc điểm của nhóm máu Rh này là chúng chỉ có thể nhận và cho người cùng nhóm máu, đặc biệt phụ nữ có nhóm máu Rh- thì con rất dễ tử vong.

Người có nhóm máu Rh+ chỉ có thể cho người cũng có nhóm máu Rh+ và nhận người có nhóm máu Rh+ hoặc Rh-

Người có nhóm máu Rh- có thể cho người có nhóm máu Rh+ hoặc Rh- nhưng chỉ nhận được người có nhóm máu Rh- mà thôi

Trường hợp người có nhóm máu Rh- được truyền máu Rh+, trong lần đầu tiên sẽ không có bất kỳ phản ứng tức thì nào xảy ra nhưng nếu tiếp tục truyền máu Rh+ lần thứ 2 sẽ gây ra những hậu quả nghiêm trọng do tai biến truyền máu. Tương tự với trường hợp mẹ Rh- sinh con (lần đầu và lần thứ hai trở đi).

Khả năng tương thích: ✓: Có thể cho - nhận.

✗: Không thể cho - nhận.

Bảng khả năng tương thích hồng cầu								
Người nhận	Người cho							
	O-	O+	A-	A+	B-	B+	AB-	AB+
O-	✓	✗	✗	✗	✗	✗	✗	✗
O+	✓	✓	✗	✗	✗	✗	✗	✗
A-	✓	✗	✓	✗	✗	✗	✗	✗
A+	✓	✓	✓	✓	✗	✗	✗	✗
B-	✓	✗	✗	✗	✓	✗	✗	✗
B+	✓	✓	✗	✗	✓	✓	✗	✗
AB-	✓	✗	✓	✗	✓	✗	✓	✗
AB+	✓	✓	✓	✓	✓	✓	✓	✓

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.5 điểm) và xây dựng chương trình thực hiện các yêu cầu sau:

1. Nhập danh sách các nhóm máu của một nhóm người. (1 điểm)
2. Cho một bộ 3 nhóm máu của 3 người là cha, mẹ, con. Hãy kiểm tra và đưa ra kết quả nhóm máu có phù hợp với quy luật di truyền hay không? (1 điểm)
3. Chọn một người X trong danh sách. Hãy liệt kê tất cả các người còn lại trong danh sách có thể cho máu người X này. (1 điểm)

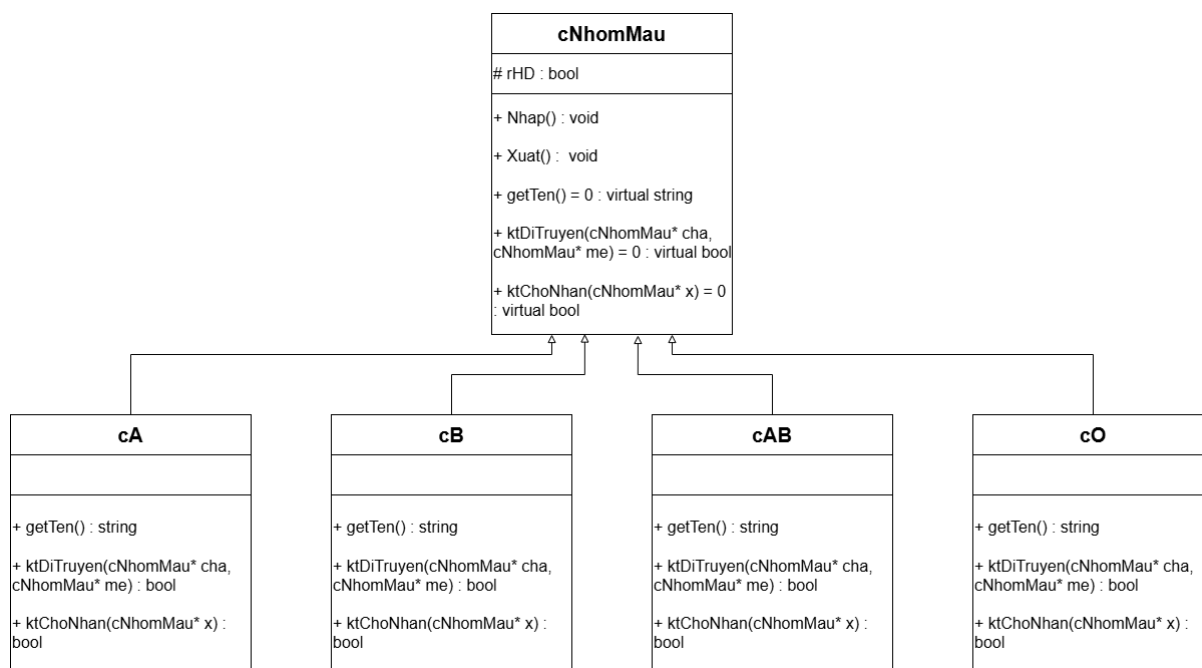
Lưu ý: Trong trường hợp sinh viên không biết về nhóm máu và di truyền học trước đây thì phải đọc kỹ thông tin trên (các thông tin trên đủ để sinh viên thực hiện các yêu cầu của đề thi) và nghiêm túc làm bài. Giám thị coi thi không giải thích gì thêm.

Bài làm:

Tóm tắt yêu cầu chương trình:

1. Nhập danh sách các nhóm máu, mỗi người một nhóm. Một nhóm máu gồm có các loại: A, B, AB, O, Rh+, Rh-.
2. Kiểm tra nhóm máu của cha - mẹ với con.
3. Kiểm tra các nhóm máu có thể cho - nhận.

Sơ đồ lớp:

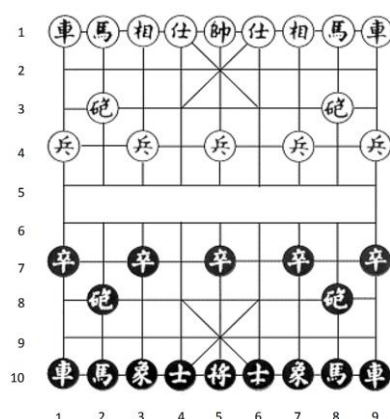


Câu 3 [HK2 14-15].

Xây dựng chương trình mô phỏng trò chơi cờ tướng với các mô tả như sau:

Bàn cờ là một hình chữ nhật do 9 đường dọc và 10 đường ngang cắt nhau vuông góc tại 90 điểm hợp thành. Một khoảng trống gọi là sông (hay hà) nằm ngang giữa bàn cờ, chia bàn cờ thành hai phần đối xứng bằng nhau.

Mỗi bên có một cung Tướng hình vuông (Cung) do 4 ô hợp thành tại các đường dọc 4, 5, 6 kể từ đường ngang cuối của mỗi bên, trong 4 ô này có vẽ hai đường chéo xuyên qua.



Vị trí các quân ban đầu trên bàn cờ

Tướng trắng (1,5)	Tướng đen (10,5)
Sĩ trắng (1,4) và (1,6)	Sĩ đen (10,4) và (10,6)
Tượng trắng (1,3) và (1,7)	Tượng đen (10,3) và (10,7)
Mã trắng (1,2) và (1,8)	Mã đen (10,2) và (10,8)
Xe trắng (1,1) và (1,9)	Xe đen (10,1) và (10,9)
Pháo trắng (3,2) và (3,8)	Pháo đen (8,2) và (8,8)
Tốt trắng (4,1), (4,3), (4,5), (4,7) và (4,9)	Tốt đen (7,1), (7,3), (7,5), (7,7) và (7,9)

Luật chơi: Quân cờ được di chuyển theo luật sau:

1. **Tướng:** Đi từng ô một, đi ngang hoặc dọc. Tướng luôn luôn phải ở trong phạm vi cung và không được ra ngoài. Cung tức là hình vuông 2x2 được đánh dấu bằng đường chéo hình chữ X.
2. **Sĩ:** Đi xéo 1 ô mỗi nước. Sĩ luôn luôn phải ở trong cung như Tướng.
3. **Tượng:** Đi chéo 2 ô (ngang 2 và dọc 2) cho mỗi nước đi. Tượng chỉ được phép ở một bên của bàn cờ, không được di chuyển sang nửa bàn cờ của đối phương. Nước đi của tượng sẽ không hợp lệ khi có một quân cờ nằm chặn giữa đường đi.
4. **Xe:** Đi ngang hay dọc trên bàn cờ miễn là đừng bị quân khác cản đường từ điểm đi đến điểm đến.
5. **Mã:** Đi ngang 2 ô và dọc 1 ô (hay dọc 2 ô và ngang 1 ô) cho mỗi nước đi. Nếu có quân nằm ngay bên cạnh mã và cản đường ngang 2 (hay đường dọc 2), mã bị cản không được đi đường đó.
6. **Pháo:** Đi ngang và dọc giống như xe. Điểm khác biệt là nếu pháo muốn ăn quân, pháo phải nhảy qua đúng 1 quân nào đó. Khi không ăn quân, tất cả những điểm từ chỗ đi đến chỗ đến phải không có quân cản.
7. **Tốt:** Đi một ô mỗi nước. Nếu tốt chưa vượt qua sông, nó chỉ có thể đi thẳng tiến. Khi đã vượt sông rồi, tốt có thể đi ngang 1 nước hay đi thẳng tiến 1 bước mỗi nước.

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.5đ), khai báo và định nghĩa các lớp gồm thuộc tính và phương thức (1.5đ) để thực hiện các yêu cầu sau:

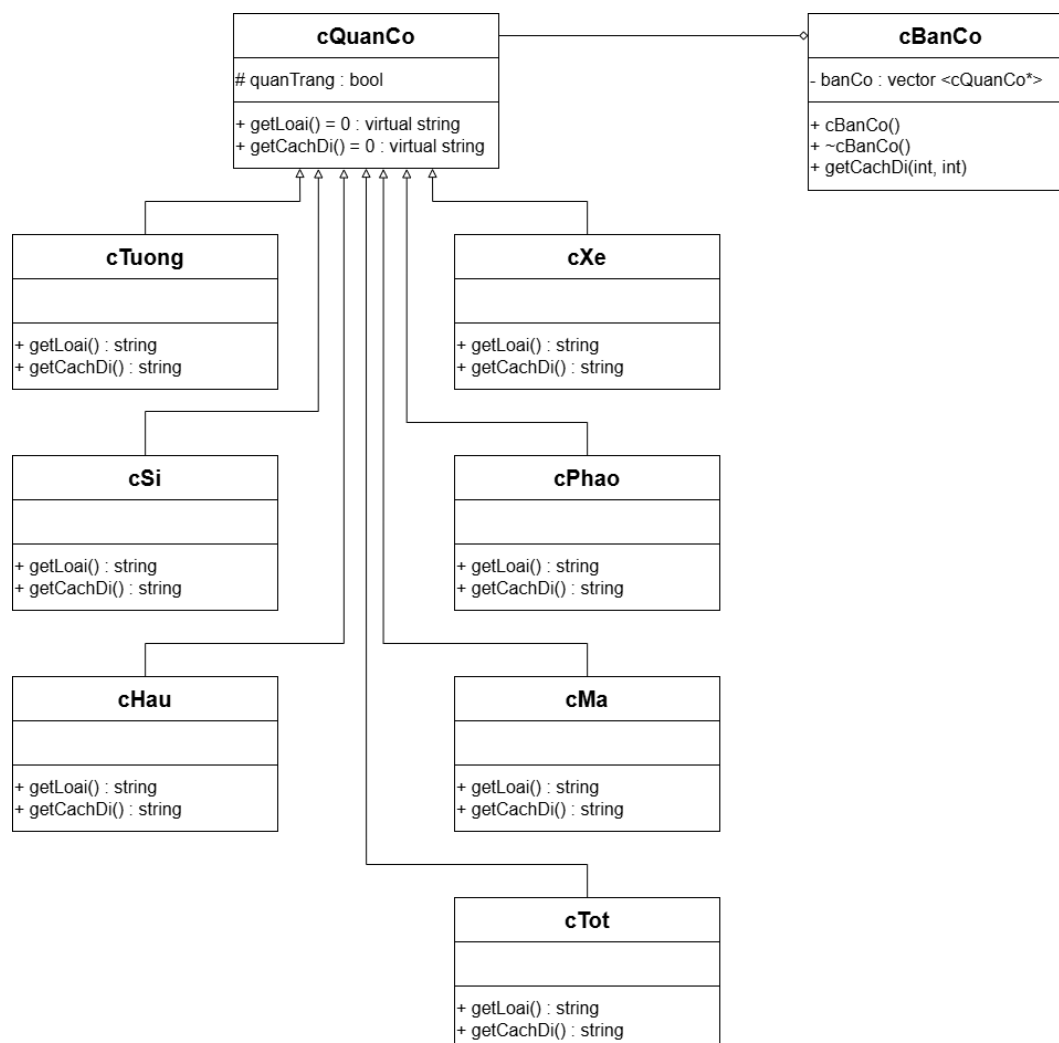
1. Tạo bàn cờ ban đầu (với các mô tả như trên) (1đ)
2. Yêu cầu người dùng chọn một quân cờ, xuất cách đi của quân cờ tương ứng (1đ)

Bài làm:

Tóm tắt yêu cầu chương trình:

- Tạo một bàn cờ là một ma trận 10x9 ô. Mỗi ô chứa 1 quân cờ. Mỗi quân cờ:
 - Có loại trắng hoặc đen.
 - Gồm 7 loại quân cờ khác nhau.
 - Kiểm tra cách đi của quân cờ.

Sơ đồ lớp:



Câu 3 [HK1 22-23].

Một tổ chức chuyên trưng bày, mua bán các sản phẩm liên quan đến nghệ thuật đang muốn xây dựng một ứng dụng để quản lý các hoá đơn khi bán các sản phẩm. Mỗi lần bán sản phẩm thành công, cửa hàng sẽ lưu trữ các hoá đơn chứa thông tin sản phẩm liên quan. Mỗi hoá đơn sẽ có thông tin: mã hoá đơn, thông tin khách hàng, ngày lập hoá đơn, danh sách sản phẩm, tổng giá (tổng giá trị các sản phẩm trong đơn hàng). Tổ chức này hiện tại chỉ kinh doanh 2 loại sản phẩm: tranh ảnh và CD âm nhạc (tương lai có thể thay đổi sản phẩm kinh doanh khác). Mỗi sản phẩm sẽ có thông tin chung cần quản trị: mã số, tiêu đề, giá bán. Ngoài thông tin chung, các sản phẩm tranh ảnh cần thêm thông tin kích thước của bức tranh (chiều rộng, chiều cao), tên hoạ sĩ. Sản phẩm CD âm nhạc sẽ có thêm tên ca sĩ, tên đơn vị sản xuất. Mỗi khách hàng sẽ được lưu trữ các thông tin: mã khách hàng, tên khách hàng, số điện thoại.

Áp dụng tư tưởng lập trình hướng đối tượng (có sử dụng kế thừa, đa hình), anh/chị hãy:

1. (1.5 điểm) Thiết kế và vẽ sơ đồ lớp cho ứng dụng theo bài toán được mô tả.
2. Cài đặt chi tiết theo sơ đồ lớp đã thiết kế và cũng như thành phần cần thiết khác để xây dựng chương trình thực hiện các tính năng sau:
 - a. (1.5 điểm) Nhập và xuất danh sách các hoá đơn bán hàng .
 - b. (1 điểm) Tính tổng thu nhập của cửa hàng.
 - c. (1 điểm) Tìm các khách hàng mua nhiều nhất ở cửa hàng (dựa vào tổng giá trị các hoá đơn khách hàng đã mua).

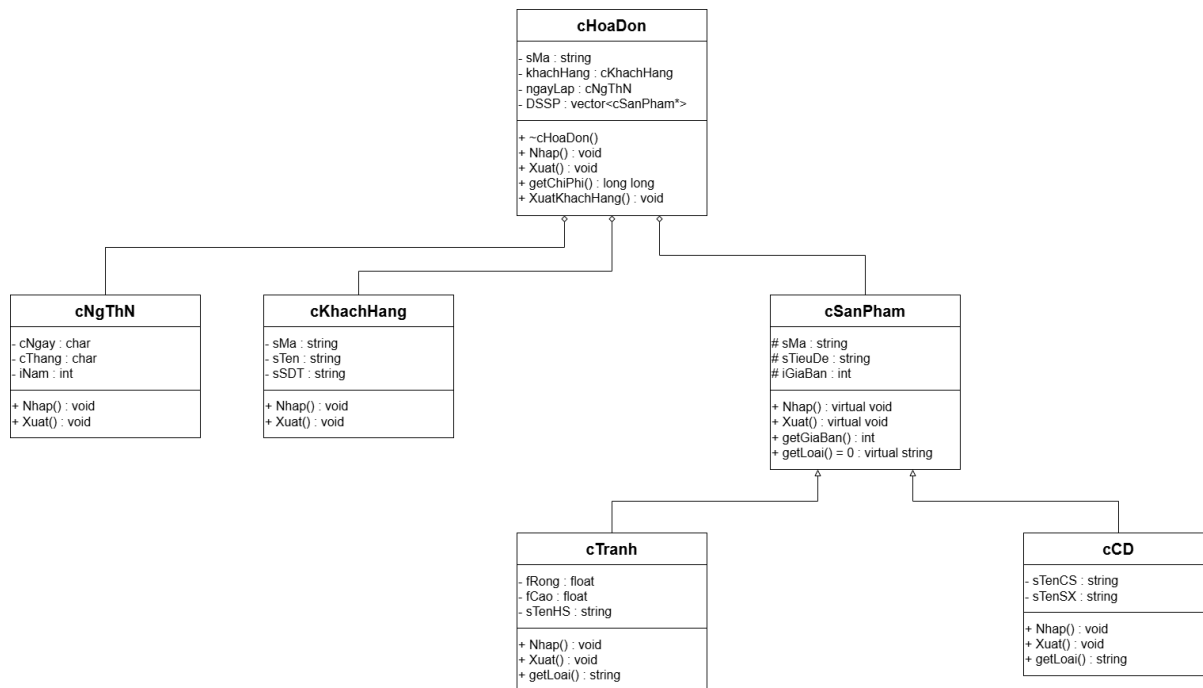
Bài làm:

Tóm tắt yêu cầu chương trình:

4. Nhập / xuất danh sách hóa đơn bán hàng. Mỗi hóa đơn gồm có:
 - a. Mã hoá đơn.
 - b. Thông tin khách hàng. Gồm có:
 - i. Mã khách hàng.
 - ii. Tên khách hàng.
 - iii. Số điện thoại.
 - c. Ngày lập hoá đơn.
 - d. Danh sách sản phẩm.
Mỗi sản phẩm gồm có:
 - i. Mã số.
 - ii. Tiêu đề.
 - iii. Giá bán.
- Có 2 loại sản phẩm :
 - iv. Tranh ảnh: Có thêm thông tin:
 1. Kích thước của bức tranh (chiều rộng, chiều cao).
 2. Tên hoạ sĩ.

- v. CD âm nhạc: Có thêm thông tin:
1. Tên ca sĩ.
 2. Tên đơn vị sản xuất.
- e. Tổng giá (tổng giá trị các sản phẩm trong đơn hàng).
5. Tính tổng thu nhập.
 6. Tìm khách hàng có tổng giá trị hóa đơn cao nhất.

Sơ đồ lớp:



Khai báo các lớp:

```

cSanPham.h
#pragma once
#include <string>

using namespace std;

class cSanPham {
protected:
    string sMa;
    string sTieuDe;
    int iGiaBan;
public:
    virtual void Nhap();
    virtual void Xuat();
    int getGiaBan();
    virtual string getLoai() = 0;
};
  
```

```

cTranh.h
#pragma once
#include "cSanPham.h"

class cTranh : public cSanPham {
private:
    float fRong;
    float fCao;
    string sTenHS;
public:
    void Nhap();
    void Xuat();
    string getLoai();
};
  
```

```

cHoaDon.h
#pragma once
#include "cKhachHang.h"
#include "cNgThN.h"
#include "cSanPham.h"

using namespace std;

class cHoaDon {
private:
    string sMa;
    cKhachHang khachHang;
    cNgThN ngayLap;
    vector <cSanPham*> DSSP;
public:
    ~cHoaDon();
    void Nhap();
    void Xuat();
    long long getChiPhi();
    void XuatKhachHang();
};

```

```

cCD.h
#pragma once
#include "cSanPham.h"

class cCD : public cSanPham {
private:
    string sTenCS;
    string sTenSX;
public:
    void Nhap();
    void Xuat();
    string getLoai();
};

```

```

cKhachHang.h
#pragma once
#include <string>

using namespace std;

class cKhachHang {
private:
    string sMa;
    string sTen;
    string sSDT;
    void Nhap();
    void Xuat();
};

```

```

cNgThN.h
#pragma once

class cNgThN {
private:
    char cNgay;
    char cThang;
    int iNam;
public:
    void Nhap();
    void Xuat();
};

```

Định nghĩa các phương thức:

```
cSanPham.cpp
#include "cSanPham.h"

void cSanPham::Nhap() {
    cout << "- Ma: ";
    cin >> sMa;
    cout << "- Tieu de: ";
    cin.ignore();
    getline(cin, sTieuDe);
    cout << "- Gia ban: ";
    cin >> iGiaBan;
}

void cSanPham::Xuat() {
    cout << "- Ma: " << sMa << endl;
    cout << "- Tieu de: " << sTieuDe << endl;
    cout << "- Gia ban: " << iGiaBan << endl;
}

int cSanPham::getGiaBan() {
    return iGiaBan;
}
```

```
cTranh.cpp
#include "cSanPham.h"
#include "cTranh.h"

void cTranh::Nhap() {
    cSanPham::Nhap();
    cout << "- Chieu rong: ";
    cin >> fRong;
    cout << "- Chieu cao: ";
    cin >> fCao;
    cout << "- Ten hoa si: ";
    cin.ignore();
    getline(cin, sTenHS);
}

void cTranh::Xuat() {
    cout << "- Loai: " << getLoai() <<
endl;
    cSanPham::Xuat();
    cout << "- Chieu rong: " << fRong << endl;
    cout << "- Chieu cao: " << fCao << endl;
    cout << "- Ten hoa si: " << sTenHS <<
endl;
}

string cTranh::getLoai() {
    return "Tranh anh";
}
```

```
cCD.cpp
#include "cSanPham.h"
#include "cCD.h"

void cCD::Nhap() {
    cSanPham::Nhap();
    cout << "- Ten ca si: ";
    cin.ignore();
    getline(cin, sTenCS);
    cout << "- Ten nha san xuat: ";
    getline(cin, sTenSX);
}

void cCD::Xuat() {
    cout << "- Loai: " << getLoai() <<
endl;
    cSanPham::Xuat();
    cout << "- Ten ca si: " << sTenCS
<< endl;
    cout << "- Ten nha san xuat: " <<
sTenSX << endl;
}
```

```

cHoaDon.cpp
#include "cHoaDon.h"

cHoaDon::~cHoaDon() {
    for (auto& sp : DSSP) {
        delete sp;
        sp = nullptr;
    }
}

void cHoaDon::Nhap() {
    cout << "- Ma hoa don: ";
    cin >> sMa;

    khachHang.Nhap();

    cout << "- Ngay lap hoa don:\n";
    ngayLap.Nhap();

    cout << "- So luong sp: ";
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        cSanPham* tam;

        cout << "- Loai SP (0 = tranh,
1 = CD): ";

        bool loai;
        cin >> loai;

        if (loai)
            tam = new cTranh;
        else
            tam = new cCD;

        tam->Nhap();
        DSSP.push_back(tam);
    }
}

void cHoaDon::Xuat() {
    cout << "- Ma hoa don: " << sMa << endl;

    khachHang.Xuat();

    cout << "- Ngay lap: ";
    ngayLap.Xuat();

    cout << "- Danh sach san pham: " << endl;

```

```

string cCD::getLoai() {
    return "CD";
}

```

```

cKhachHang.cpp
#include "cKhachHang.h"

void cKhachHang::Nhap() {
    cout << "- Ma KH: ";
    cin >> sMa;
    cout << "- Ten KH: ";
    cin.ignore();
    getline(cin, sTen);
    cout << "- SDT KH: ";
    cin >> sSDT;
}

void cKhachHang::Xuat() {
    cout << "- Ma KH: " << sMa << endl;
    cout << "- Ten KH: " << sTen << endl;
    cout << "- SDT KH: " << sSDT << endl;
}

```

```

cNgThN.cpp
#include "cNgThN.h"

void cNgThN::Nhap() {
    cout << "- Ngay: ";
    int tam;
    cin >> tam;
    cNgay = tam;
    cout << "- Thang: ";
    cin >> tam;
    cThang = tam;
    cout << "- Nam: ";
    cin >> iNam;
}

void cNgThN::Xuat() {
    cout << (int)cNgay << '/' <<
(int)cThang << '/' << iNam;
}

```

```

        for (auto sp : DSSP)
            sp->Xuat();
    }

    long long cHoaDon::getChiPhi() {
        long long tong = 0;
        for (auto sp : DSSP)
            tong += sp->getGiaBan();
        return tong;
    }

    void cHoaDon::XuatKhachHang() {
        khachHang.Xuat();
    }

```

Thân chương trình:

```

Bai03.cpp
#include <iostream>
#include <vector>
#include "cHoaDon.h"

using namespace std;

int main() {

    // Cau 1

    int n;
    cin >> n;

    vector <cHoaDon> DSHD;
    for (int i = 0; i < n; i++) {
        cHoaDon tam;
        tam.Nhap();
        DSHD.push_back(tam);
    }

    for (auto hd : DSHD)
        hd.Xuat();

    // Cau 2

    long long tongThuNhap = 0;
    for (auto hd : DSHD)
        tongThuNhap += hd.getChiPhi();
    cout << "Tong thu nhap: " << tongThuNhap << endl;

    // Cau 3

    long long maxChiPhi = 0;

    for (auto hd : DSHD) {
        long long chiPhi = hd.getChiPhi();
        if (chiPhi > maxChiPhi)

```

```
        maxChiPhi = chiPhi;
    }

    for (auto hd : DSHD)
        if (hd.getChiPhi() == maxChiPhi)
            hd.Xuat();

    return 0;
}
```

Câu 3 [HK2 19-20].

Trước hết phải khẳng định, đất đai là nguồn tài nguyên vô cùng quý giá, là tài sản quan trọng của quốc gia, là tư liệu sản xuất,... Đặc biệt, đất đai là điều kiện cần cho mọi hoạt động sản xuất và đời sống. Ở nước ta, khi còn nhiều người sống nhờ vào nông nghiệp, thì đất đai càng trở thành nguồn lực rất quan trọng.

Muốn phát huy tác dụng của nguồn lực đất đai, ngoài việc bảo vệ quỹ đất của quốc gia, còn phải quản lý đất đai hợp lý, nâng cao hiệu quả sử dụng đất sao cho vừa bảo đảm được lợi ích trước mắt, vừa tạo điều kiện sử dụng đất hiệu quả lâu dài để phát triển bền vững đất nước. Hiện nay, ở Việt Nam đất đai được phân chia thành nhiều loại, tuy nhiên chúng ta quan tâm 2 loại chính sau:

- Đất nông nghiệp.
- Đất phi nông nghiệp (đất ở).

Quan điểm nhất quán của Đảng, Nhà nước và nhân dân ta đã được xác định từ năm 1980 đến nay là đất đai thuộc sở hữu toàn dân, do Nhà nước đại diện chủ sở hữu và thống nhất quản lý. Để góp phần nâng cao hiệu quả quản lý nhà nước về đất đai, mỗi thửa đất được nhà nước quản lý và **cấp quyền sử dụng** cho một hoặc nhiều người dân (nhà nước cho phép nhiều người dân có thể đồng sở hữu quyền sử dụng đất) có nhu cầu sử dụng (**Giấy chứng nhận quyền sử dụng đất** hay còn được gọi là **Sổ hồng**).

- Với các **thửa đất nông nghiệp**, thông tin cần quản lý gồm: sổ giấy chứng nhận (chuỗi), người sở hữu quyền sử dụng đất (gồm họ và tên, năm sinh, CMND, địa chỉ thường trú), số thửa đất, số tờ bản đồ, địa chỉ thửa đất, diện tích (m^2), thời hạn sử dụng (được sử dụng đến năm nào), ngày cấp, đơn giá thuế phải đóng cho nhà nước hàng năm/ $1m^2$.
- Với các **thửa đất phi nông nghiệp (đất ở)**, thông tin cần quản lý gồm: sổ giấy chứng nhận (chuỗi), người sở hữu quyền sử dụng đất (gồm họ và tên, năm sinh, CMND, địa chỉ thường trú), số thửa đất, số tờ bản đồ, địa chỉ thửa đất, diện tích (m^2), ngày cấp, đơn giá thuế phải đóng cho nhà nước hàng năm/ $1m^2$.

Áp dụng kiến thức lập trình hướng đối tượng (kế thừa, đa hình) thiết kế sơ đồ chi tiết các lớp đối tượng (1.0đ) và khai báo các lớp (1.0đ) để xây dựng chương trình thực hiện các yêu cầu sau:

1. Tạo danh sách các giấy chứng nhận quyền sử dụng đất mà nhà nước đã cấp cho người dân (1đ)
2. Tính tiền thuế mà người sử dụng đất phải đóng cho nhà nước và cho biết thửa đất nào (thông tin thửa đất) có tiền thuế phải đóng nhiều nhất. (1đ)
3. Xuất ra màn hình thông tin **các thửa đất nông nghiệp** đã hết thời hạn sử dụng (năm sử dụng < năm hiện tại). (1đ)

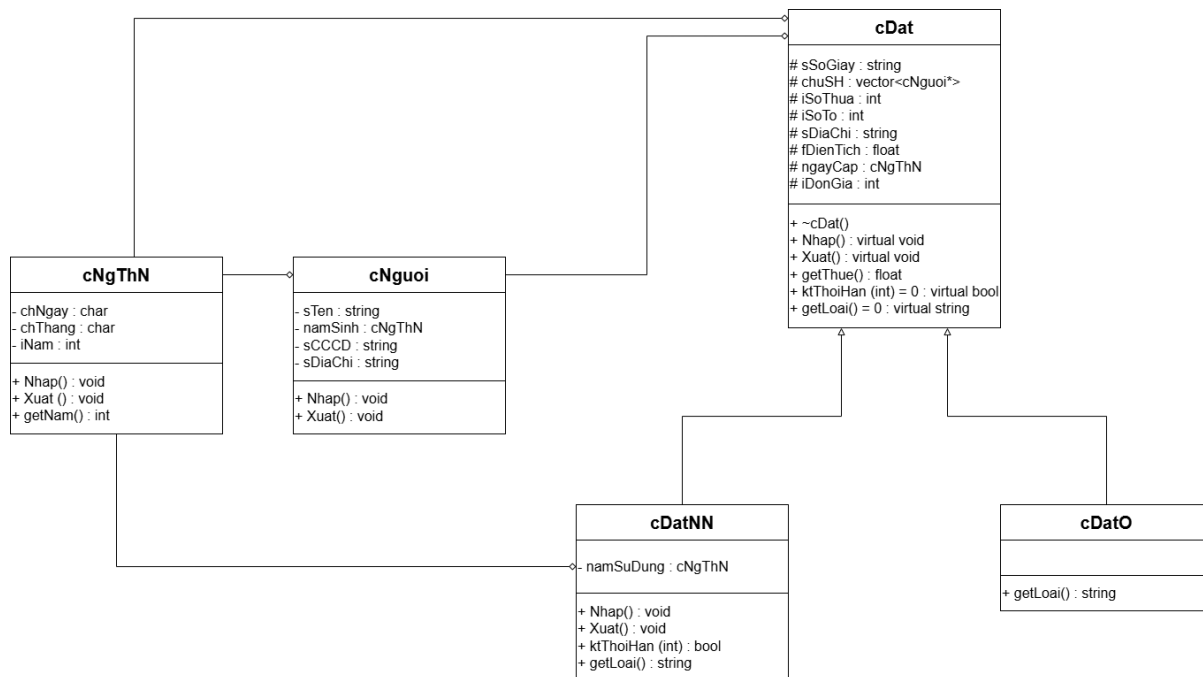
Lưu ý: Các thông tin trong đề chỉ mô phỏng các thông tin với mục tiêu để sinh viên vận dụng kiến thức lập trình hướng đối tượng. Do vậy, các thông tin trong đề KHÔNG nhất thiết phải đúng hoặc khớp với các thông tin hiện tại trong thế giới thực. Sinh viên cần bám sát các mô tả trong đề thi để làm bài.

Bài làm:

Tóm tắt yêu cầu chương trình:

1. Nhập / Xuất danh sách các giấy chứng nhận quyền sử dụng đất:
 - a. Danh sách thuộc tính chung:
 - i. Số giấy chứng nhận (chuỗi).
 - ii. Người sở hữu quyền sử dụng đất (có thể có nhiều người):
 1. Họ và tên.
 2. Năm sinh.
 3. CMND.
 4. Địa chỉ thường trú.
 - iii. Số thửa đất.
 - iv. Số tờ bản đồ.
 - v. Địa chỉ thửa đất.
 - vi. Diện tích (m²).
 - vii. Ngày cấp.
 - viii. Đơn giá thuế phải đóng cho nhà nước hàng năm/1m².
 - b. Riêng với đất nông nghiệp có thêm: Thời hạn sử dụng (được sử dụng đến năm nào).
 - c. Riêng với đất ở: Không có thêm thông tin.
2. Tính tiền thuế.
3. Kiểm tra hạn sử dụng.

Sơ đồ lớp:



Khai báo các lớp:

```

cNgThN.h
#pragma once

class cNgThN {
private:
    char chNgay;
    char chThang;
    int iNam;
public:
    void Nhap();
    void Xuat();
    int getNam();
};
    
```

```

cNguoi.h
#pragma once
#include <string>
#include "cNgThN.h"

using namespace std;

class cNguoi {
private:
    string sTen;
    cNgThN namSinh;
    string sCCCD;
    string sDiaChi;
public:
    void Nhap();
    void Xuat();
};
    
```

```

cDat.h
#pragma once
#include <vector>
#include "cNguoi.h"
#include "cNgThN.h"

using namespace std;

class cDat {
private:
    string sSoGiai;
    
```

```

cDatNN.h
#pragma once
#include "cDat.h"

class cDatNN : public cDat {
private:
    cNgThN namSuDung;
public:
    void Nhap();
    void Xuat();
    bool ktThoiHan(int);
    
```

```

vector <cNguoi*> chuSH;
int iSoThua;
int iSoTo;
string sDiaChi;
float fDienTich;
cNgThN ngayCap;
int iDonGia;
public:
~cDat();
virtual void Nhap();
virtual void Xuat();
float getThue();
virtual bool ktThoiHan(int) = 0;
virtual string getLoai() = 0;
};

```

```

string getLoai();
};

```

```

cDatO.h
#pragma once
#include "cDat.h"

class cDatO : public cDat {
public:
    string getLoai();
};

```

Định nghĩa các phương thức:

```

cNgThN.cpp
#include <iostream>
#include "cNgThN.h"

using namespace std;

void cNgThN::Nhap() {
    cout << "- Ngày: ";
    int tam;
    cin >> tam;
    chNgay = tam;
    cout << "- Tháng: ";
    cin >> tam;
    chThang = tam;
    cout << "- Nam: ";
    cin >> iNam;
}

void cNgThN::Xuat() {
    cout << (int)chNgay << '/' <<
(int)chThang << '/' << iNam;
}

int cNgThN::getNam() {
    return iNam;
}

```

```

cNguoi.cpp
#include <iostream>
#include "cNguoi.h"

void cNguoi::Nhap() {
    cout << "- Ho ten: ";
    cin.ignore();
    getline(cin, sTen);
    cout << "- Nam sinh: ";
    namSinh.Nhap();
    cout << "- So CCCD: ";
    cin >> sCCCD;
    cout << "- Dia chi: ";
    cin.ignore();
    getline(cin, sDiaChi);
}

void cNguoi::Xuat() {
    cout << "- Ho ten: " << sTen <<
endl;
    cout << "- Nam sinh: ";
    namSinh.Xuat(); cout << endl;
    cout << "- CCCD: " << sCCCD <<
endl;
    cout << "- Dia chi: " << sDiaChi <<
endl;
}

```

```

cDat.cpp
#include <iostream>
#include "cDat.h"

cDat::~cDat() {
    for (auto& ng : chuSH) {
        delete ng;
    }
}

```

```

cDatNN.cpp
#include <iostream>
#include "cDatNN.h"

void cDatNN::Nhap() {
    cDat::Nhap();
    cout << "- Nam su dung: ";
    namSuDung.Nhap(); cout << endl;
}

```

```

void cDat::Nhap() {
    cout << "- So giay: ";
    cin >> iSoGiay;

    cout << "- So chu so huu: ";
    int n;
    cin >> n;
    chuSH.clear();
    for (int i = 0; i < n; ++i) {
        auto ng = new cNguoi();
        ng->Nhap();
        chuSH.push_back(ng);
    }

    cout << "- So thua: "; cin >>
iSoThua;
    cout << "- So to: "; cin >> iSoTo;
    cout << "- Dia chi: ";
    cin.ignore(); getline(cin, sDiaChi);
    cout << "- Dien tich: "; cin >>
fDienTich;
    cout << "- Ngay cap: ";
    ngayCap.Nhap(); cout << endl;
    cout << "- Don gia: "; cin >>
iDonGia;
}

void cDat::Xuat() {
    cout << "So giay: " << iSoGiay <<
endl;
    cout << "Chu so huu:\n";
    for (auto ng : chuSH) ng->Xuat();
    cout << "So thua: " << iSoThua <<
endl;
    cout << "So to: " << iSoTo << endl;
    cout << "Dia chi: " << sDiaChi <<
endl;
    cout << "Dien tich: " << fDienTich
<< endl;
    cout << "Ngay cap: ";
    ngayCap.Xuat(); cout << endl;
    cout << "Don gia: " << iDonGia <<
endl;
}

float cDat::getThue() {
    return fDienTich * iDonGia;
}

```

```

void cDatNN::Xuat() {
    cDat::Xuat();
    cout << "Nam su dung: ";
    namSuDung.Xuat(); cout << endl;
}

bool cDatNN::ktThoiHan(int namHienTai)
{
    return namSuDung.getNam() <
namHienTai;
}

string cDatNN::getLoai() {
    return "NN";
}

```

```

cDat0.cpp
#include <iostream>
#include "cDat0.h"

string cDat0::getLoai() {
    return "0";
}

```

Thân chương trình:

```
Bai03.cpp
#include <iostream>
#include <vector>
#include "cDat.h"
#include "cDatNN.h"
#include "cDatO.h"

using namespace std;

int main() {

    // Cau 1

    int n;
    cin >> n;

    vector <cDat*> dats(n);

    for (auto& dat : dats) {
        cout << "- Loai dat (1 = NN, 0 = O) : ";
        bool loai;
        cin >> loai;

        if (loai)
            dat = new cDatNN;
        else
            dat = new cDatO;

        dat->Nhap();
    }

    // Cau 2

    float maxThue = 0;

    for (auto dat : dats)
        if (dat->getThue() > maxThue)
            maxThue = dat->getThue();

    for (auto dat : dats)
        if (dat->getThue() == maxThue)
            dat->Xuat();

    // Cau 3

    for (auto dat : dats)
        if (dat->getLoai() == "NN" && dat->ktThoiHan(2025))
            dat->Xuat();

    return 0;
}
```